Licencia

CC BY-NC-SA 4.0

Atribución-NoComercial-CompartirIgual 4.0 Internacional

CC BY NC SA

Este documento, *"Tratado de Algoritmos y Datos III"*, fue creado por **Nahuel Villafañe** y se distribuye bajo una licencia **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional**.

Puede **compartir** (copiar y redistribuir el material en cualquier medio o formato) y **adaptar** (remezclar, transformar y construir a partir del material), bajo las siguientes condiciones:

- **Atribución (BY)** — Debe dar crédito de manera adecuada, brindar un enlace a la licencia e indicar si se han realizado cambios.
- **No Comercial (NC)** — No puede hacer uso del material con fines comerciales.
- **Compartir Igual (SA)** — Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia que el original.

Cita sugerida:
Villafañe, N. (2026). *Tratado de Algoritmos y Datos III*. Buenos Aires, Argentina.

Contacto del autor: @clasesconnahu (Instagram)

Licencia completa disponible en: https://creativecommons.org/licenses/by-nc-sa/4.0/

TRATADO DE ALGORITMOS Y DATOS III

Nahuel Villafañe

Año 2026

Resumen de la Unidad 118
Bibliografía de la Unidad 118

Unidad X: Algoritmos sobre Números 119
Objetivos de la Unidad 119
Introducción a la Aritmética Modular 119
Definiciones Fundamentales 119
Propiedades de la Aritmética Modular 119
Inverso Modular 120
Algoritmo de Euclides para el Máximo Común Divisor 120
Descripción 120
Pseudocódigo 120
Ejemplo 120
Análisis de Complejidad 120
Demostración de Correctitud 120
Algoritmo de Euclides Extendido 121
Descripción 121
Pseudocódigo 121
Ejemplo 121
Aplicaciones del Euclides Extendido 121
Cálculo del Inverso Modular 122
Exponenciación Modular 122
Descripción 122
Algoritmo de Exponenciación Rápida 122
Ejemplo 122
Análisis de Complejidad 122
Introducción a la Criptografía RSA 123
Fundamentos Matemáticos 123
Generación de Claves 123
Cifrado y Descifrado 123
Ejemplo Simple 123
Implementación Básica de RSA 124
Seguridad de RSA 124
Algoritmos Adicionales sobre Números 124
Criba de Eratóstenes 124
Test de Primalidad de Miller-Rabin 124
Ejercicios Resueltos (20 Ejercicios) 125
Ejercicios Propuestos (20 Ejercicios) 126
Resumen de la Unidad 127
Bibliografía de la Unidad 127

Unidad XI: Aritmética Modular **129**
Objetivos de la Unidad 129
Introducción a la Aritmética Modular 129
Definición de Congruencia 129
Propiedades Básicas 130
Operaciones en Aritmética Modular 130
Conjunto de Residuos 130
Operaciones Aritméticas Modulares 130
Suma Modular 130
Resta Modular 130
Multiplicación Modular 130
Exponenciación Modular 131
Inverso Modular 131
Definición 131
Existencia del Inverso Modular 131
Cálculo del Inverso Modular con Euclides Extendido 131
Ejemplos de Inversos Modulares 131
Teorema Chino del Resto (CRT) 132
Enunciado del Teorema 132
Algoritmo de Solución 132
Ejemplo 132
Teorema de Fermat y Euler 132
Teorema de Fermat 132
Función de Euler 133
Propiedades de 133
Teorema de Euler 133
Aplicaciones de la Aritmética Modular 133
Verificación de Números Primos 133
Criptografía RSA 133
Códigos de Verificación 133
Hashing 133
Ejercicios Resueltos (20 Ejercicios) 134
Ejercicios Propuestos (20 Ejercicios) 134
Resumen de la Unidad 135
Bibliografía de la Unidad 135

Unidad XII: Algoritmo Extendido de Euclides **137**
Objetivos de la Unidad 137
Algoritmo de Euclides Estándar (Repaso) 137
Definición del MCD 137
Propiedad Fundamental 137

Algoritmo de Euclides 138
Ejemplo Visual 138
Algoritmo de Euclides Extendido 138
Motivación 138
Fundamento Matemático 138
Derivación del Algoritmo 138
Implementación Recursiva 139
Ejemplo Detallado 139
Implementación Iterativa 139
Aplicaciones del Algoritmo Extendido 140
Cálculo del Inverso Modular 140
Ecuaciones Diofánticas Lineales 140
Aplicación en RSA 140
Fracciones Continuas 141
Demostración de Correctitud 141
Análisis de Complejidad 141
Comparación de Implementaciones 141
Ejercicios Resueltos (20 Ejercicios) 142
Ejercicios Propuestos (20 Ejercicios) 143
Resumen de la Unidad 144
Bibliografía de la Unidad 144

Unidad I: Complejidad Computacional y NP-Completitud

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la diferencia fundamental entre problemas tratables e intratables.
- Definir formalmente las clases de complejidad P, NP, NP-completo y NP-hard.
- Realizar reducciones polinomiales entre problemas.
- Demostrar que un problema es NP-completo.
- Identificar problemas NP-completos clásicos en aplicaciones reales.
- Aplicar estrategias para enfrentar problemas NP-completos en la práctica.

Introducción: ¿Qué hace que un problema sea "difícil"?

La Pregunta Fundamental

A lo largo de Algoritmos y Datos II, hemos estudiado algoritmos eficientes que resuelven problemas en tiempo polinomial: $O(n)$, $O(n \log n)$, $O(n^2)$, etc. Pero existen problemas para los cuales no se conoce ningún algoritmo de tiempo polinomial. La pregunta central de la teoría de la complejidad computacional es:

¿P = NP?

Esta es una de las preguntas abiertas más importantes de la computación, con un premio de $1,000,000 del Instituto Clay de Matemáticas.

Problemas Fáciles vs. Difíciles

Problemas "Fáciles"	**Problemas "Difíciles"**
Ordenar una lista	Problema del viajante (TSP)
Buscar en un arreglo	Problema de la mochila (0/1)
Caminos más cortos	Coloreo de grafos
Árbol de cubrimiento mínimo	Clique máximo
Multiplicación de matrices	Programación lineal entera

Crecimiento de Funciones

Para entender la diferencia, recordemos el crecimiento de funciones:

n	$\log n$	n	n^2	2^n	$n!$
10	3.3	10	100	1,024	3,628,800
20	4.3	20	400	1,048,576	2.4×10^{18}
30	4.9	30	900	10^9	2.6×10^{32}
50	5.6	50	2,500	10^{15}	3.0×10^{64}
100	6.6	100	10,000	10^{30}	9.3×10^{157}

Mientras que n^2 es manejable para $n = 100$, 2^n es imposible incluso para $n = 100$ (requeriría más tiempo que la edad del universo).

Clase P: Problemas Tratables

Definición Formal

La clase **P** (Polynomial time) es el conjunto de todos los problemas de decisión que pueden ser resueltos por un algoritmo determinista en tiempo polinomial en el tamaño de la entrada.

Características de P

- Existe un algoritmo que resuelve el problema en $O(n^k)$ para alguna constante k.
- El algoritmo es determinista (no usa aleatoriedad).
- El tiempo de ejecución es "razonable" para entradas de tamaño práctico.

Ejemplos de Problemas en P

- ORDENAMIENTO: Dado un arreglo de n números, ¿está ordenado?
- CONEXIDAD: Dado un grafo, ¿es conexo?
- CAMINO MÁS CORTO: Dado un grafo, dos vértices s y t, y un entero k, ¿existe un camino de s a t con longitud $\leq k$?
- ÁRBOL DE CUBRIMIENTO: Dado un grafo y un entero k, ¿existe un árbol de cubrimiento de peso total $\leq k$?

Clase NP: Problemas Verificables Rápidamente

Definición Formal

La clase **NP** (Nondeterministic Polynomial time) es el conjunto de problemas de decisión para los cuales existe un **certificado** que puede ser verificado en tiempo polinomial por un algoritmo determinista.

Interpretación

Un problema está en NP si:

1. Dada una solución candidata (certificado), podemos verificar su corrección rápidamente.
2. No necesariamente podemos **encontrar** la solución rápidamente.

Ejemplo: SAT (Satisfacibilidad Booleana)

- **Problema:** Dada una fórmula booleana (ej. $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$), ¿existe una asignación de valores verdadero/falso que haga la fórmula verdadera?
- **Certificado:** Una asignación de valores a las variables.
- **Verificación:** Sustituir los valores y evaluar la fórmula. Esto toma tiempo lineal en el tamaño de la fórmula.
- **Complejidad de encontrar la solución:** Se desconoce si es polinomial.

Ejemplos de Problemas en NP

- SAT: Dada una fórmula booleana, ¿es satisfacible?
- CLIQUE: Dado un grafo y un entero k, ¿existe un subconjunto de k vértices completamente conectado?
- VERTEX COVER: Dado un grafo y un entero k, ¿existe un conjunto de k vértices que cubra todas las aristas?
- TSP: Dado un conjunto de ciudades y distancias, ¿existe un recorrido de longitud $\leq k$ que visite todas las ciudades?
- MOCHILA (0/1): Dados objetos con peso y valor, y una capacidad, ¿existe un subconjunto con valor $\geq k$?

Relación P vs. NP

$P \subseteq NP$ Si un problema se puede resolver en tiempo polinomial, podemos verificar cualquier solución simplemente resolviendo el problema. El certificado puede ser vacío.

La pregunta abierta es si $P = NP$ o $P \subset NP$. La mayoría de los expertos cree que $P \neq NP$.

[scale=0.8] (0,0) circle (2cm); (0,0) circle (1.2cm); at (0,1.6) NP; at (0,0.6) P;

NP-Completitud: Los Problemas Más Difíciles de NP

Definición de Reducción Polinomial

Un problema A se reduce polinomialmente a un problema B (escrito $A \leq_p B$) si existe una función f computable en tiempo polinomial que transforma instancias de A en instancias de B tal que:

$$xesun"sí"deA \iff f(x)esun"sí"deB$$

(A) at (0,0) Instancia de A; (B) at (4,0) Instancia de B; [-¿] (A) – node[above] f (polinomial) (B); [-¿] (A) – node[below] sí/no (0,-1); [-¿] (B) – node[below] sí/no (4,-1);

Propiedades de las Reducciones

- **Transitividad:** Si $A \leq_p B$ y $B \leq_p C$, entonces $A \leq_p C$.
- Si $A \leq_p B$ y $B \in P$, entonces $A \in P$.
- Si $A \leq_p B$ y A es NP-completo, entonces B es NP-hard.

Definición de NP-Completo

Un problema L es **NP-completo** si:

1. $L \in NP$ (está en NP)
2. Para todo $L' \in NP$, $L' \leq_p L$ (es NP-difícil)

[scale=0.8] (0,0) circle (2.5cm); (0,0) circle (1.5cm); at (0,2) NP; at (0,1) P; at (0.8,1.8) NP-completo;

Propiedades de NP-Completo

- Si cualquier problema NP-completo está en P, entonces $P = NP$.
- Si cualquier problema NP-completo no está en P, entonces todos los NP-completos no están en P.
- Los problemas NP-completos son "los más difíciles" de NP.

NP-Hard

Un problema L es **NP-hard** si para todo $L' \in NP$, $L' \leq_p L$. No se requiere que $L \in NP$.

Los problemas NP-hard pueden ser incluso más difíciles que los NP-completos (pueden no ser de decisión o no estar en NP).

El Primer Problema NP-Completo: SAT

Teorema de Cook-Levin

[Cook-Levin, 1971] El problema SAT (Satisfacibilidad Booleana) es NP-completo.

Demostración (Esbozo)

1. **SAT está en NP:** Dada una asignación, verificarla es lineal.
2. **SAT es NP-difícil:** Se puede simular cualquier máquina de Turing no determinista con una fórmula booleana.

Consecuencias del Teorema de Cook-Levin

- Proporciona un "primer" problema NP-completo.
- Para demostrar que otro problema X es NP-completo, basta con mostrar:
  1. $X \in NP$
  2. $SAT \leq_p X$ (o cualquier otro NP-completo conocido)

Problemas NP-Completos Clásicos

SAT y sus Variantes

- **3-SAT:** SAT con cláusulas de exactamente 3 literales. También NP-completo.
- **Circuit-SAT:** Dado un circuito booleano, ¿existe una entrada que haga la salida verdadera?

Problemas de Grafos

CLIQUE

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Existe un subconjunto $C \subseteq V$ de tamaño k tal que todos los vértices en C están conectados entre sí?

[scale=0.6] [circle, draw] (1) at (0,1) 1; [circle, draw] (2) at (2,1) 2; [circle, draw] (3) at (1,2) 3; [circle, draw] (4) at (0.5,0) 4; [circle, draw] (5) at (1.5,0) 5;
(1) – (2); (1) – (3); (2) – (3); (1) – (4); (2) – (5);

Clique de tamaño 3: $\{1, 2, 3\}$.

VERTEX COVER

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Existe un subconjunto $C \subseteq V$ de tamaño k tal que cada arista tiene al menos un extremo en C?

[scale=0.6] [circle, draw, fill=gray] (a) at (0,0) a; [circle, draw] (b) at (2,0) b; [circle, draw] (c) at (1,1.5) c; [circle, draw] (d) at (3,1.5) d; (a) – (b); (a) – (c); (b) – (c); (b) – (d); (c) – (d);

Vertex cover de tamaño 2: $\{a, b\}$ cubre todas las aristas.

HAMILTONIAN CYCLE

- **Entrada:** Grafo $G = (V, E)$
- **Pregunta:** ¿Existe un ciclo que visite cada vértice exactamente una vez?

GRAPH COLORING

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Se pueden colorear los vértices con k colores de modo que no haya dos vértices adyacentes con el mismo color?

Problemas de Números

SUBSET SUM

- **Entrada:** Conjunto de números enteros $\{a_1, \ldots, a_n\}$, entero t
- **Pregunta:** ¿Existe un subconjunto cuya suma sea exactamente t?

PARTITION

- **Entrada:** Conjunto de números enteros
- **Pregunta:** ¿Se pueden dividir en dos subconjuntos con la misma suma?

KNAPSACK (0/1)

- **Entrada:** Objetos con peso w_i y valor v_i, capacidad W, entero k
- **Pregunta:** ¿Existe un subconjunto de peso total $\leq W$ con valor total $\geq k$?

Demostración de NP-Completitud: Metodología

Pasos para Demostrar NP-Completitud

1. **Demostrar que $L \in NP$:**
   - Mostrar un certificado y un verificador polinomial.
2. **Seleccionar un problema NP-completo conocido L':**
   - SAT, 3-SAT, CLIQUE, VERTEX COVER, etc.
3. **Construir una reducción $L' \leq_p L$:**
   - Definir una función f que transforme instancias de L' en instancias de L.
   - Demostrar que f es computable en tiempo polinomial.
   - Demostrar que x es un "sí" de $L' \iff f(x)$ es un "sí" de L.

Ejemplo: Reducción de 3-SAT a CLIQUE

3-SAT $\leq_p$ CLIQUE

[Esbozo de la reducción] Dada una fórmula 3-SAT con m cláusulas:

1. Crear un vértice por cada literal en cada cláusula.
2. Conectar vértices que están en diferentes cláusulas y no son contradictorios (x y $\neg x$ no se conectan).
3. Preguntar si existe un clique de tamaño m.

Qué Hacer Cuando Enfrentamos un Problema NP-Completo

Estrategias Prácticas

1. **Buscar restricciones del problema:** Muchos problemas NP-completos se vuelven tratables bajo ciertas condiciones (grafos planares, árboles, etc.).
2. **Usar algoritmos de aproximación:** Encontrar soluciones cercanas a la óptima.
3. **Usar algoritmos heurísticos:** No garantizan optimalidad pero funcionan bien en la práctica.
4. **Usar algoritmos exponenciales pero optimizados:** Backtracking, branch-and-bound, programación dinámica con máscaras de bits.
5. **Usar algoritmos de parámetros fijos (FPT):** Si el parámetro es pequeño, el algoritmo puede ser práctico.
6. **Usar programación lineal entera:** Resolver con solvers comerciales.

Algoritmos de Aproximación

Un algoritmo de aproximación garantiza una solución cuyo valor está dentro de un factor constante del óptimo.

Para VERTEX COVER, el algoritmo greedy que toma todas las aristas y agrega ambos extremos da una aproximación 2.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre las clases P y NP? **P** (Polynomial time) son problemas que pueden ser **resueltos** en tiempo polinomial por una máquina determinista.

NP (Nondeterministic Polynomial time) son problemas cuyas soluciones pueden ser **verificadas** en tiempo polinomial.

La diferencia clave es que para problemas en P podemos encontrar la solución rápidamente, mientras que para problemas en NP solo podemos verificar rápidamente una solución propuesta. Se cree (pero no se ha probado) que $P \neq NP$, lo que significa que hay problemas en NP que no pueden resolverse en tiempo polinomial.

Dado el problema SUBSET SUM: ¿Existe un subconjunto de $\{1, 4, 9, 14, 23\}$ que sume 28? ¿Qué tipo de problema es? Probamos combinaciones:

- $23 + 4 + 1 = 28$
- $23 + 4 + 1 = 28$ funciona.

Existe solución: $\{23, 4, 1\}$.

SUBSET SUM es un problema NP-completo. No se conoce algoritmo polinomial para el caso general, pero para este caso particular encontramos una solución por búsqueda exhaustiva.

Demuestre que si $P = NP$, entonces todos los problemas en NP se pueden resolver en tiempo polinomial. Si $P = NP$, entonces por definición todos los problemas en NP están en P. Esto significa que para cualquier problema en NP existe un algoritmo determinista que lo resuelve en tiempo polinomial.

En particular, los problemas NP-completos (que son un subconjunto de NP) también estarían en P, por lo que podríamos resolver SAT, CLIQUE, TSP, etc., en tiempo polinomial.

Esta es la razón por la que la pregunta $P = NP$ es tan importante: si la respuesta es afirmativa, muchos problemas considerados "difíciles" tendrían soluciones eficientes.

¿Cuál es la complejidad temporal de verificar una solución propuesta para el problema CLIQUE? Dado un grafo $G = (V, E)$ y un subconjunto $C \subseteq V$ propuesto como clique:

1. Verificar que $|C| = k$: $O(1)$
2. Verificar que para todo par $u, v \in C$, $(u, v) \in E$: hay $k2$ pares, cada verificación $O(1)$.

La verificación toma $O(k^2)$ tiempo, que es polinomial en el tamaño de la entrada. Por lo tanto, CLIQUE está en NP.

Muestre una reducción de VERTEX COVER a CLIQUE. Dado un grafo $G = (V, E)$ y un entero k, construimos el grafo complemento $\overline{G} = (V, \overline{E})$ donde $\overline{E}$ contiene todas las aristas que no están en E.

La propiedad clave es: C es un vertex cover de G de tamaño k si y solo si $V \setminus C$ es un clique de $\overline{G}$ de tamaño $|V| - k$.

Reducción: Para una instancia (G, k) de VERTEX COVER, creamos la instancia $(\overline{G}, |V| - k)$ de CLIQUE.

Corrección:

- Si C es vertex cover de G de tamaño k, entonces cualquier arista de $\overline{G}$ tiene al menos un extremo fuera de C, por lo que $V \setminus C$ es clique en $\overline{G}$.

- Si S es clique en $\overline{G}$ de tamaño $|V| - k$, entonces $V \setminus S$ es vertex cover de G de tamaño k.

¿Por qué SAT es importante en la teoría de NP-completitud? SAT fue el primer problema demostrado NP-completo (Teorema de Cook-Levin, 1971). Esto es fundamental porque:

1. Proporciona un "primer" problema NP-completo a partir del cual reducir otros problemas.
2. Demuestra que la clase NP tiene problemas "completos".
3. Abre la puerta para demostrar NP-completitud de cientos de otros problemas.

Para demostrar que un problema X es NP-completo, basta con mostrar que $X \in NP$ y que $SAT \leq_p X$ (o reducir desde cualquier otro NP-completo conocido).

¿Cuál es la diferencia entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-difícil (todos los problemas de NP se reducen a él).
- **NP-hard:** Un problema tal que todos los problemas de NP se reducen a él, pero no necesariamente está en NP.

Los problemas NP-hard pueden ser más "difíciles" que los NP-completos. Por ejemplo, el problema de la parada (Halting Problem) es NP-hard pero no está en NP (es indecidible).

Todo problema NP-completo es NP-hard, pero no todo problema NP-hard es NP-completo.

Dada la fórmula $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$, ¿es satisfacible? Probamos asignaciones:

- $x_1 = V$, $x_2 = V$: primera cláusula V, segunda? $\neg V \vee x_3 = F \vee x_3 = x_3$, tercera? $\neg V \vee \neg x_3 = F \vee \neg x_3 = \neg x_3$. Necesitamos $x_3 = V$ y $x_3 = F$, imposible.
- $x_1 = V$, $x_2 = F$: primera V, segunda $\neg V \vee x_3 = x_3$, tercera $\neg F \vee \neg x_3 = V \vee \neg x_3 = V$. Basta con $x_3 = V$.

Asignación: $x_1 = V$, $x_2 = F$, $x_3 = V$ satisface todas las cláusulas. Es satisfacible.

¿Cuál es la complejidad de 2-SAT (SAT con cláusulas de exactamente 2 literales)? 2-SAT está en P. Se puede resolver en tiempo $O(n^2)$ usando

implicaciones y componentes fuertemente conexas. Esto muestra que la restricción a cláusulas de tamaño 2 hace que el problema sea tratable, mientras que 3-SAT ya es NP-completo.

Muestre que VERTEX COVER está en NP. Para demostrar que VERTEX COVER está en NP, debemos exhibir un certificado y un verificador polinomial:

- **Certificado:** Un subconjunto $C \subseteq V$ de k vértices.
- **Verificador:**
  1. Verificar que $|C| = k$: $O(1)$
  2. Para cada arista $(u, v) \in E$, verificar que $u \in C$ o $v \in C$: $O(|E|)$

El verificador corre en tiempo $O(|E|)$, que es polinomial en el tamaño de la entrada. Por lo tanto, VERTEX COVER $\in NP$.

¿Qué significa que un problema sea "NP-completo bajo reducciones de Cook"? Una **reducción de Cook** (o reducción de Turing) permite usar el problema B como una caja negra (oráculo) que resuelve instancias en tiempo unitario, y se permite hacer múltiples consultas. Es más general que la reducción de Karp (reducción many-one) que usa una sola consulta.

La mayoría de las demostraciones de NP-completitud usan reducciones de Karp, que son un caso especial de reducciones de Cook.

¿Es el problema de ordenamiento NP-completo? Justifique. No. El problema de ordenamiento está en P (existen algoritmos $O(n \log n)$ como Merge Sort). Por lo tanto, no puede ser NP-completo a menos que $P = NP$.

Para que un problema sea NP-completo, debe estar en NP y ser NP-difícil. El ordenamiento está en P, pero no se conoce que sea NP-difícil (y probablemente no lo sea).

Dado el problema de la mochila (0/1) con $n = 4$ objetos: pesos $(2, 3, 4, 5)$, valores $(3, 4, 5, 6)$, capacidad $W = 7$, encuentre la solución óptima. Probamos combinaciones:

- Objetos 1 y 2: peso $2 + 3 = 5$, valor $3 + 4 = 7$
- Objetos 1 y 3: peso $2 + 4 = 6$, valor $3 + 5 = 8$
- Objetos 1 y 4: peso $2 + 5 = 7$, valor $3 + 6 = 9$
- Objetos 2 y 3: peso $3 + 4 = 7$, valor $4 + 5 = 9$
- Objetos 2 y 4: peso $3 + 5 = 8 > 7$, no
- Objetos 3 y 4: peso $4 + 5 = 9 > 7$, no

- Objetos 1,2,3: peso $2 + 3 + 4 = 9 > 7$, no

Soluciones óptimas: $\{1, 4\}$ con valor 9, o $\{2, 3\}$ con valor 9.

¿Cuál es la complejidad de la búsqueda exhaustiva para un problema NP-completo con $n = 50$? Para muchos problemas NP-completos (como SAT o SUBSET SUM), la búsqueda exhaustiva tiene complejidad $O(2^n)$. Para $n = 50$:

$$2^{50} \approx 1.12 \times 10^{15} operaciones$$

Suponiendo 10^9 operaciones por segundo, tomaría aproximadamente 10^6 segundos, ¡unos 11.5 días! Esto muestra por qué los problemas NP-completos son intratables para instancias grandes.

Muestre que si un problema NP-completo se puede resolver en tiempo $O(n^{100})$, entonces $P = NP$. Si existe un algoritmo polinomial para un problema NP-completo L, entonces para cualquier $L' \in NP$ tenemos $L' \leq_p L$. Como la reducción es polinomial y el algoritmo para L es polinomial, la composición es polinomial. Por lo tanto, todo problema en NP se puede resolver en tiempo polinomial, lo que implica $P = NP$.

Esto es cierto independientemente del exponente, incluso si es n^{100}. Lo que importa es que sea polinomial.

¿Qué es el problema HAMILTONIAN CYCLE? ¿Es NP-completo? **HAMILTONIAN CYCLE:** Dado un grafo $G = (V, E)$, ¿existe un ciclo que visite cada vértice exactamente una vez?

Sí, HAMILTONIAN CYCLE es NP-completo. Se puede reducir desde VERTEX COVER o desde SAT. La versión para grafos dirigidos también es NP-completa.

La versión para grafos completos con pesos (TSP) también es NP-completa.

¿Cuál es la relación entre P, NP, NP-completo y NP-hard? Dibuje un diagrama de Venn. Si asumimos $P \neq NP$:

[scale=0.7] (0,0) circle (3cm); (0,0) circle (1.5cm); (-1.5,1.5) circle (1.2cm); at (0,2.5) NP; at (0,1) P; at (-1.8,2.2) NP-completo; at (2.5,0) NP-hard;

- $P \subseteq NP$
- NP-completo $\subset$ NP
- NP-hard contiene a NP-completo y se extiende fuera de NP

¿Es posible que $P = NP$? ¿Qué implicaría? Es posible, pero la mayoría de los expertos cree que es improbable. Si $P = NP$:

- Todos los problemas NP-completos tendrían algoritmos polinomiales.

- La criptografía de clave pública (RSA, etc.) sería vulnerable, ya que factorización está en NP.
- Muchos problemas de optimización serían resolubles eficientemente.
- Sería uno de los descubrimientos más importantes de la computación.

El consenso general es que $P \neq NP$, pero no hay demostración.

Reduzca VERTEX COVER a SUBSET SUM. **Reducción:** Dado un grafo $G = (V, E)$ con $V = \{1, \ldots, n\}$ y $E = \{e_1, \ldots, e_m\}$, construimos:

- Un número a_i por cada vértice i: representación en base grande con dígitos para las aristas incidentes.
- Un número t como objetivo.

Cada vértice se representa como un número en base $B = m + 1$: el j-ésimo dígito es 1 si la arista e_j es incidente al vértice i.

El objetivo t es un número con todos los dígitos en 1 (para que cada arista sea cubierta al menos una vez).

Un subconjunto de vértices suma t si y solo si es un vertex cover.

Explique por qué el problema del viajante (TSP) es NP-difícil. TSP (Traveling Salesman Problem): Dadas n ciudades y distancias, encontrar el recorrido más corto que visita todas exactamente una vez.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa porque:

1. Está en NP (verificar un recorrido es lineal).
2. Se puede reducir HAMILTONIAN CYCLE a TSP (asignar peso 1 a las aristas del grafo, 2 a las que no existen).

TSP es NP-hard porque la versión de optimización es al menos tan difícil como la versión de decisión.

¿Qué es el problema de la clique máxima? ¿Es NP-completo? **CLIQUE:** Dado un grafo G y un entero k, ¿existe un subconjunto de k vértices completamente conectado?

La versión de decisión de CLIQUE es NP-completa. Se puede reducir desde 3-SAT construyendo un grafo donde los vértices representan literales y se conectan si son compatibles (no son contradictorios y están en diferentes cláusulas).

La versión de optimización (encontrar la clique máxima) es NP-hard.

¿Qué son los problemas co-NP? Dé un ejemplo. **co-NP** es la clase de problemas cuyo complemento está en NP. Es decir, un problema L está en co-NP si $\overline{L} \in NP$.

Ejemplo: UNSAT (verificar que una fórmula no es satisfacible) está en co-NP. Dado un certificado de que no existe asignación, no es fácil verificarlo (a diferencia de SAT, donde una asignación es un certificado fácil).

Se cree que $NP \neq co-NP$, y que los problemas NP-completos no están en co-NP (a menos que $P = NP$).

Ejercicios Propuestos (20 Ejercicios)

1. Demuestre que SUBSET SUM está en NP.
2. ¿Es el problema de encontrar el elemento máximo en un arreglo NP-completo? ¿Por qué?
3. Muestre una reducción de SAT a 3-SAT.
4. ¿Cuál es la diferencia entre reducción de Karp y reducción de Cook?
5. Demuestre que si $A \leq_p B$ y $B \in P$, entonces $A \in P$.
6. ¿Es el problema de ordenamiento topológico NP-completo?
7. Muestre que VERTEX COVER en grafos bipartitos está en P.
8. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
9. Demuestre que 3-SAT es NP-completo (esbozo).
10. ¿Qué implica que un problema sea NP-completo para la búsqueda de algoritmos?
11. Muestre una reducción de CLIQUE a VERTEX COVER.
12. ¿Cuál es la complejidad de la programación dinámica para SUBSET SUM?
13. ¿Qué es el problema de la partición? ¿Es NP-completo?
14. Demuestre que PARTITION $\leq_p$ SUBSET SUM.
15. ¿Qué es un algoritmo de aproximación? Dé un ejemplo.
16. ¿Cuál es la complejidad de verificar una solución para TSP?
17. ¿Qué es el problema del circuito de Hamilton en grafos dirigidos?

18. ¿Cómo se relaciona P con co-NP?
19. ¿Qué es el problema de la mochila fraccionaria? ¿Es NP-completo?
20. Investigue el problema INDEPENDENT SET y su relación con VERTEX COVER.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La diferencia fundamental entre problemas tratables (P) e intratables (NP-completos).
- Las definiciones formales de P, NP, NP-completo y NP-hard.
- El concepto de reducción polinomial y su importancia para demostrar NP-completitud.
- El Teorema de Cook-Levin y la importancia de SAT como primer problema NP-completo.
- Los problemas NP-completos clásicos y cómo reconocerlos.
- Estrategias prácticas para enfrentar problemas NP-completos en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

Unidad III: Introducción a Problemas NP-hard

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición y características de los problemas NP-hard.
- Distinguir entre problemas NP-completos y NP-hard.
- Identificar problemas NP-hard clásicos en diferentes dominios.
- Aplicar estrategias para enfrentar problemas NP-hard en la práctica.
- Comprender la jerarquía de complejidad más allá de NP.

Definición de NP-hard

Definición Formal

Un problema L es **NP-hard** (NP-difícil) si para todo problema $L' \in NP$, existe una reducción polinomial $L' \leq_p L$.

A diferencia de los problemas NP-completos, un problema NP-hard **no necesita estar en NP**.

[scale=0.8] (0,0) ellipse (3.5cm and 2.5cm); (0,0) circle (2cm); (0,0) circle (1cm);
at (0,2.2) NP; at (0,1) P; at (2.5,1.5) NP-completo; at (3.5,0) NP-hard;
[-¿, thick] (2,1.2) – (3,1.8);

Relación entre NP-completo y NP-hard

Todo problema NP-completo es NP-hard. Por definición, un problema NP-completo L satisface:

1. $L \in NP$
2. Para todo $L' \in NP$, $L' \leq_p L$

La segunda condición es exactamente la definición de NP-hard.

No todo problema NP-hard es NP-completo. [Contraejemplo] El **Problema de la Parada** (Halting Problem) es NP-hard pero no está en NP (es indecidible). Otros ejemplos incluyen problemas de optimización cuya versión de decisión está en NP.

Problemas NP-hard Clásicos

Problemas de Optimización

TSP (Traveling Salesman Problem) - Versión de Optimización

TSP (Optimización): Dadas n ciudades y distancias $d(i, j)$ entre cada par, encontrar el recorrido más corto que visita cada ciudad exactamente una vez y regresa al punto de partida.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa. La versión de optimización es NP-hard.

KNAPSACK - Versión de Optimización

KNAPSACK (Optimización): Dados n objetos con pesos w_i y valores v_i, y una capacidad W, encontrar el subconjunto que maximiza el valor total sin exceder la capacidad.

VERTEX COVER - Versión de Optimización

VERTEX COVER (Optimización): Dado un grafo $G = (V, E)$, encontrar el conjunto de vértices más pequeño que cubre todas las aristas.

Problemas de Conteo

#SAT (SAT counting): Dada una fórmula booleana, contar el número de asignaciones que la satisfacen.

#SAT es P-completo, una clase de complejidad aún más difícil que NP.

Problemas de Decisión que no están en NP

Problema de la Parada (Halting Problem)

HALTING: Dado un programa P y una entrada x, ¿termina el programa P al ejecutarse con entrada x?

El Problema de la Parada es indecidible (no existe algoritmo que lo resuelva para todos los casos). Es NP-hard pero no está en NP.

Problema de Correspondencia de Post

PCP: Dada una colección de fichas con una cadena superior y una inferior, ¿existe una secuencia de fichas (con repetición permitida) que forme la misma cadena en la parte superior e inferior?

Jerarquía de Complejidad Más Allá de NP

Clases de Complejidad

[scale=0.7] (0,3) ellipse (4cm and 1cm); (0,1.5) ellipse (3.5cm and 1cm); (0,0) ellipse (3cm and 1cm);
at (0,3.5) PSPACE; at (0,2) NP; at (0,0.5) P; at (-2.5,2.5) co-NP; at (2.5,2.5) NP-completo; at (3,3.8) PSPACE-completo;

Clase PSPACE

PSPACE es la clase de problemas que pueden ser resueltos usando espacio polinomial (sin restricción de tiempo).

- $P \subseteq NP \subseteq PSPACE$
- Se cree que estas inclusiones son estrictas.
- Ejemplo de problema PSPACE-completo: QBF (Quantified Boolean Formulas)

Clase EXP

EXP es la clase de problemas que pueden ser resueltos en tiempo exponencial.

- $NP \subseteq EXP$
- Se cree que $NP \neq EXP$

Clase P

#P es la clase de problemas de conteo asociados a problemas en NP. Dado un problema de decisión en NP, contar el número de soluciones.

- #SAT es P-completo
- #P es al menos tan difícil como NP

Estrategias para Enfrentar Problemas NP-hard

Algoritmos de Aproximación

Un **algoritmo de aproximación** es un algoritmo que produce una solución cercana a la óptima, con una garantía de calidad.

[Vertex Cover] El algoritmo greedy que elige todos los vértices de todas las aristas es una aproximación 2.

Algoritmos Heurísticos

- **Búsqueda Local:** Comenzar con una solución y mejorarla mediante pequeños cambios.
- **Algoritmos Genéticos:** Basados en evolución y selección natural.
- **Recocido Simulado:** Inspirado en el enfriamiento de metales.
- **Búsqueda Tabú:** Evita ciclos manteniendo una lista de movimientos recientes.

Algoritmos de Parámetros Fijos (FPT)

Un problema es **FPT** (Fixed-Parameter Tractable) si se puede resolver en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro del problema.

VERTEX COVER se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación.

Algoritmos Exactos Exponenciales Optimizados

- **Meet-in-the-Middle:** Dividir el problema en dos mitades y combinar.
- **Branch and Bound:** Poda de ramas que no pueden mejorar la mejor solución encontrada.

- **Programación Dinámica con Máscaras de Bits:** $O(2^n n^2)$ para TSP.

Programación Lineal Entera (ILP)

- Formular el problema como un programa lineal entero.
- Usar solvers comerciales (CPLEX, Gurobi, SCIP).
- Útil para instancias de tamaño moderado.

Problemas NP-hard Notables

Problemas de Grafos

COLORACIÓN DE GRAFOS

Dado un grafo G, encontrar el número cromático $\chi(G)$ (mínimo número de colores necesarios para colorear los vértices sin que dos adyacentes compartan color).

- La versión de decisión ("¿se puede colorear con k colores?") es NP-completa para $k \geq 3$.
- La versión de optimización (encontrar $\chi(G)$) es NP-hard.
- Para $k = 2$, es polinomial (verificar bipartitud).

CLIQUE MÁXIMA

Dado un grafo G, encontrar el clique de tamaño máximo.

CAMINO MÁS LARGO

Dado un grafo G y dos vértices s, t, encontrar el camino más largo de s a t (sin repetir vértices).

Problemas de Números

FACTORIZACIÓN DE ENTEROS

Dado un entero N, encontrar su factorización en números primos.

No se sabe si la factorización es NP-completa, pero se cree que está en la intersección de NP y co-NP. Es la base de la criptografía RSA.

LOGARITMO DISCRETO

Dados g, h y un primo p, encontrar x tal que $g^x \equiv h \pmod{p}$.

Problemas de Planificación

JOB SHOP SCHEDULING

Dadas m máquinas y n trabajos, cada trabajo consiste en una secuencia de operaciones en máquinas específicas, minimizar el tiempo total (makespan).

BIN PACKING (Optimización)

Dados objetos con tamaños s_i, encontrar el mínimo número de contenedores de capacidad C necesarios para empaquetarlos.

Problemas de Cadenas

ALINEAMIENTO DE SECUENCIAS MÚLTIPLE

Dadas múltiples cadenas, encontrar un alineamiento que maximice una puntuación de similitud.

Problemas de Satisfacibilidad Cuantificada (QBF)

Dada una fórmula booleana con cuantificadores $\forall$ y $\exists$, determinar si es verdadera.

QBF es PSPACE-completo, considerado más difícil que los problemas NP-completos.

Técnicas de Enfrentamiento Práctico

Algoritmos de Aproximación para Problemas Clásicos

Aproximación para VERTEX COVER

[language=C, caption=Aproximación 2 para Vertex Cover] APROX-VERTEX-COVER(G) C = empty set for each edge (u,v) in E if u not in C and v not in C add u and v to C return C

Aproximación para TSP Métrico

Para TSP métrico (desigualdad triangular), el algoritmo de árbol de expansión mínimo da una aproximación 2.

[language=C, caption=Aproximación 2 para TSP Métrico] APROX-TSP(G) T = MST(G) H = double each edge of T // recorrido de Euler return shortcut(H) // eliminar vértices repetidos

Aproximación para KNAPSACK

[language=C, caption=Esquema de Aproximación para KNAPSACK] APROX-KNAPSACK(items, W, epsilon) // Escalar valores para hacer el problema más pequeño scale = epsilon * $\max_v alue/n foreachitemiv'_i = floor(v_i/scale)//ResolverKNAPSACK$

Algoritmos Exactos Exponenciales

FPT para VERTEX COVER

[language=C, caption=FPT para Vertex Cover] FPT-VERTEX-COVER(G, k) if no edges: return empty set if k == 0: return no solution choose an edge (u,v) // Ramificar: u en la solución o v en la solución C1 = u FPT-VERTEX-COVER(G - u, k-1) C2 = v FPT-VERTEX-COVER(G - v, k-1) return mejor de C1 y C2 Complejidad: $O(2^k \cdot n)$.

Meet-in-the-Middle para SUBSET SUM

[language=C, caption=Meet-in-the-Middle para Subset Sum] MITM-SUBSET-SUM(S, t) n = —S— A = primeros n/2 elementos B = últimos n/2 elementos // Generar todas las sumas de subconjuntos de A sumsA = todas las sumas de subconjuntos de A // Para cada suma en B, buscar complemento en sumsA for each subset sum s of B if t - s in sumsA return True return False Complejidad: $O(2^{n/2})$.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-hard.
- **NP-hard:** Un problema tal que todo problema en NP se reduce a él, pero no necesariamente está en NP.

Es decir, NP-hard es una condición más débil (solo requiere la propiedad de reducción), mientras que NP-completo requiere además que el problema pertenezca a NP.

Ejemplo: TSP (optimización) es NP-hard pero no necesariamente NP-completo porque la versión de optimización no es un problema de decisión.

¿Puede un problema indecidible ser NP-hard? Justifique. Sí. El Problema de la Parada (Halting Problem) es indecidible y es NP-hard. Para demostrar que es NP-hard, se reduce SAT al Problema de la Parada:

- Dada una fórmula ϕ, construir un programa que pruebe todas las asignaciones y termine si encuentra una que satisfaga ϕ.
- El programa termina si y solo si ϕ es satisfacible.

Esto muestra que SAT $\leq_p$ HALTING, por lo que HALTING es NP-hard.

¿Qué es un algoritmo de aproximación? Dé un ejemplo. Un algoritmo de aproximación es un algoritmo que encuentra una solución cercana a la óptima, con una garantía de calidad demostrable.

Ejemplo: Vertex Cover tiene un algoritmo de aproximación 2:

1. Tomar todas las aristas del grafo.
2. Por cada arista, agregar ambos extremos al cover.
3. El cover resultante tiene tamaño a lo sumo el doble del óptimo.

La garantía es que $|C| \leq 2 \cdot |C_{opt}|$.

¿Qué significa que un algoritmo sea FPT (Fixed-Parameter Tractable)? Un algoritmo FPT resuelve un problema parametrizado en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro (como el tamaño de la solución) y f es una función arbitraria (típicamente exponencial en k).

Ejemplo: Vertex Cover se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación. Para k pequeño (ej. $k \leq 30$), el algoritmo es práctico.

La ventaja de FPT es que separa la dificultad exponencial al parámetro k del tamaño n del problema.

¿Cuál es la relación entre P, NP, NP-hard y PSPACE?

- $P \subseteq NP \subseteq PSPACE$
- NP-hard $\not\subseteq$ NP (problemas NP-hard pueden estar fuera de NP)
- NP-completo = NP $\cap$ NP-hard

Se cree que todas estas inclusiones son estrictas: $P \subset NP \subset PSPACE$, pero no está demostrado.

Un problema PSPACE-completo (como QBF) es más difícil que cualquier problema NP-completo (a menos que NP = PSPACE).

¿Por qué el problema de la factorización es importante en criptografía? La seguridad del sistema RSA se basa en la dificultad de factorizar grandes números compuestos. Si existiera un algoritmo polinomial para factorización, RSA sería inseguro.

Factorización está en NP (el certificado son los factores primos) y en co-NP (el certificado de no primalidad). No se sabe si es NP-completo, y muchos creen que no lo es. Está en la intersección NP $\cap$ co-NP.

El algoritmo más rápido conocido para factorización es el General Number Field Sieve (GNFS), que es subexponencial pero no polinomial.

Demuestre que el problema de optimización de VERTEX COVER es NP-hard. La versión de decisión de VERTEX COVER ("¿existe un vertex cover de tamaño $\leq k$?") es NP-completa. Para demostrar que la versión de optimización es NP-hard, observamos que:

1. Si tuviéramos un algoritmo que encuentra el vertex cover mínimo, podríamos responder la pregunta de decisión comparando el tamaño del cover encontrado con k.

2. Por lo tanto, la versión de optimización es al menos tan difícil como la versión de decisión.

Como la versión de decisión es NP-completa, la versión de optimización es NP-hard.

¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es NP-completo? La coloración de grafos pregunta si se pueden colorear los vértices de un grafo con k colores de modo que vértices adyacentes tengan colores diferentes.

- Para $k = 1$: trivial, solo funciona si el grafo no tiene aristas.

- Para $k = 2$: polinomial (verificar si el grafo es bipartito con BFS).

- Para $k \geq 3$: NP-completo.

La versión de optimización (encontrar el número cromático) es NP-hard.

Describa el algoritmo de aproximación para TSP métrico y su factor de aproximación. **Algoritmo:**

1. Calcular el árbol de expansión mínimo (MST) del grafo.

2. Duplicar cada arista del MST para obtener un grafo euleriano.

3. Encontrar un recorrido de Euler.

4. Aplicar atajos (shortcut) para eliminar vértices repetidos, obteniendo un ciclo Hamiltoniano.

Factor de aproximación: 2. Es decir, $C_{aprox} \leq 2 \cdot C_{opt}$.

Demostración: El MST tiene peso $\leq C_{opt}$. Duplicar las aristas da peso $2 \cdot C_{opt}$. Los atajos no aumentan el peso por la desigualdad triangular.

¿Qué es el problema SAT y por qué es más difícil que SAT? #SAT es el problema de contar el número de asignaciones que satisfacen una fórmula booleana.

Es más difícil que SAT porque:

1. Resolver SAT solo requiere saber si existe al menos una asignación.

2. Contar todas las asignaciones requiere explorar el espacio de soluciones más exhaustivamente.

3. #SAT es #P-completo, y se cree que #P es más difícil que NP.

Si se pudiera resolver #SAT en tiempo polinomial, entonces $P = NP$ (porque se podría responder SAT viendo si el conteo es > 0).

¿Qué es el problema de la programación lineal entera (ILP) y por qué es NP-hard? ILP (Integer Linear Programming) es el problema de optimizar una función lineal sujeta a restricciones lineales, con la condición de que las variables deben ser enteras.

Es NP-hard porque muchos problemas NP-completos (como KNAPSACK, VERTEX COVER) pueden formularse como ILP. Por ejemplo, KNAPSACK se formula como:

$$Maximizar \sum v_i x_i sujetoa \sum w_i x_i \leq W, x_i \in \{0, 1\}$$

La versión sin restricción de integralidad (programación lineal) está en P.

¿Qué diferencia hay entre un algoritmo de aproximación y una heurística?

- **Algoritmo de aproximación:** Tiene una garantía demostrable sobre la calidad de la solución (ej. "nunca es peor que el doble del óptimo").

- **Heurística:** No tiene garantía teórica; funciona bien en la práctica pero puede fallar catastróficamente en algunos casos.

Ejemplo: La heurística del vecino más cercano para TSP no tiene factor de aproximación constante (puede ser arbitrariamente mala), mientras que el algoritmo basado en MST garantiza factor 2.

¿Qué es el problema de la mochila múltiple (Multiple Knapsack)? El problema de la mochila múltiple es una generalización donde hay m mochilas, cada una con capacidad W_j, y se deben asignar objetos a las mochilas maximizando el valor total.

Es NP-hard incluso para $m = 2$. Se puede reducir KNAPSACK a Multiple Knapsack con $m = 2$ (usando una mochila dummy).

Tiene aplicaciones en:

- Carga de contenedores
- Asignación de recursos
- Publicidad en línea (asignación de anuncios a espacios)

Demuestre que TSP es NP-hard. Para demostrar que TSP es NP-hard, reducimos HAMILTONIAN CYCLE a TSP:

1. Dado un grafo $G = (V, E)$ de HAMILTONIAN CYCLE.
2. Construir instancia de TSP con n ciudades.
3. Definir distancias: $d(i, j) = 1$ si $(i, j) \in E$, $d(i, j) = 2$ si no.
4. Preguntar si existe un recorrido de longitud $\leq n$.

Existe un ciclo Hamiltoniano en G si y solo si existe un recorrido TSP de longitud n.

Como HAMILTONIAN CYCLE es NP-completo, TSP es NP-hard.

¿Qué es el problema de la planificación (Job Shop Scheduling) y por qué es difícil? Job Shop Scheduling es el problema de asignar trabajos a máquinas en el tiempo. Cada trabajo consiste en una secuencia de operaciones, cada operación requiere una máquina específica por un tiempo determinado.

Objetivo: minimizar el makespan (tiempo total).

Es NP-hard incluso para 2 máquinas y casos muy restringidos. La dificultad surge de:

- Secuenciación de operaciones
- Asignación de recursos
- Conflictos por máquinas compartidas

Es un problema fundamental en ingeniería de producción y logística.

¿Qué es un esquema de aproximación (PTAS) y un esquema de aproximación totalmente polinomial (FPTAS)?

- **PTAS** (Polynomial-Time Approximation Scheme): Un algoritmo que para cualquier $\epsilon > 0$ produce una solución con factor $1+\epsilon$ (o $1-\epsilon$ para maximización) en tiempo polinomial en n (pero puede ser exponencial en $1/\epsilon$).

- **FPTAS** (Fully PTAS): El tiempo también es polinomial en $1/\epsilon$.

KNAPSACK tiene FPTAS, mientras que TSP no tiene PTAS a menos que $P = NP$.

¿Qué es el problema de la parada (Halting Problem) y por qué es indecidible? El Problema de la Parada pregunta: Dado un programa P y una entrada x, ¿termina P al ejecutarse con entrada x?

Turing demostró que es indecidible (no existe algoritmo que lo resuelva para todos los casos). La demostración usa diagonalización:

1. Supongamos que existe $H(P, x)$ que decide si P termina con entrada x.

2. Construimos $D(P)$ que llama a $H(P, P)$ y entra en bucle si H dice que termina.

3. Evaluar $D(D)$ lleva a una contradicción.

Es NP-hard porque podemos reducir SAT a él (como vimos antes).

¿Qué es el problema de la correspondencia de Post (PCP)? PCP (Post Correspondence Problem): Dadas n fichas, cada una con una cadena superior u_i y una cadena inferior v_i, ¿existe una secuencia de fichas (con repetición) tal que la concatenación de las cadenas superiores es igual a la concatenación de las cadenas inferiores?

Es indecidible. Es un ejemplo clásico de problema no computable que se usa para demostrar indecidibilidad de otros problemas.

Aunque es indecidible, hay algoritmos que funcionan para instancias pequeñas.

¿Qué es un algoritmo de búsqueda local y cuándo es útil? Un algoritmo de búsqueda local comienza con una solución candidata y la mejora iterativamente mediante pequeños cambios (movimientos locales).

Ejemplo para TSP:

1. Comenzar con un recorrido aleatorio.

2. Aplicar movimientos 2-opt (intercambiar dos aristas) para mejorar.
3. Repetir hasta no encontrar mejoras.

Es útil cuando:

- El espacio de búsqueda es muy grande
- Se necesita una solución rápida (no necesariamente óptima)
- Se pueden definir movimientos locales efectivos

¿Qué es el problema de la búsqueda de cliques máximos y por qué es difícil? El problema de la clique máxima es encontrar el subconjunto de vértices completamente conectado más grande en un grafo.

Es NP-hard porque:

1. La versión de decisión ("¿existe un clique de tamaño $\geq k$?") es NP-completa.
2. La versión de optimización es al menos tan difícil.

No se conoce algoritmo de aproximación con factor constante para clique máxima (es inaproximable a menos que $P = NP$).

En la práctica, se usan algoritmos de branch-and-bound con poda.

Ejercicios Propuestos (20 Ejercicios)

1. ¿Cuál es la diferencia entre un problema NP-hard y un problema NP-completo? Dé ejemplos.
2. Demuestre que la versión de optimización de KNAPSACK es NP-hard.
3. ¿Qué es el algoritmo de aproximación para VERTEX COVER? ¿Cuál es su factor?
4. ¿Qué es un algoritmo FPT? Dé un ejemplo.
5. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
6. Demuestre que TSP (optimización) es NP-hard.
7. ¿Qué es el problema de la factorización y por qué es importante en criptografía?

8. ¿Qué es el problema de la planificación (Job Shop Scheduling)?
9. ¿Qué es un PTAS y un FPTAS? Dé ejemplos.
10. ¿Qué es el problema de la parada y por qué es indecidible?
11. ¿Qué es el problema de la correspondencia de Post?
12. ¿Qué es un algoritmo de búsqueda local? Dé un ejemplo.
13. ¿Qué es el problema de la búsqueda de cliques máximos?
14. ¿Qué es el problema de la mochila múltiple?
15. ¿Qué es el problema de la programación lineal entera?
16. ¿Qué es el problema de la alineamiento de secuencias múltiples?
17. ¿Qué es el problema QBF y cómo se relaciona con PSPACE?
18. ¿Qué es el problema SAT y por qué es más difícil que SAT?
19. ¿Qué es el algoritmo de meet-in-the-middle para SUBSET SUM?
20. Investigue el problema de la ruta más larga en grafos.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de NP-hard y su relación con NP-completo.
- Que los problemas NP-hard no necesitan estar en NP, pueden ser incluso más difíciles (indecidibles).
- La jerarquía de complejidad más allá de NP: PSPACE, EXP, P.
- Estrategias prácticas para enfrentar problemas NP-hard: aproximación, heurísticas, FPT, algoritmos exactos exponenciales.
- Problemas NP-hard clásicos en grafos, números, planificación y cadenas.
- La importancia de estos problemas en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34-35.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1), 230-265.

Unidad III: Introducción a Problemas NP-hard

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición y características de los problemas NP-hard.
- Distinguir entre problemas NP-completos y NP-hard.
- Identificar problemas NP-hard clásicos en diferentes dominios.
- Aplicar estrategias para enfrentar problemas NP-hard en la práctica.
- Comprender la jerarquía de complejidad más allá de NP.

Definición de NP-hard

Definición Formal

Un problema L es **NP-hard** (NP-difícil) si para todo problema $L' \in NP$, existe una reducción polinomial $L' \leq_p L$.

A diferencia de los problemas NP-completos, un problema NP-hard **no necesita estar en NP**.

[scale=0.8] (0,0) ellipse (3.5cm and 2.5cm); (0,0) circle (2cm); (0,0) circle (1cm);
at (0,2.2) NP; at (0,1) P; at (2.5,1.5) NP-completo; at (3.5,0) NP-hard;
[-¿, thick] (2,1.2) – (3,1.8);

Relación entre NP-completo y NP-hard

Todo problema NP-completo es NP-hard. Por definición, un problema NP-completo L satisface:

1. $L \in NP$
2. Para todo $L' \in NP$, $L' \leq_p L$

La segunda condición es exactamente la definición de NP-hard.

No todo problema NP-hard es NP-completo. [Contraejemplo] El **Problema de la Parada** (Halting Problem) es NP-hard pero no está en NP (es indecidible). Otros ejemplos incluyen problemas de optimización cuya versión de decisión está en NP.

Problemas NP-hard Clásicos

Problemas de Optimización

TSP (Traveling Salesman Problem) - Versión de Optimización

TSP (Optimización): Dadas n ciudades y distancias $d(i, j)$ entre cada par, encontrar el recorrido más corto que visita cada ciudad exactamente una vez y regresa al punto de partida.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa. La versión de optimización es NP-hard.

KNAPSACK - Versión de Optimización

KNAPSACK (Optimización): Dados n objetos con pesos w_i y valores v_i, y una capacidad W, encontrar el subconjunto que maximiza el valor total sin exceder la capacidad.

VERTEX COVER - Versión de Optimización

VERTEX COVER (Optimización): Dado un grafo $G = (V, E)$, encontrar el conjunto de vértices más pequeño que cubre todas las aristas.

Problemas de Conteo

#SAT (SAT counting): Dada una fórmula booleana, contar el número de asignaciones que la satisfacen.

#SAT es P-completo, una clase de complejidad aún más difícil que NP.

Problemas de Decisión que no están en NP

Problema de la Parada (Halting Problem)

HALTING: Dado un programa P y una entrada x, ¿termina el programa P al ejecutarse con entrada x?

El Problema de la Parada es indecidible (no existe algoritmo que lo resuelva para todos los casos). Es NP-hard pero no está en NP.

Problema de Correspondencia de Post

PCP: Dada una colección de fichas con una cadena superior y una inferior, ¿existe una secuencia de fichas (con repetición permitida) que forme la misma cadena en la parte superior e inferior?

Jerarquía de Complejidad Más Allá de NP

Clases de Complejidad

[scale=0.7] (0,3) ellipse (4cm and 1cm); (0,1.5) ellipse (3.5cm and 1cm); (0,0) ellipse (3cm and 1cm);
at (0,3.5) PSPACE; at (0,2) NP; at (0,0.5) P; at (-2.5,2.5) co-NP; at (2.5,2.5) NP-completo; at (3,3.8) PSPACE-completo;

Clase PSPACE

PSPACE es la clase de problemas que pueden ser resueltos usando espacio polinomial (sin restricción de tiempo).

- $P \subseteq NP \subseteq PSPACE$
- Se cree que estas inclusiones son estrictas.
- Ejemplo de problema PSPACE-completo: QBF (Quantified Boolean Formulas)

Clase EXP

EXP es la clase de problemas que pueden ser resueltos en tiempo exponencial.

- $NP \subseteq EXP$
- Se cree que $NP \neq EXP$

Clase P

#P es la clase de problemas de conteo asociados a problemas en NP. Dado un problema de decisión en NP, contar el número de soluciones.

- #SAT es P-completo
- #P es al menos tan difícil como NP

Estrategias para Enfrentar Problemas NP-hard

Algoritmos de Aproximación

Un **algoritmo de aproximación** es un algoritmo que produce una solución cercana a la óptima, con una garantía de calidad.

[Vertex Cover] El algoritmo greedy que elige todos los vértices de todas las aristas es una aproximación 2.

Algoritmos Heurísticos

- **Búsqueda Local:** Comenzar con una solución y mejorarla mediante pequeños cambios.
- **Algoritmos Genéticos:** Basados en evolución y selección natural.
- **Recocido Simulado:** Inspirado en el enfriamiento de metales.
- **Búsqueda Tabú:** Evita ciclos manteniendo una lista de movimientos recientes.

Algoritmos de Parámetros Fijos (FPT)

Un problema es **FPT** (Fixed-Parameter Tractable) si se puede resolver en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro del problema.

VERTEX COVER se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación.

Algoritmos Exactos Exponenciales Optimizados

- **Meet-in-the-Middle:** Dividir el problema en dos mitades y combinar.
- **Branch and Bound:** Poda de ramas que no pueden mejorar la mejor solución encontrada.

- **Programación Dinámica con Máscaras de Bits:** $O(2^n n^2)$ para TSP.

Programación Lineal Entera (ILP)

- Formular el problema como un programa lineal entero.
- Usar solvers comerciales (CPLEX, Gurobi, SCIP).
- Útil para instancias de tamaño moderado.

Problemas NP-hard Notables

Problemas de Grafos

COLORACIÓN DE GRAFOS

Dado un grafo G, encontrar el número cromático $\chi(G)$ (mínimo número de colores necesarios para colorear los vértices sin que dos adyacentes compartan color).

- La versión de decisión ("¿se puede colorear con k colores?") es NP-completa para $k \geq 3$.
- La versión de optimización (encontrar $\chi(G)$) es NP-hard.
- Para $k = 2$, es polinomial (verificar bipartitud).

CLIQUE MÁXIMA

Dado un grafo G, encontrar el clique de tamaño máximo.

CAMINO MÁS LARGO

Dado un grafo G y dos vértices s, t, encontrar el camino más largo de s a t (sin repetir vértices).

Problemas de Números

FACTORIZACIÓN DE ENTEROS

Dado un entero N, encontrar su factorización en números primos.

No se sabe si la factorización es NP-completa, pero se cree que está en la intersección de NP y co-NP. Es la base de la criptografía RSA.

LOGARITMO DISCRETO

Dados g, h y un primo p, encontrar x tal que $g^x \equiv h \pmod{p}$.

Problemas de Planificación

JOB SHOP SCHEDULING

Dadas m máquinas y n trabajos, cada trabajo consiste en una secuencia de operaciones en máquinas específicas, minimizar el tiempo total (makespan).

BIN PACKING (Optimización)

Dados objetos con tamaños s_i, encontrar el mínimo número de contenedores de capacidad C necesarios para empaquetarlos.

Problemas de Cadenas

ALINEAMIENTO DE SECUENCIAS MÚLTIPLE

Dadas múltiples cadenas, encontrar un alineamiento que maximice una puntuación de similitud.

Problemas de Satisfacibilidad Cuantificada (QBF)

Dada una fórmula booleana con cuantificadores $\forall$ y $\exists$, determinar si es verdadera.

QBF es PSPACE-completo, considerado más difícil que los problemas NP-completos.

Técnicas de Enfrentamiento Práctico

Algoritmos de Aproximación para Problemas Clásicos

Aproximación para VERTEX COVER

[language=C, caption=Aproximación 2 para Vertex Cover] APROX-VERTEX-COVER(G) C = empty set for each edge (u,v) in E if u not in C and v not in C add u and v to C return C

Aproximación para TSP Métrico

Para TSP métrico (desigualdad triangular), el algoritmo de árbol de expansión mínimo da una aproximación 2.

[language=C, caption=Aproximación 2 para TSP Métrico] APROX-TSP(G) T = MST(G) H = double each edge of T // recorrido de Euler return shortcut(H) // eliminar vértices repetidos

Aproximación para KNAPSACK

[language=C, caption=Esquema de Aproximación para KNAPSACK] APROX-KNAPSACK(items, W, epsilon) // Escalar valores para hacer el problema más pequeño scale = epsilon * max$_v alue/n foreachitemiv'_i = floor(v_i/scale)//ResolverKNAPSACI$

Algoritmos Exactos Exponenciales

FPT para VERTEX COVER

[language=C, caption=FPT para Vertex Cover] FPT-VERTEX-COVER(G, k) if no edges: return empty set if k == 0: return no solution choose an edge (u,v) // Ramificar: u en la solución o v en la solución C1 = u FPT-VERTEX-COVER(G - u, k-1) C2 = v FPT-VERTEX-COVER(G - v, k-1) return mejor de C1 y C2 Complejidad: $O(2^k \cdot n)$.

Meet-in-the-Middle para SUBSET SUM

[language=C, caption=Meet-in-the-Middle para Subset Sum] MITM-SUBSET-SUM(S, t) n = —S— A = primeros n/2 elementos B = últimos n/2 elementos // Generar todas las sumas de subconjuntos de A sumsA = todas las sumas de subconjuntos de A // Para cada suma en B, buscar complemento en sumsA for each subset sum s of B if t - s in sumsA return True return False Complejidad: $O(2^{n/2})$.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-hard.
- **NP-hard:** Un problema tal que todo problema en NP se reduce a él, pero no necesariamente está en NP.

Es decir, NP-hard es una condición más débil (solo requiere la propiedad de reducción), mientras que NP-completo requiere además que el problema pertenezca a NP.

Ejemplo: TSP (optimización) es NP-hard pero no necesariamente NP-completo porque la versión de optimización no es un problema de decisión.

¿Puede un problema indecidible ser NP-hard? Justifique. Sí. El Problema de la Parada (Halting Problem) es indecidible y es NP-hard. Para demostrar que es NP-hard, se reduce SAT al Problema de la Parada:

- Dada una fórmula ϕ, construir un programa que pruebe todas las asignaciones y termine si encuentra una que satisfaga ϕ.
- El programa termina si y solo si ϕ es satisfacible.

Esto muestra que SAT $\leq_p$ HALTING, por lo que HALTING es NP-hard.

¿Qué es un algoritmo de aproximación? Dé un ejemplo. Un algoritmo de aproximación es un algoritmo que encuentra una solución cercana a la óptima, con una garantía de calidad demostrable.

Ejemplo: Vertex Cover tiene un algoritmo de aproximación 2:

1. Tomar todas las aristas del grafo.
2. Por cada arista, agregar ambos extremos al cover.
3. El cover resultante tiene tamaño a lo sumo el doble del óptimo.

La garantía es que $|C| \leq 2 \cdot |C_{opt}|$.

¿Qué significa que un algoritmo sea FPT (Fixed-Parameter Tractable)? Un algoritmo FPT resuelve un problema parametrizado en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro (como el tamaño de la solución) y f es una función arbitraria (típicamente exponencial en k).

Ejemplo: Vertex Cover se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación. Para k pequeño (ej. $k \leq 30$), el algoritmo es práctico.

La ventaja de FPT es que separa la dificultad exponencial al parámetro k del tamaño n del problema.

¿Cuál es la relación entre P, NP, NP-hard y PSPACE?

- $P \subseteq NP \subseteq PSPACE$
- NP-hard $\not\subseteq$ NP (problemas NP-hard pueden estar fuera de NP)
- NP-completo = NP $\cap$ NP-hard

Se cree que todas estas inclusiones son estrictas: $P \subset NP \subset PSPACE$, pero no está demostrado.

Un problema PSPACE-completo (como QBF) es más difícil que cualquier problema NP-completo (a menos que NP = PSPACE).

¿Por qué el problema de la factorización es importante en criptografía? La seguridad del sistema RSA se basa en la dificultad de factorizar grandes números compuestos. Si existiera un algoritmo polinomial para factorización, RSA sería inseguro.

Factorización está en NP (el certificado son los factores primos) y en co-NP (el certificado de no primalidad). No se sabe si es NP-completo, y muchos creen que no lo es. Está en la intersección NP $\cap$ co-NP.

El algoritmo más rápido conocido para factorización es el General Number Field Sieve (GNFS), que es subexponencial pero no polinomial.

Demuestre que el problema de optimización de VERTEX COVER es NP-hard. La versión de decisión de VERTEX COVER ("¿existe un vertex cover de tamaño $\leq k$?") es NP-completa. Para demostrar que la versión de optimización es NP-hard, observamos que:

1. Si tuviéramos un algoritmo que encuentra el vertex cover mínimo, podríamos responder la pregunta de decisión comparando el tamaño del cover encontrado con k.

2. Por lo tanto, la versión de optimización es al menos tan difícil como la versión de decisión.

Como la versión de decisión es NP-completa, la versión de optimización es NP-hard.

¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es NP-completo? La coloración de grafos pregunta si se pueden colorear los vértices de un grafo con k colores de modo que vértices adyacentes tengan colores diferentes.

- Para $k = 1$: trivial, solo funciona si el grafo no tiene aristas.

- Para $k = 2$: polinomial (verificar si el grafo es bipartito con BFS).

- Para $k \geq 3$: NP-completo.

La versión de optimización (encontrar el número cromático) es NP-hard.

Describa el algoritmo de aproximación para TSP métrico y su factor de aproximación. **Algoritmo:**

1. Calcular el árbol de expansión mínimo (MST) del grafo.

2. Duplicar cada arista del MST para obtener un grafo euleriano.

3. Encontrar un recorrido de Euler.

4. Aplicar atajos (shortcut) para eliminar vértices repetidos, obteniendo un ciclo Hamiltoniano.

Factor de aproximación: 2. Es decir, $C_{aprox} \leq 2 \cdot C_{opt}$.

Demostración: El MST tiene peso $\leq C_{opt}$. Duplicar las aristas da peso $2 \cdot C_{opt}$. Los atajos no aumentan el peso por la desigualdad triangular.

¿Qué es el problema SAT y por qué es más difícil que SAT? #SAT es el problema de contar el número de asignaciones que satisfacen una fórmula booleana.

Es más difícil que SAT porque:

1. Resolver SAT solo requiere saber si existe al menos una asignación.

2. Contar todas las asignaciones requiere explorar el espacio de soluciones más exhaustivamente.

3. #SAT es #P-completo, y se cree que #P es más difícil que NP.

Si se pudiera resolver #SAT en tiempo polinomial, entonces $P = NP$ (porque se podría responder SAT viendo si el conteo es > 0).

¿Qué es el problema de la programación lineal entera (ILP) y por qué es NP-hard? ILP (Integer Linear Programming) es el problema de optimizar una función lineal sujeta a restricciones lineales, con la condición de que las variables deben ser enteras.

Es NP-hard porque muchos problemas NP-completos (como KNAPSACK, VERTEX COVER) pueden formularse como ILP. Por ejemplo, KNAPSACK se formula como:

$$Maximizar \sum v_i x_i sujetoa \sum w_i x_i \leq W, x_i \in \{0, 1\}$$

La versión sin restricción de integralidad (programación lineal) está en P.

¿Qué diferencia hay entre un algoritmo de aproximación y una heurística?

- **Algoritmo de aproximación:** Tiene una garantía demostrable sobre la calidad de la solución (ej. "nunca es peor que el doble del óptimo").

- **Heurística:** No tiene garantía teórica; funciona bien en la práctica pero puede fallar catastróficamente en algunos casos.

Ejemplo: La heurística del vecino más cercano para TSP no tiene factor de aproximación constante (puede ser arbitrariamente mala), mientras que el algoritmo basado en MST garantiza factor 2.

¿Qué es el problema de la mochila múltiple (Multiple Knapsack)? El problema de la mochila múltiple es una generalización donde hay m mochilas, cada una con capacidad W_j, y se deben asignar objetos a las mochilas maximizando el valor total.

Es NP-hard incluso para $m = 2$. Se puede reducir KNAPSACK a Multiple Knapsack con $m = 2$ (usando una mochila dummy).

Tiene aplicaciones en:

- Carga de contenedores
- Asignación de recursos
- Publicidad en línea (asignación de anuncios a espacios)

Demuestre que TSP es NP-hard. Para demostrar que TSP es NP-hard, reducimos HAMILTONIAN CYCLE a TSP:

1. Dado un grafo $G = (V, E)$ de HAMILTONIAN CYCLE.
2. Construir instancia de TSP con n ciudades.
3. Definir distancias: $d(i, j) = 1$ si $(i, j) \in E$, $d(i, j) = 2$ si no.
4. Preguntar si existe un recorrido de longitud $\leq n$.

Existe un ciclo Hamiltoniano en G si y solo si existe un recorrido TSP de longitud n.

Como HAMILTONIAN CYCLE es NP-completo, TSP es NP-hard.

¿Qué es el problema de la planificación (Job Shop Scheduling) y por qué es difícil? Job Shop Scheduling es el problema de asignar trabajos a máquinas en el tiempo. Cada trabajo consiste en una secuencia de operaciones, cada operación requiere una máquina específica por un tiempo determinado.

Objetivo: minimizar el makespan (tiempo total).

Es NP-hard incluso para 2 máquinas y casos muy restringidos. La dificultad surge de:

- Secuenciación de operaciones
- Asignación de recursos
- Conflictos por máquinas compartidas

Es un problema fundamental en ingeniería de producción y logística.

¿Qué es un esquema de aproximación (PTAS) y un esquema de aproximación totalmente polinomial (FPTAS)?

- **PTAS** (Polynomial-Time Approximation Scheme): Un algoritmo que para cualquier $\epsilon > 0$ produce una solución con factor $1+\epsilon$ (o $1-\epsilon$ para maximización) en tiempo polinomial en n (pero puede ser exponencial en $1/\epsilon$).

- **FPTAS** (Fully PTAS): El tiempo también es polinomial en $1/\epsilon$.

KNAPSACK tiene FPTAS, mientras que TSP no tiene PTAS a menos que $P = NP$.

¿Qué es el problema de la parada (Halting Problem) y por qué es indecidible? El Problema de la Parada pregunta: Dado un programa P y una entrada x, ¿termina P al ejecutarse con entrada x?

Turing demostró que es indecidible (no existe algoritmo que lo resuelva para todos los casos). La demostración usa diagonalización:

1. Supongamos que existe $H(P, x)$ que decide si P termina con entrada x.

2. Construimos $D(P)$ que llama a $H(P, P)$ y entra en bucle si H dice que termina.

3. Evaluar $D(D)$ lleva a una contradicción.

Es NP-hard porque podemos reducir SAT a él (como vimos antes).

¿Qué es el problema de la correspondencia de Post (PCP)? PCP (Post Correspondence Problem): Dadas n fichas, cada una con una cadena superior u_i y una cadena inferior v_i, ¿existe una secuencia de fichas (con repetición) tal que la concatenación de las cadenas superiores es igual a la concatenación de las cadenas inferiores?

Es indecidible. Es un ejemplo clásico de problema no computable que se usa para demostrar indecidibilidad de otros problemas.

Aunque es indecidible, hay algoritmos que funcionan para instancias pequeñas.

¿Qué es un algoritmo de búsqueda local y cuándo es útil? Un algoritmo de búsqueda local comienza con una solución candidata y la mejora iterativamente mediante pequeños cambios (movimientos locales).

Ejemplo para TSP:

1. Comenzar con un recorrido aleatorio.

2. Aplicar movimientos 2-opt (intercambiar dos aristas) para mejorar.
3. Repetir hasta no encontrar mejoras.

Es útil cuando:

- El espacio de búsqueda es muy grande
- Se necesita una solución rápida (no necesariamente óptima)
- Se pueden definir movimientos locales efectivos

¿Qué es el problema de la búsqueda de cliques máximos y por qué es difícil? El problema de la clique máxima es encontrar el subconjunto de vértices completamente conectado más grande en un grafo.

Es NP-hard porque:

1. La versión de decisión ("¿existe un clique de tamaño $\geq k$?") es NP-completa.
2. La versión de optimización es al menos tan difícil.

No se conoce algoritmo de aproximación con factor constante para clique máxima (es inaproximable a menos que $P = NP$).

En la práctica, se usan algoritmos de branch-and-bound con poda.

Ejercicios Propuestos (20 Ejercicios)

1. ¿Cuál es la diferencia entre un problema NP-hard y un problema NP-completo? Dé ejemplos.
2. Demuestre que la versión de optimización de KNAPSACK es NP-hard.
3. ¿Qué es el algoritmo de aproximación para VERTEX COVER? ¿Cuál es su factor?
4. ¿Qué es un algoritmo FPT? Dé un ejemplo.
5. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
6. Demuestre que TSP (optimización) es NP-hard.
7. ¿Qué es el problema de la factorización y por qué es importante en criptografía?

8. ¿Qué es el problema de la planificación (Job Shop Scheduling)?
9. ¿Qué es un PTAS y un FPTAS? Dé ejemplos.
10. ¿Qué es el problema de la parada y por qué es indecidible?
11. ¿Qué es el problema de la correspondencia de Post?
12. ¿Qué es un algoritmo de búsqueda local? Dé un ejemplo.
13. ¿Qué es el problema de la búsqueda de cliques máximos?
14. ¿Qué es el problema de la mochila múltiple?
15. ¿Qué es el problema de la programación lineal entera?
16. ¿Qué es el problema de la alineamiento de secuencias múltiples?
17. ¿Qué es el problema QBF y cómo se relaciona con PSPACE?
18. ¿Qué es el problema SAT y por qué es más difícil que SAT?
19. ¿Qué es el algoritmo de meet-in-the-middle para SUBSET SUM?
20. Investigue el problema de la ruta más larga en grafos.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de NP-hard y su relación con NP-completo.
- Que los problemas NP-hard no necesitan estar en NP, pueden ser incluso más difíciles (indecidibles).
- La jerarquía de complejidad más allá de NP: PSPACE, EXP, P.
- Estrategias prácticas para enfrentar problemas NP-hard: aproximación, heurísticas, FPT, algoritmos exactos exponenciales.
- Problemas NP-hard clásicos en grafos, números, planificación y cadenas.
- La importancia de estos problemas en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34-35.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1), 230-265.

Unidad V: Caminos Mínimos - Dijkstra y Bellman-Ford

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de caminos mínimos desde una fuente única.
- Implementar y analizar el algoritmo de Dijkstra para pesos no negativos.
- Implementar y analizar el algoritmo de Bellman-Ford para pesos negativos.
- Detectar ciclos de peso negativo utilizando Bellman-Ford.
- Aplicar estos algoritmos a problemas prácticos de optimización de rutas.
- Comparar ambos algoritmos y elegir el apropiado según el contexto.

Introducción al Problema de Caminos Mínimos

Definición Formal

Dado un grafo dirigido ponderado $G = (V, E)$ con función de peso $w : E \to R$, y un vértice fuente $s \in V$, encontrar para cada vértice $v \in V$ el camino de menor peso desde s a v.

El peso de un camino $p = \langle v_0, v_1, \ldots, v_k \rangle$ es:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

La distancia mínima se define como:

$$\delta(s,v) = \{\min\{w(p) : pescaminodesav\} siexistecamino \infty encasocontrario$$

Propiedades Fundamentales

[Subestructura óptima] Un subcamino de un camino mínimo es también un camino mínimo.

[Desigualdad triangular] Para cualquier arista $(u,v) \in E$:

$$\delta(s,v) \leq \delta(s,u) + w(u,v)$$

Relajación de Aristas

La técnica central de ambos algoritmos es la **relajación**:

[language=C, caption=Relajación de una arista] RELAX(u, v, w) if d[v] ¿ d[u] + w(u, v) d[v] = d[u] + w(u, v) parent[v] = u

[scale=0.8] [circle, draw] (u) at (0,0) u; [circle, draw] (v) at (3,0) v; [-¿] (u) – node[above] $w(u,v)$ (v);
at (0,-1.5) $d[u] = 5$; at (3,-1.5) $d[v] = 9$;
at (1.5,-2.5) $5 + w(u,v) < 9$;

Algoritmo de Dijkstra

Descripción General

Dijkstra encuentra caminos mínimos desde una fuente s a todos los demás vértices. El algoritmo mantiene un conjunto S de vértices cuyas distancias ya están finalizadas, y una cola de prioridad Q con los vértices restantes.

Requisitos

- Todos los pesos de las aristas deben ser **no negativos**.
- El grafo puede ser dirigido o no dirigido.

Pseudocódigo Detallado

[language=C, caption=Algoritmo de Dijkstra] DIJKSTRA(G, w, s) // Inicialización for each vertex v in G.V v.d = INFINITY v.parent = NIL s.d = 0

// Cola de prioridad mínima por d Q = G.V

while Q is not empty u = EXTRACT-MIN(Q) // vértice con menor d for each vertex v in G.Adj[u] if v in Q and v.d ¿ u.d + w(u, v) v.d = u.d + w(u, v) v.parent = u DECREASE-KEY(Q, v, v.d)

Ejemplo Paso a Paso

Consideremos el siguiente grafo:

[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1.5) a; [circle, draw] (b) at (2,-1.5) b; [circle, draw] (c) at (4,0) c; [circle, draw] (d) at (6,1) d; [circle, draw] (e) at (6,-1) e;

[-¿] (s) – node[above left] 4 (a); [-¿] (s) – node[below left] 2 (b); [-¿] (a) – node[above] 1 (c); [-¿] (b) – node[below] 5 (c); [-¿] (b) – node[right] 3 (a); [-¿] (c) – node[above] 2 (d); [-¿] (c) – node[below] 4 (e); [-¿] (d) – node[right] 1 (e);

Paso 1 - Inicialización:

$$d[s] = 0,\ d[a] = \infty,\ d[b] = \infty,\ d[c] = \infty,\ d[d] = \infty,\ d[e] = \infty$$

$$Q = \{s(0), a(\infty), b(\infty), c(\infty), d(\infty), e(\infty)\}$$

Paso 2 - Extraer s (d=0):

- Relajar (s, a): $d[a] = \min(\infty, 0 + 4) = 4$
- Relajar (s, b): $d[b] = \min(\infty, 0 + 2) = 2$

$$Q = \{b(2), a(4), c(\infty), d(\infty), e(\infty)\}$$

Paso 3 - Extraer b (d=2):

- Relajar (b, a): $d[a] = \min(4, 2 + 3 = 5) = 4$ (no cambia)
- Relajar (b, c): $d[c] = \min(\infty, 2 + 5 = 7) = 7$

$$Q = \{a(4), c(7), d(\infty), e(\infty)\}$$

Paso 4 - Extraer a (d=4):

- Relajar (a, c): $d[c] = \min(7, 4 + 1 = 5) = 5$

$$Q = \{c(5), d(\infty), e(\infty)\}$$

Paso 5 - Extraer c (d=5):

- Relajar (c, d): $d[d] = \min(\infty, 5 + 2 = 7) = 7$
- Relajar (c, e): $d[e] = \min(\infty, 5 + 4 = 9) = 9$

$$Q = \{d(7), e(9)\}$$

Paso 6 - Extraer d (d=7):

- Relajar (d, e): $d[e] = \min(9, 7 + 1 = 8) = 8$

$$Q = \{e(8)\}$$

Paso 7 - Extraer e (d=8): Fin.
Resultados finales:

$$d[s] = 0,\ d[a] = 4,\ d[b] = 2,\ d[c] = 5,\ d[d] = 7,\ d[e] = 8$$

Implementación con Montículo Binario

[language=C, caption=Dijkstra con montículo binario] typedef struct int vertex; int dist; HeapNode;

void dijkstra(Graph* G, int s) int* dist = malloc(G-¿V * sizeof(int)); int* parent = malloc(G-¿V * sizeof(int)); int* inHeap = malloc(G-¿V * sizeof(int));

// Inicialización for (int i = 0; i ¡ G-¿V; i++) dist[i] = INF; parent[i] = -1; inHeap[i] = 1; dist[s] = 0;

// Construir heap HeapNode* heap = malloc(G-¿V * sizeof(HeapNode)); int heapSize = G-¿V; for (int i = 0; i ¡ G-¿V; i++) heap[i].vertex = i; heap[i].dist = dist[i]; buildMinHeap(heap, heapSize);

while (heapSize ¿ 0) HeapNode min = extractMin(heap, heapSize); int u = min.vertex; inHeap[u] = 0;

Node* adj = G-¿adj[u]; while (adj != NULL) int v = adj-¿vertex; int weight = adj-¿weight; if (inHeap[v] dist[v] ¿ dist[u] + weight) dist[v] = dist[u] + weight; parent[v] = u; decreaseKey(heap, heapSize, v, dist[v]); adj = adj-¿next;

// Imprimir resultados for (int i = 0; i ¡ G-¿V; i++) printf("Distancia a

Análisis de Complejidad

- **Con arreglo simple:** $O(V^2)$
  - EXTRACT-MIN: $O(V)$ por operación, $O(V^2)$ total
  - DECREASE-KEY: $O(1)$ con arreglo
- **Con montículo binario:** $O((V + E) \log V) = O(E \log V)$
  - EXTRACT-MIN: $O(\log V)$ por operación, $O(V \log V)$ total
  - DECREASE-KEY: $O(\log V)$ por operación, $O(E \log V)$ total
- **Con montículo de Fibonacci:** $O(E + V \log V)$
  - EXTRACT-MIN: $O(\log V)$ amortizado
  - DECREASE-KEY: $O(1)$ amortizado

Demostración de Correctitud

Dijkstra calcula correctamente $\delta(s, v)$ para todo $v \in V$ si todos los pesos de aristas son no negativos.

Por inducción en el número de vértices extraídos de Q.

Invariante: Cuando un vértice u es extraído de Q, $u.d = \delta(s, u)$.

Caso base: s es el primer vértice extraído. $s.d = 0 = \delta(s, s)$.

Paso inductivo: Supongamos que todos los vértices extraídos antes de u tienen $d = \delta$. Sea u el próximo vértice extraído. Si $u.d > \delta(s, u)$, existe un camino más corto su. Sea x el primer vértice en ese camino que no ha sido extraído, y sea y su predecesor en el camino (que sí ha sido extraído). Entonces:

$$\delta(s, u) \geq \delta(s, y) + w(y, x) \geq y.d + w(y, x) \geq x.d$$

La última desigualdad se debe a que relajamos (y, x) cuando y fue extraído. Como x no ha sido extraído y $u.d > \delta(s, u) \geq x.d$, tendríamos $u.d > x.d$, contradiciendo que u fue elegido como el mínimo.

Limitaciones y Advertencias

Dijkstra **NO** funciona con pesos negativos. En presencia de aristas negativas, un vértice puede necesitar ser actualizado después de ser extraído.

[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[-¿] (s) – node[above] 3 (a); [-¿] (s) – node[below] 4 (b); [-¿] (a) – node[above] 1 (c); [-¿] (b) – node[below] -2 (c);

Dijkstra extrae s, luego a (3), actualiza c a 4, luego extrae b (4), pero c ya fue extraído con valor 4 cuando el camino $s \to b \to c$ tiene peso $4+(-2) = 2$, que es menor.

Algoritmo de Bellman-Ford

Descripción General

Bellman-Ford encuentra caminos mínimos desde una fuente s a todos los demás vértices, incluso con pesos negativos, y detecta ciclos de peso negativo.

Pseudocódigo Detallado

[language=C, caption=Algoritmo de Bellman-Ford] BELLMAN-FORD(G, w, s) // Inicialización for each vertex v in G.V v.d = INFINITY v.parent = NIL s.d = 0

// Relajar todas las aristas —V—-1 veces for i = 1 to —V—-1 for each edge (u, v) in G.E if v.d ¿ u.d + w(u, v) v.d = u.d + w(u, v) v.parent = u

// Verificar ciclos de peso negativo for each edge (u, v) in G.E if v.d ¿ u.d + w(u, v) return FALSE // ciclo de peso negativo detectado

return TRUE

Ejemplo Paso a Paso

Consideremos el siguiente grafo con una arista negativa:

[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[-¿] (s) – node[above left] 4 (a); [-¿] (s) – node[below left] 2 (b); [-¿] (a) – node[above] 1 (c); [-¿] (b) – node[below] 5 (c); [-¿] (b) – node[right] -3 (a);

Inicialización:

$$d[s] = 0,\ d[a] = \infty,\ d[b] = \infty,\ d[c] = \infty$$

Iteración 1 (relajar todas las aristas):

- (s, a): $d[a] = \min(\infty, 0 + 4) = 4$
- (s, b): $d[b] = \min(\infty, 0 + 2) = 2$
- (a, c): $d[c] = \min(\infty, 4 + 1) = 5$
- (b, c): $d[c] = \min(5, 2 + 5 = 7) = 5$
- (b, a): $d[a] = \min(4, 2 + (-3) = -1) = -1$

Después de iteración 1: $d[a] = -1$, $d[b] = 2$, $d[c] = 5$

Iteración 2:

- (s, a): $d[a] = \min(-1, 0 + 4 = 4) = -1$
- (s, b): $d[b] = \min(2, 0 + 2 = 2) = 2$
- (a, c): $d[c] = \min(5, -1 + 1 = 0) = 0$
- (b, c): $d[c] = \min(0, 2 + 5 = 7) = 0$
- (b, a): $d[a] = \min(-1, 2 + (-3) = -1) = -1$

Después de iteración 2: $d[a] = -1$, $d[b] = 2$, $d[c] = 0$

Iteración 3 ($|V| - 1 = 3$ iteraciones):

- (a, c): $d[c] = \min(0, -1 + 1 = 0) = 0$

Sin cambios.

Verificación de ciclos negativos: Ninguna arista puede relajarse más.

Resultados finales: $d[s] = 0$, $d[a] = -1$, $d[b] = 2$, $d[c] = 0$

Implementación Completa

[language=C, caption=Bellman-Ford con detección de ciclos] typedef struct int u, v; int weight; Edge;

int bellmanFord(Graph* G, int s, int* dist, int* parent) int V = G-¿V; int E = G-¿E; Edge* edges = G-¿edges;

// Inicialización for (int i = 0; i ¡ V; i++) dist[i] = INF; parent[i] = -1; dist[s] = 0;

// Relajar —V—-1 veces for (int i = 1; i ¡ V; i++) int updated = 0; for (int j = 0; j ¡ E; j++) int u = edges[j].u; int v = edges[j].v; int w = edges[j].weight; if (dist[u] != INF dist[v] ¿ dist[u] + w) dist[v] = dist[u] + w; parent[v] = u; updated = 1; // Optimización: si no hubo actualizaciones, terminar temprano if (!updated) break;

// Verificar ciclos de peso negativo for (int j = 0; j ¡ E; j++) int u = edges[j].u; int v = edges[j].v; int w = edges[j].weight; if (dist[u] != INF dist[v] ¿ dist[u] + w) return 0; // ciclo negativo detectado return 1; // éxito

Análisis de Complejidad

- Tiempo: $O(VE)$
  - Inicialización: $O(V)$
  - $|V| - 1$ iteraciones sobre $|E|$ aristas: $O(VE)$
  - Verificación de ciclos: $O(E)$
- Espacio: $O(V)$ para distancias y padres

Demostración de Correctitud

Bellman-Ford calcula correctamente $\delta(s, v)$ para todo v si no hay ciclos de peso negativo alcanzables desde s. Si existe tal ciclo, devuelve FALSE.

Lema 1: Después de i iteraciones, $d[v]$ es el peso del camino mínimo de s a v con a lo sumo i aristas.

Demostración por inducción en i: Base $i = 0$ trivial. Paso inductivo: la i-ésima iteración relaja todas las aristas, extendiendo caminos de hasta i aristas a caminos de hasta $i + 1$ aristas.

Lema 2: Si no hay ciclos negativos, todo camino mínimo tiene a lo sumo $|V| - 1$ aristas (es simple). Por lo tanto, después de $|V| - 1$ iteraciones, $d[v] = \delta(s, v)$.

Detección de ciclos: Si después de $|V| - 1$ iteraciones aún se puede relajar una arista (u, v), entonces existe un camino de s a v con más de $|V| - 1$ aristas y peso menor, lo que implica un ciclo negativo.

Detección de Ciclos de Peso Negativo

[language=C, caption=Encontrar un ciclo de peso negativo] int findNegativeCycle(Graph* G, int s, int* cycle) int V = G-¿V; int* dist = malloc(V * sizeof(int)); int* parent = malloc(V * sizeof(int));

// Ejecutar Bellman-Ford bellmanFord(G, s, dist, parent);

// Buscar arista que aún puede relajarse for (int j = 0; j ¡ G-¿E; j++) int u = G-¿edges[j].u; int v = G-¿edges[j].v; int w = G-¿edges[j].weight; if (dist[u] != INF dist[v] ¿ dist[u] + w) // Encontramos un ciclo negativo // Reconstruir el ciclo int visited[V]; memset(visited, 0, sizeof(visited)); int node = u; int cycleLen = 0; while (!visited[node]) visited[node] = 1; cycle[cycleLen++] = node; node = parent[node]; // cycle contiene el ciclo negativo return 1; return 0;

Ejemplo de Ciclo de Peso Negativo

[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[-¿] (s) – node[above] 2 (a); [-¿] (s) – node[below] 3 (b); [-¿] (a) – node[above] 1 (c); [-¿] (b) – node[below] 2 (c); [-¿] (c) – node[right] -6 (a);
// ciclo negativo: a-¿c-¿a tiene peso -5

Ciclo $a \to c \to a$: $1 + (-6) = -5$. Bellman-Ford detectará que después de 3 iteraciones aún se puede relajar.

Comparación entre Dijkstra y Bellman-Ford

Característica	**Dijkstra**	**Bellman-Ford**
Pesos negativos	No	Sí
Detección de ciclos negativos	No	Sí
Complejidad temporal	$O(E \log V)$	$O(VE)$
Complejidad espacial	$O(V)$	$O(V)$
Estructura de datos	Cola de prioridad	Arreglo simple
Tipo de algoritmo	Greedy	Programación dinámica

Cuándo usar cada uno

- **Dijkstra:** Grafos con pesos no negativos, especialmente si son dispersos ($E \ll V^2$). Es más rápido en la práctica.
- **Bellman-Ford:** Grafos con pesos negativos, o cuando se necesita detectar ciclos negativos. También útil para grafos densos donde la implementación simple de Dijkstra ($O(V^2)$) puede ser competitiva.

Aplicaciones Prácticas

Navegación GPS

- Modelar calles como aristas, intersecciones como vértices.
- Dijkstra es ideal porque las distancias son positivas.
- Optimizaciones: A* (heurística) para acelerar.

Enrutamiento en Redes

- Protocolos como OSPF usan Dijkstra.
- RIP usa Bellman-Ford (detección de bucles).

Finanzas y Arbitraje

- Detección de oportunidades de arbitraje en mercados de divisas.
- Modelar tipos de cambio como logaritmos negativos.
- Ciclo de peso negativo indica oportunidad de arbitraje.

[Arbitraje de divisas] Si $1USD = 0.85EUR$, $1EUR = 1.20CHF$, $1CHF = 1.05USD$, entonces:

$$0.85 \times 1.20 \times 1.05 \approx 1.071 > 1$$

Hay arbitraje. Bellman-Ford puede detectarlo.

Ejercicios Resueltos (20 Ejercicios)

Ejecute Dijkstra desde el vértice s en el grafo con aristas: $(s, a, 5)$, $(s, b, 3)$, $(a, c, 2)$, $(b, c, 1)$, $(c, d, 4)$, $(b, d, 6)$.

1. Inicial: $d[s] = 0$, $d[a] = \infty$, $d[b] = \infty$, $d[c] = \infty$, $d[d] = \infty$
2. Extraer s: actualizar $d[a] = 5$, $d[b] = 3$
3. Extraer b: actualizar $d[c] = \min(\infty, 3{+}1 = 4)$, $d[d] = \min(\infty, 3{+}6 = 9)$
4. Extraer c: actualizar $d[d] = \min(9, 4 + 4 = 8)$
5. Extraer a: actualizar $d[c] = \min(4, 5 + 2 = 7)$ sigue 4
6. Extraer d: fin

Resultados: $d[s] = 0$, $d[a] = 5$, $d[b] = 3$, $d[c] = 4$, $d[d] = 8$.

Ejecute Bellman-Ford en el mismo grafo.

1. Inicial: $d[s] = 0$, otros ∞
2. Iteración 1: $d[a] = 5$, $d[b] = 3$, $d[c] = 4$ (vía b), $d[d] = 9$ (vía b)
3. Iteración 2: $d[d] = \min(9, 4 + 4 = 8)$

4. Iteración 3: sin cambios

Mismos resultados.

Demuestre que Dijkstra falla si hay una arista negativa. Contraejemplo:

[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (t) at (4,0) t;
[-¿] (s) – node[above] 3 (a); [-¿] (s) – node[below] 4 (b); [-¿] (a) – node[above] 1 (t); [-¿] (b) – node[below] -2 (t);

Dijkstra extrae s, luego a (3), actualiza t a 4, luego extrae b (4), pero t ya está extraído con valor 4 cuando el camino $s \to b \to t$ pesa $4 + (-2) = 2$, que es menor.

¿Qué hace Bellman-Ford si detecta un ciclo de peso negativo? Si Bellman-Ford detecta un ciclo de peso negativo, significa que existen caminos de peso arbitrariamente pequeño (hacia $-\infty$) para los vértices alcanzables desde ese ciclo. El algoritmo retorna FALSE, indicando que no se pueden calcular distancias finitas.

En aplicaciones como arbitraje financiero, esto indica una oportunidad de ganancia infinita.

Implemente Dijkstra para un grafo representado con matriz de adyacencia. void dijkstraMatrix(int** graph, int V, int s) int* dist = malloc(V * sizeof(int)); int* visited = malloc(V * sizeof(int));

for (int i = 0; i ¡ V; i++) dist[i] = INF; visited[i] = 0; dist[s] = 0;

for (int count = 0; count ¡ V - 1; count++) // Encontrar vértice no visitado con distancia mínima int u = -1; int minDist = INF; for (int i = 0; i ¡ V; i++) if (!visited[i] dist[i] ¡ minDist) minDist = dist[i]; u = i; if (u == -1) break; visited[u] = 1;

// Actualizar distancias de los vecinos for (int v = 0; v ¡ V; v++) if (graph[u][v] !visited[v] dist[v] ¿ dist[u] + graph[u][v]) dist[v] = dist[u] + graph[u][v]; Complejidad: $O(V^2)$.

Ejecute Bellman-Ford en un grafo con ciclo negativo. Grafo: $s \to a$ peso 1, $a \to b$ peso 1, $b \to a$ peso -3.

1. Inicial: $d[s] = 0$, $d[a] = \infty$, $d[b] = \infty$

2. Iteración 1: $d[a] = 1$, $d[b] = 2$

3. Iteración 2: $d[a] = \min(1, 2+(-3) = -1) = -1$, $d[b] = \min(2, -1+1 = 0) = 0$

4. Iteración 3: $d[a] = \min(-1, 0 + (-3) = -3) = -3$, $d[b] = \min(0, -3 + 1 = -2) = -2$

Después de 3 iteraciones, aún se puede relajar. El algoritmo detecta ciclo negativo y retorna FALSE.

¿Por qué Dijkstra requiere que los pesos sean no negativos? La corrección de Dijkstra se basa en que cuando un vértice es extraído de la cola, su distancia es definitiva. Con pesos negativos, un camino más largo (en número de aristas) podría tener menor peso total, y ese camino podría llegar a un vértice después de que ya fue extraído.

Formalmente, la desigualdad $d[u] \leq d[x] + w(x, u)$ no se garantiza si hay pesos negativos, porque $w(x, u)$ podría ser negativo y hacer que $d[u] > d[x] + w(x, u)$ incluso si x aún no fue procesado.

Demuestre que Bellman-Ford puede optimizarse para terminar antes si no hay cambios en una iteración. Si en una iteración completa ninguna arista provoca una relajación, entonces $d[v]$ ya es $\delta(s, v)$ para todo v, y las iteraciones restantes no cambiarán nada. Esta optimización es especialmente útil cuando el grafo tiene caminos cortos.

En el peor caso, igual se requieren $|V| - 1$ iteraciones, pero en la práctica puede ser mucho más rápido.

Aplique Dijkstra al grafo con aristas: $(1, 2, 4)$, $(1, 3, 2)$, $(2, 3, 1)$, $(2, 4, 5)$, $(3, 4, 8)$, $(3, 5, 10)$, $(4, 5, 2)$.

1. $d[1] = 0$, otros ∞

2. Extraer 1: $d[2] = 4$, $d[3] = 2$

3. Extraer 3: $d[2] = \min(4, 2+1 = 3)$, $d[4] = 2+8 = 10$, $d[5] = 2+10 = 12$

4. Extraer 2: $d[4] = \min(10, 3 + 5 = 8)$, $d[5] = \min(12, 3+?)$ sin arista

5. Extraer 4: $d[5] = \min(12, 8 + 2 = 10)$

6. Extraer 5: fin

Resultados: $d[1] = 0$, $d[2] = 3$, $d[3] = 2$, $d[4] = 8$, $d[5] = 10$.

¿Cómo se puede usar Bellman-Ford para encontrar el camino mínimo en un grafo con pesos negativos pero sin ciclos negativos? Simplemente ejecutamos Bellman-Ford normalmente. Después de $|V| - 1$ iteraciones, $d[v]$ contiene $\delta(s, v)$. El algoritmo retorna TRUE indicando que no hay ciclos negativos. Luego se pueden reconstruir los caminos usando los punteros parent.

Es la solución estándar para grafos con pesos negativos, como los que aparecen en problemas de arbitraje financiero.

Ejercicios Propuestos (20 Ejercicios)

1. Ejecute Dijkstra desde el vértice A en un grafo de 6 vértices.
2. Demuestre que Dijkstra es correcto para pesos no negativos por inducción.
3. Ejecute Bellman-Ford en un grafo con 4 vértices y una arista negativa.
4. Implemente Dijkstra con montículo de Fibonacci.
5. ¿Qué es la optimización de cola de prioridad en Dijkstra?
6. Encuentre un ciclo de peso negativo en un grafo dado.
7. Demuestre que Bellman-Ford detecta ciclos negativos correctamente.
8. ¿Cómo se puede usar Bellman-Ford para encontrar el camino mínimo con restricciones de número de aristas?
9. Compare la complejidad de Dijkstra y Bellman-Ford en grafos densos.
10. Implemente la detección de ciclos negativos con reconstrucción del ciclo.
11. Ejecute Dijkstra en un grafo con pesos negativos y muestre el error.
12. ¿Qué es el algoritmo de D'Esopo-Pape y cuándo se usa?
13. Demuestre que si no hay ciclos negativos, Bellman-Ford termina en $O(VE)$.
14. Aplique Bellman-Ford a un grafo lineal de 5 vértices.
15. ¿Cómo se puede paralelizar Dijkstra?
16. ¿Qué es el problema de caminos mínimos con restricciones de recursos?
17. Implemente Bellman-Ford con detección temprana de convergencia.
18. Ejecute ambos algoritmos en un grafo aleatorio y compare tiempos.
19. ¿Qué es el algoritmo de Johnson para grafos dispersos con pesos negativos?
20. Investigue aplicaciones de Dijkstra en sistemas de navegación.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de caminos mínimos desde una fuente única (SSSP)
- El algoritmo de Dijkstra: greedy, requiere pesos no negativos, $O(E \log V)$
- El algoritmo de Bellman-Ford: programación dinámica, permite pesos negativos, $O(VE)$
- Cómo detectar ciclos de peso negativo con Bellman-Ford
- Aplicaciones prácticas en navegación, redes y finanzas

Bibliografía de la Unidad

- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87-90.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 24.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.
- Ford, L. R., & Fulkerson, D. R. (1962). *Flows in Networks*. Princeton University Press.

Unidad VI: Algoritmo de Floyd-Warshall - Caminos Mínimos entre Todos los Pares

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de caminos mínimos entre todos los pares de vértices (APSP).
- Implementar y analizar el algoritmo de Floyd-Warshall.
- Aplicar Floyd-Warshall a grafos con pesos positivos y negativos (sin ciclos negativos).
- Detectar ciclos de peso negativo utilizando Floyd-Warshall.
- Adaptar Floyd-Warshall para calcular el cierre transitivo de un grafo.
- Comparar Floyd-Warshall con otras soluciones al problema APSP.

Introducción al Problema APSP

Definición Formal

Dado un grafo dirigido ponderado $G = (V, E)$ con función de peso $w : E \to R$, encontrar para todo par de vértices (i, j) la distancia mínima $\delta(i, j)$.

Enfoques para APSP

- **Fuerza bruta:** Ejecutar un algoritmo de fuente única desde cada vértice.

  - Con Dijkstra (pesos no negativos): $O(V \cdot E \log V) = O(V^3 \log V)$ en grafos densos
  - Con Bellman-Ford: $O(V^2 E) = O(V^4)$ en grafos densos

- **Floyd-Warshall:** $O(V^3)$ tiempo, $O(V^2)$ espacio.
- **Johnson:** $O(VE \log V)$ para grafos dispersos.

Representación Matricial

El grafo se representa mediante una matriz de adyacencia W de tamaño $n \times n$:

$$W[i][j] = \{ 0\ sii = jw(i,j)si(i,j) \in E\infty sii \neq jy(i,j) \notin E$$

Algoritmo de Floyd-Warshall

Descripción General

Floyd-Warshall utiliza programación dinámica. Sea $d^{(k)}[i][j]$ la distancia mínima de i a j usando solo vértices intermedios del conjunto $\{1, 2, \ldots, k\}$.

Ecuación de Recurrencia

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$$

con caso base $d^{(0)}[i][j] = W[i][j]$.

[scale=0.7] [circle, draw] (i) at (0,0) i; [circle, draw] (j) at (4,0) j; [circle, draw] (k) at (2,1.5) k;
[-¿, thick] (i) – node[below] camino directo (j); [-¿, thick, blue] (i) – node[left] $i \to k$ (k); [-¿, thick, blue] (k) – node[right] $k \to j$ (j);

Pseudocódigo

[language=C, caption=Algoritmo de Floyd-Warshall] FLOYD-WARSHALL(W, n) // Inicialización D = W // copia de la matriz de pesos for i = 1 to n D[i][i] = 0
// Algoritmo principal for k = 1 to n for i = 1 to n for j = 1 to n if D[i][k] + D[k][j] ¡ D[i][j] D[i][j] = D[i][k] + D[k][j] next[i][j] = k // para reconstrucción
return D

Ejemplo Paso a Paso

Consideremos el siguiente grafo:

[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,1.5) 2; [circle, draw] (3) at (2,-1.5) 3; [circle, draw] (4) at (4,0) 4;
[-¿] (1) – node[above] 3 (2); [-¿] (1) – node[below] 8 (3); [-¿] (2) – node[above] 2 (4); [-¿] (3) – node[below] 1 (4); [-¿] (2) – node[right] 5 (3);

Matriz inicial $D^{(0)}$:

$$(\,0\,)\,38\infty\infty052\infty\infty01\infty\infty\infty0$$

Para $k = 1$: Usamos el vértice 1 como intermedio.

- $i = 2$, $j = 3$: $D[2][1] + D[1][3] = \infty + 8 = \infty$, no cambia
- $i = 2$, $j = 4$: $\infty + \infty = \infty$, no cambia
- $i = 3$, $j = 2$: $\infty + \infty = \infty$, no cambia
- $i = 3$, $j = 4$: $\infty + \infty = \infty$, no cambia

$D^{(1)} = D^{(0)}$

Para $k = 2$: Usamos el vértice 2 como intermedio.

- $i = 1$, $j = 3$: $D[1][2] + D[2][3] = 3 + 5 = 8$, no mejora (8 vs 8)
- $i = 1$, $j = 4$: $D[1][2] + D[2][4] = 3 + 2 = 5$, mejora ($\infty \to 5$)
- $i = 3$, $j = 1$: $D[3][2] + D[2][1] = \infty + \infty = \infty$, no cambia
- $i = 3$, $j = 4$: $D[3][2] + D[2][4] = \infty + 2 = \infty$, no cambia

$$D^{(2)} = (\,0\,)\,385\infty052\infty\infty01\infty\infty\infty0$$

Para $k = 3$: Usamos el vértice 3 como intermedio.

- $i = 1$, $j = 4$: $D[1][3] + D[3][4] = 8 + 1 = 9$, no mejora (5 vs 9)
- $i = 2$, $j = 4$: $D[2][3] + D[3][4] = 5 + 1 = 6$, no mejora (2 vs 6)
- $i = 1$, $j = 2$: $D[1][3] + D[3][2] = 8 + \infty = \infty$, no cambia

$D^{(3)} = D^{(2)}$

Para $k = 4$: Usamos el vértice 4 como intermedio.

- $i = 1$, $j = 2$: $D[1][4] + D[4][2] = 5 + \infty = \infty$
- $i = 1$, $j = 3$: $D[1][4] + D[4][3] = 5 + \infty = \infty$
- $i = 2$, $j = 1$: $D[2][4] + D[4][1] = 2 + \infty = \infty$
- $i = 2$, $j = 3$: $D[2][4] + D[4][3] = 2 + \infty = \infty$
- $i = 3$, $j = 1$: $D[3][4] + D[4][1] = 1 + \infty = \infty$
- $i = 3$, $j = 2$: $D[3][4] + D[4][2] = 1 + \infty = \infty$

Matriz final:

$$D = (\,0\,)\,385\infty052\infty\infty01\infty\infty\infty0$$

Implementación Completa

[language=C, caption=Implementación de Floyd-Warshall] include ¡stdio.h¿ include ¡limits.h¿

define INF INT$_MAXdefineMAX100$

void floydWarshall(int graph[MAX][MAX], int n) int dist[MAX][MAX]; int next[MAX][MAX];

// Inicializar matrices for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) dist[i][j] = graph[i][j]; if (graph[i][j] != INF i != j) next[i][j] = j; else next[i][j] = -1; dist[i][i] = 0;

// Algoritmo principal for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (dist[i][k] != INF dist[k][j] != INF dist[i][k] + dist[k][j] ¡ dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j]; next[i][j] = next[i][k];

// Detectar ciclos de peso negativo for (int i = 0; i ¡ n; i++) if (dist[i][i] ¡ 0) printf("Ciclo de peso negativo detectado"); return;

// Imprimir resultados printf("Matriz de distancias minimas:"); for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (dist[i][j] == INF) printf("INF "); else printf(" printf("");

// Reconstrucción de caminos void printPath(int next[MAX][MAX], int i, int j) if (next[i][j] == -1) printf("No hay camino"); return; printf("while (i != j) i = next[i][j]; printf(" -¿ printf("");

Reconstrucción de Caminos

Para reconstruir el camino de i a j, se mantiene una matriz $next[i][j]$ que indica el siguiente vértice en el camino.

[language=C, caption=Reconstrucción del camino] void printShortest-Path(int next[MAX][MAX], int i, int j) if (next[i][j] == -1) printf("No existe camino de return;

printf("Camino de int u = i; while (u != j) u = next[u][j]; printf(" -¿ printf("");

Análisis de Complejidad

Tiempo

- Tres bucles anidados: $O(V^3)$
- Cada iteración realiza una suma y una comparación: $O(1)$
- Total: $O(V^3)$

Espacio

- Matriz de distancias: $O(V^2)$
- Matriz de siguiente vértice (opcional): $O(V^2)$
- Total: $O(V^2)$

Comparación con Alternativas

Algoritmo	**Tiempo**	**Espacio**
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Dijkstra (desde cada vértice)	$O(VE \log V)$	$O(V + E)$
Bellman-Ford (desde cada vértice)	$O(V^2E)$	$O(V)$
Johnson	$O(VE \log V)$	$O(V + E)$

Detección de Ciclos de Peso Negativo

Método

Floyd-Warshall puede detectar ciclos de peso negativo verificando la diagonal de la matriz de distancias después de ejecutar el algoritmo:

$$SiD[i][i] < 0 para algúni, existeunciclodepesonegativo.$$

[language=C, caption=Detección de ciclos negativos] int detectNegative-Cycle(int dist[MAX][MAX], int n) for (int i = 0; i ¡ n; i++) if (dist[i][i] ¡ 0) printf("Ciclo de peso negativo encontrado en el vertice return 1; return 0;

Ejemplo de Ciclo Negativo

[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,1) 2; [circle, draw] (3) at (2,-1) 3;
[-¿] (1) – node[above] 1 (2); [-¿] (2) – node[right] 1 (3); [-¿] (3) – node[below] -3 (1);

Ciclo $1 \to 2 \to 3 \to 1$ tiene peso $1 + 1 + (-3) = -1$.
Después de Floyd-Warshall, $D[1][1]$ será -1, detectando el ciclo.

Cierre Transitivo

Definición

El cierre transitivo de un grafo dirigido es una matriz T donde $T[i][j] = 1$ si existe un camino de i a j, y 0 en caso contrario.

Algoritmo

[language=C, caption=Cierre transitivo con Floyd-Warshall] void transitiveClosure(int graph[MAX][MAX], int n) int reach[MAX][MAX];
// Inicializar for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) reach[i][j] = (i == j) —— graph[i][j];
// Algoritmo de Floyd-Warshall para cierre transitivo for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) reach[i][j] = reach[i][j] —— (reach[i][k] reach[k][j]);
// Imprimir matriz de alcanzabilidad printf(”Matriz de cierre transitivo:”); for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) printf(” printf(””);

Ejemplo

[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,0) 2; [circle, draw] (3) at (4,0) 3; [circle, draw] (4) at (6,0) 4;
[-¿] (1) – (2); [-¿] (2) – (3); [-¿] (3) – (4); [-¿] (4) – (2);

Cierre transitivo:

(1) 111011101110111

Variantes y Optimizaciones

Floyd-Warshall para Matrices de Distancias con Pesos Grandes

[language=C, caption=Manejo de INF con prevención de overflow] for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) if (dist[i][k] == INF) continue; for (int j = 0; j ¡ n; j++) if (dist[k][j] == INF) continue; if (dist[i][k] + dist[k][j] ¡ dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j];

Floyd-Warshall con Registro de Caminos

[language=C, caption=Versión con reconstrucción de caminos] void floyd-WarshallWithPath(int graph[MAX][MAX], int n) int dist[MAX][MAX]; int path[MAX][MAX];

// Inicialización for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) dist[i][j] = graph[i][j]; path[i][j] = (graph[i][j] != INF i != j) ? j : -1; dist[i][i] = 0; path[i][i] = i;

// Algoritmo principal for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (dist[i][k] != INF dist[k][j] != INF dist[i][k] + dist[k][j] ¡ dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j]; path[i][j] = path[i][k];

// Reconstrucción de caminos for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (i != j dist[i][j] != INF) printf("Camino int u = i; while (u != j) u = path[u][j]; printf(" -¿ printf(" (distancia

Ejercicios Resueltos (20 Ejercicios)

Ejecute Floyd-Warshall en el grafo con 3 vértices: aristas $(1, 2, 4)$, $(1, 3, 11)$, $(2, 3, 2)$, $(3, 1, 3)$. Matriz inicial:

$$D^{(0)} = (\,0\,)\,411302\infty\infty0$$

$k = 1$:

- $(2, 3)$: $3 + 11 = 14$ vs 2, sigue 2
- $(3, 2)$: $\infty + 4 = \infty$, no cambia

$k = 2$:

- $(1, 3)$: $4 + 2 = 6$ vs 11, mejora a 6

- $(3,1)$: $\infty + 3 = \infty$, no cambia
- $(3,3)$: $\infty + 2 = \infty$, no cambia

$$D^{(2)} = (\,0\,)\,46302\infty\infty0$$

$k = 3$:

- $(1,2)$: $6 + \infty = \infty$, no cambia
- $(2,1)$: $2 + \infty = \infty$, no cambia
- $(2,2)$: $2 + \infty = \infty$, no cambia

Matriz final:

$$D = (\,0\,)\,46302\infty\infty0$$

Demuestre que Floyd-Warshall calcula correctamente las distancias mínimas. Por inducción en k. La hipótesis inductiva es que después de la iteración k, $D[i][j]$ es la distancia mínima de i a j usando solo vértices intermedios en $\{1, \ldots, k\}$.

Caso base: $k = 0$, $D[i][j] = W[i][j]$, que es la distancia sin vértices intermedios (arista directa o infinito).

Paso inductivo: Supongamos cierto para $k - 1$. Para k, el camino mínimo de i a j usando vértices $\{1, \ldots, k\}$ o bien no usa k (entonces es el de $k - 1$) o bien usa k (entonces se descompone en $i \to k$ y $k \to j$). La recurrencia captura exactamente esto.

Al final, $k = n$, todos los vértices son intermedios permitidos, por lo que $D[i][j] = \delta(i, j)$.

¿Cómo detecta Floyd-Warshall ciclos de peso negativo? Si existe un ciclo de peso negativo, entonces se puede mejorar la distancia de un vértice a sí mismo. Inicialmente $D[i][i] = 0$. Si hay un ciclo negativo, al usar los vértices del ciclo como intermedios, $D[i][i]$ se volverá negativo. Por lo tanto, después del algoritmo, si algún $D[i][i] < 0$, hay un ciclo negativo.

Esto también implica que no se pueden calcular distancias finitas si hay ciclos negativos alcanzables.

Aplique Floyd-Warshall para encontrar el cierre transitivo del grafo con aristas $(1,2)$, $(2,3)$, $(3,1)$, $(3,4)$. Matriz de adyacencia inicial:

$$T^{(0)} = (\,1\,)\,100011010110001$$

$k = 1$: $T[3][2] = T[3][2] \vee (T[3][1] \wedge T[1][2]) = 0 \vee (1 \wedge 1) = 1$

$k = 2$: $T[1][3] = 0 \vee (1 \wedge 1) = 1$, $T[3][1] = 1 \vee (1 \wedge 0) = 1$, $T[3][4] = 1 \vee (1 \wedge 0) = 1$

$k = 3$: $T[1][4] = 0 \vee (1 \wedge 1) = 1$, $T[2][4] = 0 \vee (1 \wedge 1) = 1$

Matriz final (todos alcanzables desde 1,2,3):

$$T = (\,1\,)\,111111111110001$$

Compare la complejidad de Floyd-Warshall con ejecutar Dijkstra V veces.

- Floyd-Warshall: $O(V^3)$ siempre.
- Dijkstra V veces:
  - Con montículo binario: $O(V \cdot E \log V)$
  - En grafos densos ($E \approx V^2$): $O(V^3 \log V)$
  - En grafos dispersos ($E \approx V$): $O(V^2 \log V)$

Floyd-Warshall es competitivo en grafos densos y es más simple de implementar. Para grafos dispersos, Dijkstra V veces o Johnson pueden ser mejores.

¿Qué sucede si el grafo tiene un ciclo de peso negativo no alcanzable desde algunos vértices? Si el ciclo negativo no es alcanzable desde i, entonces $D[i][j]$ sigue siendo finito para los j alcanzables. La detección de ciclos negativos en $D[i][i]$ solo detecta ciclos alcanzables desde i. Para una detección global, se debe verificar la diagonal de toda la matriz.

En la práctica, si hay algún ciclo negativo en cualquier parte del grafo, las distancias no están bien definidas para algunos pares.

Implemente Floyd-Warshall para un grafo representado con lista de adyacencia. void floydWarshallFromAdjList(Graph* G) int n = G-¿V; int** dist = malloc(n * sizeof(int*)); for (int i = 0; i ¡ n; i++) dist[i] = malloc(n * sizeof(int)); for (int j = 0; j ¡ n; j++) dist[i][j] = (i == j) ? 0 : INF;

// Convertir lista de adyacencia a matriz for (int u = 0; u ¡ n; u++) Node* adj = G-¿adj[u]; while (adj != NULL) int v = adj-¿vertex; int w = adj-¿weight; dist[u][v] = w; adj = adj-¿next;

// Floyd-Warshall for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (dist[i][k] != INF dist[k][j] != INF dist[i][k] + dist[k][j] ¡ dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j];

¿Cómo se puede modificar Floyd-Warshall para encontrar el camino más largo (en lugar del más corto)? Para encontrar caminos más largos en un grafo sin ciclos positivos (DAG), se puede cambiar min por max e inicializar con $-\infty$ en lugar de ∞:

for (int k = 0; k ¡ n; k++) for (int i = 0; i ¡ n; i++) for (int j = 0; j ¡ n; j++) if (dist[i][k] != NEG$_I NF dist[k][j] != NEG_I NF dist[i][k] + dist[k][j] > dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j]$;

Si hay ciclos positivos, el problema es NP-hard.

Aplique Floyd-Warshall al grafo completo de 3 vértices con pesos: $(1, 2, 1)$, $(2, 3, 2)$, $(3, 1, 3)$, $(2, 1, 4)$, $(3, 2, 5)$, $(1, 3, 6)$. Matriz inicial:

$$D^{(0)} = (\,0\,)\,16402350$$

$k = 1$:

- $(2, 3)$: $4 + 6 = 10$ vs 2, sigue 2
- $(3, 2)$: $3 + 1 = 4$ vs 5, mejora a 4

$$D^{(1)} = (\,0\,)\,16402340$$

$k = 2$:

- $(1, 3)$: $1 + 2 = 3$ vs 6, mejora a 3
- $(3, 1)$: $4 + 4 = 8$ vs 3, sigue 3

$$D^{(2)} = (\,0\,)\,13402340$$

$k = 3$:

- $(1, 2)$: $3 + 4 = 7$ vs 1, sigue 1
- $(2, 1)$: $2 + 3 = 5$ vs 4, sigue 4

Matriz final:

$$D = (\,0\,)\,13402340$$

Ejercicios Propuestos (20 Ejercicios)

1. Ejecute Floyd-Warshall en un grafo de 4 vértices con aristas $(1, 2, 2)$, $(1, 3, 6)$, $(2, 3, 3)$, $(2, 4, 1)$, $(3, 4, 4)$.
2. Demuestre la corrección de Floyd-Warshall por inducción en k.
3. Implemente Floyd-Warshall con detección de ciclos negativos.

4. ¿Cómo se puede usar Floyd-Warshall para encontrar el diámetro de un grafo?

5. Aplique Floyd-Warshall a un grafo con un ciclo de peso negativo y detecte el ciclo.

6. Compare Floyd-Warshall con Johnson para grafos dispersos.

7. Implemente Floyd-Warshall para calcular el cierre transitivo.

8. ¿Qué es la desigualdad triangular y cómo la usa Floyd-Warshall?

9. Demuestre que si no hay ciclos negativos, Floyd-Warshall termina con distancias correctas.

10. Aplique Floyd-Warshall a un grafo de 5 vértices en forma de estrella.

11. ¿Cómo se puede paralelizar Floyd-Warshall?

12. Implemente Floyd-Warshall con reconstrucción de caminos.

13. ¿Qué es el problema de la ruta más confiable y cómo se adapta Floyd-Warshall?

14. Aplique Floyd-Warshall a un grafo donde todas las aristas tienen peso 1.

15. ¿Cómo se puede usar Floyd-Warshall para encontrar la cerradura transitiva de un DAG?

16. Implemente una versión optimizada que use solo una matriz.

17. ¿Qué es el algoritmo de Warshall y cómo se relaciona con Floyd-Warshall?

18. Aplique Floyd-Warshall a un grafo bipartito completo.

19. Demuestre que Floyd-Warshall funciona con pesos negativos si no hay ciclos negativos.

20. Investigue aplicaciones de Floyd-Warshall en análisis de redes sociales.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de caminos mínimos entre todos los pares (APSP)
- El algoritmo de Floyd-Warshall: programación dinámica, $O(V^3)$ tiempo, $O(V^2)$ espacio
- La recurrencia $d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$
- Cómo detectar ciclos de peso negativo
- Adaptación para calcular el cierre transitivo
- Comparación con otras soluciones al problema APSP

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 25.
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
- Warshall, S. (1962). A theorem on Boolean matrices. *Journal of the ACM*, 9(1), 11-12.

Unidad VII: Flujo en Redes - Teorema de Flujo Máximo - Corte Mínimo

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de flujo máximo en redes.
- Definir formalmente redes de flujo, cortes y flujos.
- Comprender y demostrar el Teorema de Flujo Máximo - Corte Mínimo.
- Implementar y analizar el algoritmo de Ford-Fulkerson.
- Implementar la variante de Edmonds-Karp usando BFS.
- Aplicar flujo máximo a problemas como matching bipartito y particionamiento de redes.

Introducción al Problema de Flujo Máximo

Definición de Red de Flujo

Una **red de flujo** $G = (V, E)$ es un grafo dirigido con las siguientes propiedades:

- Una **fuente** $s \in V$ (solo aristas salientes).
- Un **sumidero** $t \in V$ (solo aristas entrantes).
- Cada arista $(u, v) \in E$ tiene una **capacidad** $c(u, v) \geq 0$.

[scale=0.7] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c; [circle, draw] (t) at (6,0) t; [-¿] (s) – node[above] 16 (a); [-¿] (s) – node[below] 13 (b); [-¿] (a) – node[above] 12 (c); [-¿] (b) – node[below] 14 (c); [-¿] (c) – node[above] 20 (t); [-¿] (a) – node[right] 4 (b); [-¿] (b) – node[left] 9 (a);

Definición de Flujo

Un **flujo** es una función $f : V \times V \to R$ que satisface:

1. **Restricción de capacidad:** $0 \leq f(u, v) \leq c(u, v)$ para todo $u, v \in V$.
2. **Conservación del flujo:** Para todo $u \in V \setminus \{s, t\}$:
$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0$$
3. **Antisimetría:** $f(u, v) = -f(v, u)$.

El **valor del flujo** es:
$$|f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t)$$

Interpretación Física

- Las aristas son tuberías con capacidad máxima.
- El flujo es la cantidad de material que viaja por la tubería.
- La fuente produce material, el sumidero lo consume.
- En los nodos intermedios, lo que entra debe salir.

Cortes en Redes de Flujo

Definición de Corte

Un **corte** (S, T) es una partición de V tal que $s \in S$ y $t \in T$.

La **capacidad del corte** es:
$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

El **flujo a través del corte** es:
$$f(S, T) = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$$

[scale=0.7] [thick, dashed] (1.5,-1.5) – (1.5,2); at (0.5,1.5) S; at (3,1.5) T; [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (1,1) a; [circle, draw] (b) at (3,1) b; [circle, draw] (c) at (2,-1) c; [circle, draw] (t) at (4,0) t; [-¿] (s) – (a); [-¿] (a) – (b); [-¿] (b) – (t); [-¿] (s) – (c); [-¿] (c) – (t);

Propiedades de los Cortes

Para cualquier flujo f y cualquier corte (S, T):

$$|f| = f(S, T)$$

Por conservación del flujo, la suma de flujos que salen de S menos los que entran es igual al flujo neto de s a t.

Para cualquier flujo f y cualquier corte (S, T):

$$|f| \leq c(S, T)$$

Teorema de Flujo Máximo - Corte Mínimo

[Flujo Máximo - Corte Mínimo] En cualquier red de flujo, el valor máximo de un flujo es igual a la capacidad mínima de un corte.

$$\max_f |f| = \min_{(S,T)} c(S, T)$$

[Esbozo]

1. Si f es un flujo máximo, no hay caminos de aumento en la red residual.
2. Sea S el conjunto de vértices alcanzables desde s en la red residual.
3. Entonces $t \notin S$, por lo que (S, T) es un corte.
4. Para toda arista (u, v) con $u \in S$, $v \in T$, se tiene $f(u, v) = c(u, v)$ (de lo contrario (u, v) estaría en la red residual).
5. Por lo tanto, $|f| = f(S, T) = c(S, T)$.

Red Residual y Caminos de Aumento

Definición de Red Residual

Dado un flujo f, la **red residual** G_f tiene las mismas aristas más las aristas inversas:

$$c_f(u, v) = c(u, v) - f(u, v)$$

si $(u, v) \in E$, y $c_f(v, u) = f(u, v)$.

[scale=0.7] at (0,1) Original:; [circle, draw] (u) at (0,0) u; [circle, draw] (v) at (3,0) v; [-¿] (u) – node[above] 10 (v);
at (6,1) Con flujo 5:; [circle, draw] (u2) at (6,0) u; [circle, draw] (v2) at (9,0) v; [-¿] (u2) – node[above] 5 (v2); [-¿] (v2) – node[below] 5 (u2);

Camino de Aumento

Un **camino de aumento** es un camino desde s hasta t en la red residual G_f.

La **capacidad residual del camino** es:

$$c_f(p) = \min_{(u,v) \in p} c_f(u, v)$$

Aumento del Flujo

Si encontramos un camino de aumento p, podemos aumentar el flujo:

$$f'(u, v) = f(u, v) + c_f(p) para (u, v) \in p$$

$$f'(v, u) = f(v, u) - c_f(p) para (v, u) \in p$$

Algoritmo de Ford-Fulkerson

Pseudocódigo

[language=C, caption=Algoritmo de Ford-Fulkerson] FORD-FULKERSON(G, s, t) for each edge (u,v) in G.E f(u,v) = 0 f(v,u) = 0

while exists path p from s to t in G$_f c_f(p) = min c_f(u, v) : (u, v) in p for each edge (u, v) in p f($u
$f(u, v) + c_f(p) f(v, u) = f(v, u) - c_f(p)$

return f

Ejemplo Paso a Paso

Consideremos la siguiente red:

[scale=0.7] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (t) at (4,0) t;
[-¿] (s) – node[above] 16 (a); [-¿] (s) – node[below] 13 (b); [-¿] (a) – node[above] 12 (t); [-¿] (b) – node[below] 14 (t); [-¿] (a) – node[right] 4 (b); [-¿] (b) – node[left] 9 (a);

Paso 1: Inicializar flujo $f = 0$.
Paso 2: Encontrar camino de aumento. Por ejemplo, $p_1 = s \to a \to t$.

- Capacidades residuales: $c_f(s, a) = 16$, $c_f(a, t) = 12$, $c_f(p_1) = 12$
- Actualizar: $f(s, a) = 12$, $f(a, t) = 12$, $f(a, s) = -12$, $f(t, a) = -12$

Flujo actual: $|f| = 12$.
Paso 3: Otro camino, $p_2 = s \to b \to t$.

- Capacidades residuales: $c_f(s, b) = 13$, $c_f(b, t) = 14$, $c_f(p_2) = 13$
- Actualizar: $f(s, b) = 13$, $f(b, t) = 13$, $f(b, s) = -13$, $f(t, b) = -13$

Flujo actual: $|f| = 12 + 13 = 25$.
Paso 4: Otro camino, $p_3 = s \to a \to b \to t$.

- Capacidades residuales actuales: $c_f(s, a) = 16 - 12 = 4$, $c_f(a, b) = 4$ (original $a \to b$), $c_f(b, t) = 14 - 13 = 1$
- $c_f(p_3) = \min(4, 4, 1) = 1$
- Actualizar: $f(s, a) = 13$, $f(a, b) = 1$, $f(b, t) = 14$

Flujo actual: $|f| = 25 + 1 = 26$.
Paso 5: No hay más caminos de aumento desde s a t en la red residual.
Flujo máximo: $|f| = 26$.

Implementación Completa

[language=C, caption=Implementación de Ford-Fulkerson] include ¡stdio.h¿ include ¡stdlib.h¿ include ¡limits.h¿ include ¡string.h¿

define MAX$_V$100$defineINFINT_MAX$

typedef struct int cap[MAX$_V$][MAX_V]; $intn; Graph$;

// DFS para encontrar camino de aumento int dfs(Graph* G, int s, int t, int* parent, int* visited) visited[s] = 1; if (s == t) return 1;

for (int v = 0; v ¡ G-¿n; v++) if (!visited[v] G-¿cap[s][v] ¿ 0) parent[v] = s; if (dfs(G, v, t, parent, visited)) return 1; return 0;

int fordFulkerson(Graph* G, int s, int t) int parent[MAX$_V$]; $intmaxFlow =$ 0;

while (1) int visited[MAX$_V$] = 0; $if(!dfs(G, s, t, parent, visited))break$;

// Encontrar capacidad residual minima en el camino int pathFlow = INF; for (int v = t; v != s; v = parent[v]) int u = parent[v]; if (G-¿cap[u][v] ¡ pathFlow) pathFlow = G-¿cap[u][v];

// Actualizar capacidades residuales for (int v = t; v != s; v = parent[v]) int u = parent[v]; G-¿cap[u][v] -= pathFlow; G-¿cap[v][u] += pathFlow;
maxFlow += pathFlow;
return maxFlow;

Algoritmo de Edmonds-Karp

Descripción

Edmonds-Karp es una implementación de Ford-Fulkerson que utiliza BFS para encontrar caminos de aumento, garantizando que se elija siempre el camino más corto (en número de aristas).

Pseudocódigo

[language=C, caption=Algoritmo de Edmonds-Karp] int bfs(Graph* G, int s, int t, int* parent) int visited[MAX$_V$] = 0; $intqueue[MAX_V], front = 0, rear = 0$;
visited[s] = 1; queue[rear++] = s;
while (front ¡ rear) int u = queue[front++];
for (int v = 0; v ¡ G-¿n; v++) if (!visited[v] G-¿cap[u][v] ¿ 0) visited[v] = 1; parent[v] = u; if (v == t) return 1; queue[rear++] = v; return 0;
int edmondsKarp(Graph* G, int s, int t) int parent[MAX$_V$]; $intmaxFlow = 0$;
while (bfs(G, s, t, parent)) int pathFlow = INF; for (int v = t; v != s; v = parent[v]) int u = parent[v]; if (G-¿cap[u][v] ¡ pathFlow) pathFlow = G-¿cap[u][v];
for (int v = t; v != s; v = parent[v]) int u = parent[v]; G-¿cap[u][v] -= pathFlow; G-¿cap[v][u] += pathFlow;
maxFlow += pathFlow;
return maxFlow;

Análisis de Complejidad

- Cada BFS toma $O(E)$ tiempo.
- Cada arista puede volverse crítica $O(V)$ veces.
- Número total de aumentos: $O(VE)$.
- Complejidad total: $O(VE^2)$.

Demostración de Complejidad

Edmonds-Karp realiza $O(VE)$ aumentos.

[Esbozo] Cuando una arista (u, v) se vuelve crítica, la distancia de s a u aumenta en al menos 2. Como la distancia máxima es $V - 1$, cada arista puede volverse crítica $O(V)$ veces.

Teorema de Flujo Máximo - Corte Mínimo

Demostración Completa

Para cualquier red de flujo, el valor máximo de un flujo es igual a la capacidad mínima de un corte.

1. **Parte 1:** $|f| \leq c(S, T)$ para cualquier flujo f y cualquier corte (S, T).

   Por el lema anterior, $|f| = f(S, T)$. Como $f(u, v) \leq c(u, v)$ para (u, v) de S a T, y $f(v, u) \geq 0$, tenemos:

   $$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

2. **Parte 2:** Existe un flujo f y un corte (S, T) tales que $|f| = c(S, T)$.

   Sea f un flujo máximo. Construimos S como el conjunto de vértices alcanzables desde s en la red residual G_f. Como f es máximo, no hay camino de s a t en G_f, por lo que $t \notin S$. Entonces (S, T) es un corte.

   Para cualquier $u \in S$, $v \in T$ con $(u, v) \in E$, no puede haber (u, v) en G_f, por lo que $f(u, v) = c(u, v)$. Para (v, u) con $v \in T$, $u \in S$, si $(v, u) \in E$, entonces $f(v, u) = 0$ (de lo contrario habría una arista en G_f).

   Por lo tanto:

   $$|f| = f(S, T) = \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

Aplicaciones del Flujo Máximo

Matching Máximo en Grafos Bipartitos

[scale=0.6] [circle, draw] (s) at (0,0) s; [circle, draw] (l1) at (2,1.5) l_1; [circle, draw] (l2) at (2,0) l_2; [circle, draw] (l3) at (2,-1.5) l_3; [circle, draw]

(r1) at (5,1.5) r_1; [circle, draw] (r2) at (5,0) r_2; [circle, draw] (r3) at (5,-1.5) r_3; [circle, draw] (t) at (7,0) t;
[-¿] (s) – (l1); [-¿] (s) – (l2); [-¿] (s) – (l3); [-¿] (l1) – (r1); [-¿] (l1) – (r2); [-¿] (l2) – (r2); [-¿] (l2) – (r3); [-¿] (l3) – (r1); [-¿] (l3) – (r3); [-¿] (r1) – (t); [-¿] (r2) – (t); [-¿] (r3) – (t);

Segmentación de Imágenes

- Problema de separar objeto del fondo.
- Modelar píxeles como nodos, aristas entre píxeles adyacentes.
- Fuente conectada a píxeles de objeto probable, sumidero a píxeles de fondo.
- El corte mínimo da la segmentación óptima.

Redes de Transporte

- Maximizar flujo de mercancías de fábricas a mercados.
- Modelar carreteras con capacidades.

Problema de Asignación

- Asignar trabajadores a tareas.
- Modelar con capacidades unitarias.

Ejercicios Resueltos (20 Ejercicios)

Calcule el flujo máximo en la red con aristas: $s \to a$ (10), $s \to b$ (5), $a \to b$ (15), $a \to t$ (10), $b \to t$ (10).

1. Camino $s \to a \to t$: flujo 10. Residual: $s \to a$ (0), $a \to t$ (0), $a \to s$ (10), $t \to a$ (10)
2. Camino $s \to b \to t$: flujo 5. Residual: $s \to b$ (0), $b \to t$ (5), $b \to s$ (5), $t \to b$ (5)
3. Camino $s \to a \to b \to t$: $c_f(s, a) = 0$, no hay

Flujo total: $10 + 5 = 15$. Corte mínimo: $S = \{s\}$, $T = \{a, b, t\}$, capacidad $10 + 5 = 15$.

Demuestre que el flujo máximo es igual a la capacidad del corte mínimo. Ver demostración del Teorema de Flujo Máximo - Corte Mínimo.

Aplique Ford-Fulkerson a la red con aristas: $s \to a$ (3), $s \to b$ (2), $a \to c$ (2), $b \to c$ (3), $b \to d$ (2), $c \to t$ (4), $d \to t$ (3), $a \to b$ (1).

1. $s \to a \to c \to t$: flujo 2. Residual: $s \to a$ (1), $a \to c$ (0), $c \to t$ (2)
2. $s \to b \to c \to t$: flujo 2. Residual: $s \to b$ (0), $b \to c$ (1), $c \to t$ (0)
3. $s \to a \to b \to d \to t$: flujo 1. Residual: $s \to a$ (0), $a \to b$ (0), $b \to d$ (1), $d \to t$ (2)
4. $s \to b \to d \to t$: flujo 1

Flujo total: $2 + 2 + 1 + 1 = 6$.

¿Qué es la red residual y por qué es importante? La red residual G_f representa la capacidad adicional disponible en cada arista después de enviar un flujo f. Incluye aristas inversas que permiten "deshacer" flujo enviado previamente.

Es importante porque:

- Permite encontrar caminos de aumento.
- Los caminos en G_f corresponden a formas de aumentar el flujo.
- El teorema de flujo máximo se demuestra usando G_f.

Implemente Ford-Fulkerson con BFS (Edmonds-Karp). Ver implementación en la sección anterior.

¿Cuál es la complejidad de Edmonds-Karp y por qué? $O(VE^2)$. Cada BFS toma $O(E)$. El número de aumentos es $O(VE)$ porque cada arista puede volverse crítica $O(V)$ veces y cada aumento tiene al menos una arista crítica.

¿Cómo se puede usar flujo máximo para encontrar un matching máximo en un grafo bipartito? Se construye una red:

1. Fuente s conectada a todos los vértices de la partición izquierda (capacidad 1).
2. Sumidero t conectado desde todos los vértices de la partición derecha (capacidad 1).
3. Aristas originales del grafo bipartito (capacidad 1).

4. El flujo máximo es igual al tamaño del matching máximo.

Demuestre que si todas las capacidades son enteras, Ford-Fulkerson termina en tiempo finito. Cada aumento incrementa el flujo en al menos 1. El flujo máximo está acotado por la suma de capacidades que salen de s, que es finita. Por lo tanto, después de un número finito de aumentos se alcanza el flujo máximo.

¿Qué sucede si las capacidades son irracionales? Ford-Fulkerson puede no terminar (ejemplo de Zwick). La versión con BFS (Edmonds-Karp) termina porque siempre elige caminos de aumento más cortos.

Encuentre el corte mínimo en la red del ejercicio 1. Corte mínimo: $S = \{s, a, b\}$, $T = \{c, d, t\}$. Capacidad: $a \to c$ (0 después del flujo? original 2), $b \to c$ (1), $b \to d$ (1), $a \to t$ (0), $b \to t$ (0). Suma = 2+1+1=4? Verificar.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule el flujo máximo en una red de 6 vértices.
2. Demuestre el Teorema de Flujo Máximo - Corte Mínimo.
3. Implemente Ford-Fulkerson con DFS.
4. ¿Qué es el algoritmo de push-relabel y cómo mejora Ford-Fulkerson?
5. Aplique flujo máximo para encontrar matching máximo en un grafo bipartito.
6. Demuestre que el corte mínimo es único o dé un contraejemplo.
7. ¿Cómo se puede usar flujo máximo para particionar un grafo?
8. Implemente Edmonds-Karp.
9. ¿Qué es la capacidad de un corte y cómo se calcula?
10. Encuentre el flujo máximo en una red con capacidades unitarias.
11. ¿Qué es el problema de flujo con demandas?
12. Demuestre que el flujo máximo es igual a la capacidad del corte mínimo.
13. Aplique flujo máximo a un problema de asignación de tareas.
14. ¿Qué es el teorema de descomposición de flujo?

15. Implemente una variante de Ford-Fulkerson con escalado de capacidades.
16. ¿Cómo se detecta un corte mínimo después de ejecutar Ford-Fulkerson?
17. Aplique flujo máximo a una red de 5 vértices con capacidades aleatorias.
18. ¿Qué es el problema de flujo de costo mínimo?
19. Demuestre que el algoritmo de Edmonds-Karp tiene complejidad $O(VE^2)$.
20. Investigue aplicaciones de flujo máximo en redes de telecomunicaciones.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de redes de flujo, flujos y cortes.
- El Teorema de Flujo Máximo - Corte Mínimo y su demostración.
- El algoritmo de Ford-Fulkerson y su variante Edmonds-Karp.
- El concepto de red residual y caminos de aumento.
- La complejidad $O(VE^2)$ de Edmonds-Karp.
- Aplicaciones prácticas en matching bipartito y segmentación.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 26.
- Edmonds, J., & Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248-264.
- Ford, L. R., & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399-404.

Unidad VIII: Algoritmos de String (Cadenas)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de búsqueda de patrones en cadenas.
- Implementar y analizar el algoritmo de búsqueda ingenua (fuerza bruta).
- Implementar y analizar el algoritmo KMP (Knuth-Morris-Pratt).
- Implementar y analizar el algoritmo de Boyer-Moore.
- Comprender la función de prefijo y la tabla de saltos.
- Aplicar estos algoritmos a problemas prácticos de búsqueda de texto.

Introducción al Problema de Búsqueda de Patrones

Definición del Problema

Dado un **texto** T de longitud n y un **patrón** P de longitud m ($m \leq n$), encontrar todas las ocurrencias de P en T.

$T = "ABABABAC"$, $P = "ABA"$ Ocurrencias en posiciones: 0, 2, 4

Aplicaciones

- Editores de texto (Ctrl+F)
- Motores de búsqueda

- Análisis de ADN
- Detección de plagio
- Compresión de datos

Notación

- $T[0..n-1]$: texto
- $P[0..m-1]$: patrón
- $T[i..j]$: subcadena de T desde i hasta j

Algoritmo Ingenuo (Fuerza Bruta)

Descripción

El algoritmo más simple: probar todas las posiciones posibles del texto donde podría comenzar el patrón.

Pseudocódigo

[language=C, caption=Búsqueda ingenua] NAIVE-SEARCH(T, P) n = length(T) m = length(P) for i = 0 to n - m j = 0 while j ¡ m and T[i+j] == P[j] j = j + 1 if j == m print "Patron encontrado en posicion", i

Ejemplo

$T =$ "$ABABABAC$", $P =$ "ABA"

```
Posición 0: A B A vs A B A
Posición 1: B A B vs A B A
Posición 2: A B A vs A B A
Posición 3: B A B vs A B A
Posición 4: A B A vs A B A
Posición 5: B A C vs A B A
```

Análisis de Complejidad

- Peor caso: $O(m \cdot (n - m + 1)) = O(mn)$
- Ejemplo de peor caso: $T = "AAAAA..."$, $P = "AAA"$
- Mejor caso: $O(n)$ (primera comparación falla siempre)

Algoritmo KMP (Knuth-Morris-Pratt)

Descripción

KMP aprovecha la información de las comparaciones previas para evitar retroceder en el texto. Utiliza una **función de prefijo** (o tabla de fallos) que indica el mayor prefijo propio del patrón que también es sufijo del patrón procesado.

Función de Prefijo

Para un patrón P, definimos $\pi[q]$ como la longitud del prefijo propio más largo de $P[0..q]$ que también es sufijo de $P[0..q]$.

$P = "ABABAC"$

- $\pi[0] = 0$ (prefijo propio de "A" es vacío)
- $\pi[1] = 0$ ("AB" no tiene prefijo propio igual a sufijo)
- $\pi[2] = 1$ ("ABA" tiene prefijo "A" igual a sufijo "A")
- $\pi[3] = 2$ ("ABAB" tiene prefijo "AB" igual a sufijo "AB")
- $\pi[4] = 3$ ("ABABA" tiene prefijo "ABA" igual a sufijo "ABA")
- $\pi[5] = 0$ ("ABABAC" no tiene)

Cálculo de la Función de Prefijo

[language=C, caption=Cálculo de la función de prefijo] COMPUTE-PREFIX-FUNCTION(P) m = length(P) pi = array of size m pi[0] = 0 k = 0 for q = 1 to m-1 while k ¿ 0 and P[k] != P[q] k = pi[k-1] if P[k] == P[q] k = k + 1 pi[q] = k return pi

Ejemplo de Cálculo

$P = "ABABAC"$

```
q=1, k=0: P[0]=A vs P[1]=B, diferentes, pi[1]=0
q=2, k=0: P[0]=A vs P[2]=A, iguales, k=1, pi[2]=1
q=3, k=1: P[1]=B vs P[3]=B, iguales, k=2, pi[3]=2
q=4, k=2: P[2]=A vs P[4]=A, iguales, k=3, pi[4]=3
q=5, k=3: P[3]=B vs P[5]=C, diferentes, k=pi[2]=1
         P[1]=B vs P[5]=C, diferentes, k=pi[0]=0
         P[0]=A vs P[5]=C, diferentes, pi[5]=0
```

Algoritmo KMP

[language=C, caption=Algoritmo KMP] KMP-SEARCH(T, P) n = length(T) m = length(P) pi = COMPUTE-PREFIX-FUNCTION(P) q = 0 // número de caracteres del patrón coincidentes

for i = 0 to n-1 while q ¿ 0 and P[q] != T[i] q = pi[q-1] if P[q] == T[i] q = q + 1 if q == m print "Patron encontrado en posicion", i - m + 1 q = pi[q-1] // continuar buscando

Ejemplo de Ejecución

$T = "ABABABAC"$, $P = "ABA"$, $\pi = [0, 0, 1]$

```
i=0, T[0]=A, q=0: P[0]=A == A, q=1
i=1, T[1]=B, q=1: P[1]=B == B, q=2
i=2, T[2]=A, q=2: P[2]=A == A, q=3, encontrado en 0, q=pi[2]=1
i=3, T[3]=B, q=1: P[1]=B == B, q=2
i=4, T[4]=A, q=2: P[2]=A == A, q=3, encontrado en 2, q=pi[2]=1
i=5, T[5]=B, q=1: P[1]=B == B, q=2
i=6, T[6]=A, q=2: P[2]=A == A, q=3, encontrado en 4, q=pi[2]=1
i=7, T[7]=C, q=1: P[1]=B vs C, q=pi[0]=0, P[0]=A vs C, q=0
```

Análisis de Complejidad

- Construcción de π: $O(m)$
- Búsqueda: $O(n)$
- Total: $O(n + m)$

Algoritmo de Boyer-Moore

Descripción

Boyer-Moore es uno de los algoritmos más eficientes en la práctica. Compara el patrón de derecha a izquierda y utiliza dos heurísticas:

1. Heurística del mal carácter (bad character rule)
2. Heurística del buen sufijo (good suffix rule)

Heurística del Mal Carácter

Cuando ocurre una falta de coincidencia, desplaza el patrón para alinear el carácter del texto con la ocurrencia más a la derecha de ese carácter en el patrón.

[language=C, caption=Tabla de mal carácter] COMPUTE-BAD-CHARACTER-TABLE(P) m = length(P) // Inicializar con -1 para todos los caracteres for each character c in alphabet lastOccurrence[c] = -1 for i = 0 to m-1 lastOccurrence[P[i]] = i return lastOccurrence

Heurística del Buen Sufijo

Cuando ocurre una falta de coincidencia, desplaza el patrón para alinear el sufijo ya coincidente con otra ocurrencia del mismo sufijo en el patrón.

Tabla de Buen Sufijo

[language=C, caption=Tabla de buen sufijo] COMPUTE-GOOD-SUFFIX-TABLE(P) m = length(P) suffix = array of size m border = array of size m+1

// Calcular sufijos suffix[m-1] = m g = m-1 for i = m-2 downto 0 if i ¿ g and suffix[i + m - 1 - f] ¡ i - g suffix[i] = suffix[i + m - 1 - f] else if i ¡ g g = i f = i while g ¿= 0 and P[g] == P[g + m - 1 - f] g = g - 1 suffix[i] = f - g return suffix

Algoritmo Completo

[language=C, caption=Algoritmo de Boyer-Moore] BOYER-MOORE-SEARCH(T, P) n = length(T) m = length(P) badChar = COMPUTE-BAD-CHARACTER-TABLE(P) goodSuffix = COMPUTE-GOOD-SUFFIX-TABLE(P)

s = 0 // desplazamiento del patrón while s ¡= n - m j = m - 1 // empezar desde el final while j ¿= 0 and P[j] == T[s + j] j = j - 1 if j ¡ 0 print "Patron encontrado en posicion", s s = s + goodSuffix[0] // desplazamiento else // Calcular desplazamiento por mal carácter badShift = j - badChar[T[s + j]] if badShift ¡ 1 badShift = 1 // Usar el máximo de las dos heurísticas goodShift = goodSuffix[j] s = s + max(badShift, goodShift)

Ejemplo de Ejecución

$T = "HEREISASIMPLEEXAMPLE"$, $P = "EXAMPLE"$

```
Primera comparación: H vs E (mal carácter E en P en posición 6)
Desplazamiento: 6 - badChar['H'] = 6 - (-1) = 7

Segunda comparación: S vs E (mal carácter E en posición 6)
Desplazamiento: 6 - badChar['S'] = 6 - (-1) = 7

... (continúa)
```

Análisis de Complejidad

- Preprocesamiento: $O(m + |\Sigma|)$ donde Σ es el alfabeto
- Peor caso: $O(mn)$ (raro en la práctica)
- Caso promedio: $O(n/m)$ (muy eficiente)

Comparación de Algoritmos

Algoritmo	Preprocesamiento	Búsqueda	Memoria
Ingenuo	$O(1)$	$O(mn)$	$O(1)$
KMP	$O(m)$	$O(n)$	$O(m)$
Boyer-Moore	$O(m + \|\Sigma\|)$	$O(n/m)$ promedio	$O(m + \|\Sigma\|)$

Otros Algoritmos de String

Algoritmo de Rabin-Karp

Utiliza hashing para comparar subcadenas en tiempo promedio $O(n + m)$.

Algoritmo Z

Similar a KMP pero calcula la función Z.

Aho-Corasick

Para búsqueda de múltiples patrones simultáneamente.

Ejercicios Resueltos (20 Ejercicios)

Busque el patrón "ABA" en el texto "ABABABAC" usando el algoritmo ingenuo. Posiciones: 0, 2, 4. Ver ejemplo anterior.

Calcule la función de prefijo para $P = "AAABAAA"$.

```
[0]=0
i=1, P[1]=A, P[0]=A, k=1, [1]=1
i=2, P[2]=A, P[1]=A, k=2, [2]=2
i=3, P[3]=B, P[2]=A, k=[1]=1, P[1]=A vs B, k=[0]=0, P[0]=A vs B, [3]=0
i=4, P[4]=A, P[0]=A, k=1, [4]=1
i=5, P[5]=A, P[1]=A, k=2, [5]=2
i=6, P[6]=A, P[2]=A, k=3, [6]=3
```

Resultado: $\pi = [0, 1, 2, 0, 1, 2, 3]$

Ejecute KMP para $T = "AAABAAA"$, $P = "AAA"$. $\pi = [0, 1, 2]$

```
i=0, T[0]=A, q=0: P[0]=A, q=1
i=1, T[1]=A, q=1: P[1]=A, q=2
i=2, T[2]=A, q=2: P[2]=A, q=3, encontrado en 0, q=[2]=2
i=3, T[3]=B, q=2: P[2]=A vs B, q=[1]=1, P[1]=A vs B, q=[0]=0, P[0]=A vs B, q=0
i=4, T[4]=A, q=0: P[0]=A, q=1
i=5, T[5]=A, q=1: P[1]=A, q=2
i=6, T[6]=A, q=2: P[2]=A, q=3, encontrado en 4
```

Ocurrencias: 0, 4

Explique por qué KMP es más eficiente que el algoritmo ingenuo. KMP nunca retrocede en el texto. Cuando ocurre una falta de coincidencia, utiliza la función de prefijo para saber cuánto avanzar en el patrón, evitando comparaciones redundantes. En el peor caso, el algoritmo ingenuo hace $O(mn)$ comparaciones, mientras que KMP hace $O(n)$.

Calcule la tabla de mal carácter para $P = "EXAMPLE"$.

```
E: 0
X: 1
A: 2
M: 3
P: 4
L: 5
E: 6 (sobrescribe posición 0)
```

Para cualquier otro carácter: -1

¿Cuál es la ventaja de Boyer-Moore sobre KMP? Boyer-Moore compara de derecha a izquierda, lo que permite mayores desplazamientos. En la práctica, especialmente para alfabetos grandes y patrones largos, Boyer-Moore es significativamente más rápido (sublineal, $O(n/m)$ en promedio).

Encuentre todas las ocurrencias de "AB" en "ABABAB" usando KMP. $P =$ "AB", $\pi = [0, 0]$

```
i=0: q=1
i=1: q=2 -> encontrado en 0, q=0
i=2: q=1
i=3: q=2 -> encontrado en 2
i=4: q=1
i=5: q=2 -> encontrado en 4
```

Ocurrencias: 0, 2, 4

¿Qué es la función de prefijo y cómo se calcula? La función de prefijo $\pi[q]$ es la longitud del prefijo propio más largo de $P[0..q]$ que también es sufijo de $P[0..q]$. Se calcula en $O(m)$ usando un algoritmo que aprovecha los valores ya calculados.

Demuestre que KMP tiene complejidad $O(n + m)$. Cada iteración del bucle principal incrementa i o disminuye q (a través del while). q nunca es negativo y se incrementa a lo sumo n veces. Por lo tanto, el número total de iteraciones es $O(n + m)$.

Aplique Boyer-Moore para buscar "AB" en "AABAB".

```
Posición 0: comparar P[1]=B vs T[1]=A -> mal carácter A, badShift=1-(-1)=2
Posición 2: comparar P[1]=B vs T[3]=B, P[0]=A vs T[2]=A -> encontrado en 2
Posición 3: comparar P[1]=B vs T[4]=B, P[0]=A vs T[3]=B -> mal carácter B, ba
```

Ocurrencias: 2

Ejercicios Propuestos (20 Ejercicios)

1. Busque "ABC" en "AABCABC" usando el algoritmo ingenuo.
2. Calcule la función de prefijo para "ABABAB".
3. Ejecute KMP para buscar "ABAB" en "ABABABAB".
4. Calcule la tabla de mal carácter para "MISSISSIPPI".
5. ¿Qué es la heurística del buen sufijo en Boyer-Moore?
6. Implemente el algoritmo ingenuo en C.
7. ¿En qué casos el algoritmo ingenuo es tan bueno como KMP?
8. Demuestre que la función de prefijo se puede calcular en $O(m)$.
9. Aplique Boyer-Moore a un texto largo y un patrón corto.
10. ¿Qué es el algoritmo de Rabin-Karp y cómo usa hashing?
11. Implemente KMP en Python.
12. ¿Qué es el algoritmo Z y cómo se relaciona con KMP?
13. Encuentre todas las ocurrencias de "ANA" en "BANANA".
14. ¿Cómo se manejan caracteres que no están en el patrón en Boyer-Moore?
15. Implemente la función de prefijo para un patrón dado.
16. ¿Qué es el algoritmo de Aho-Corasick y para qué sirve?
17. Compare KMP y Boyer-Moore en diferentes escenarios.
18. ¿Qué es la complejidad en el peor caso de Boyer-Moore?
19. Implemente Boyer-Moore en Java.
20. Investigue aplicaciones de estos algoritmos en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de búsqueda de patrones en cadenas.
- El algoritmo ingenuo ($O(mn)$) como caso base.
- El algoritmo KMP ($O(n + m)$) usando función de prefijo.
- El algoritmo de Boyer-Moore (sublineal en la práctica) con heurísticas de mal carácter y buen sufijo.
- Comparación de los algoritmos y sus aplicaciones.

Bibliografía de la Unidad

- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10), 762-772.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 32.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.

Unidad IX: Algoritmos de Búsqueda de Patrones - KMP y Boyer-Moore

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender en profundidad el algoritmo KMP y su función de prefijo.
- Implementar y analizar KMP en detalle.
- Comprender las heurísticas de Boyer-Moore (mal carácter y buen sufijo).
- Implementar y analizar Boyer-Moore.
- Comparar ambos algoritmos y elegir el apropiado según el contexto.
- Aplicar estos algoritmos a problemas de búsqueda en textos reales.

Algoritmo KMP (Knuth-Morris-Pratt)

Filosofía del Algoritmo

KMP se basa en una idea fundamental: cuando ocurre una falta de coincidencia, ya sabemos parte del texto (la parte que coincidió con el prefijo del patrón). Esta información se puede usar para evitar retroceder en el texto.

[scale=0.7] (0,0) rectangle (10,0.5); at (1,0.25) A; at (2,0.25) B; at (3,0.25) A; at (4,0.25) B; at (5,0.25) A; at (6,0.25) B; at (7,0.25) A; at (8,0.25) C; at (9,0.25) ?;
(0,-1) rectangle (3,-0.5); at (0.5,-0.75) A; at (1.5,-0.75) B; at (2.5,-0.75) A;

[red, thick] (3,-1) – (3,0.5); [red, thick] (3,-1) – (3,-0.5);
at (5,-1.5) Falta coincidencia en posición 3;

Función de Prefijo - Análisis Detallado

Definición Formal

Para un patrón P de longitud m, la función de prefijo π se define como:

$$\pi[q] = \max\{k : 0 \leq k < qyP[0..k-1] = P[q-k..q-1]\}$$

Es decir, la longitud del prefijo propio más largo que también es sufijo de $P[0..q-1]$.

Ejemplo Detallado Para $P = "ABABAC"$:

q	Subcadena	$\pi[q]$
0	"A"	0
1	"AB"	0
2	"ABA"	1 (prefijo "A" = sufijo "A")
3	"ABAB"	2 (prefijo "AB" = sufijo "AB")
4	"ABABA"	3 (prefijo "ABA" = sufijo "ABA")
5	"ABABAC"	0

Implementación de la Función de Prefijo

[language=C, caption=Función de prefijo] void computePrefixFunction(char* P, int m, int* pi) pi[0] = 0; int k = 0;

for (int q = 1; q ¡ m; q++) // Mientras haya una falta de coincidencia, retroceder while (k ¿ 0 P[k] != P[q]) k = pi[k - 1]; // Si coinciden, avanzar if (P[k] == P[q]) k++; pi[q] = k;

Visualización de la Función de Prefijo

[scale=0.6] (0,0) rectangle (6,0.5); at (0.5,0.25) A; at (1.5,0.25) B; at (2.5,0.25) A; at (3.5,0.25) B; at (4.5,0.25) A; at (5.5,0.25) C;
[blue, thick] (0,-0.5) – (1,-0.5); [blue, thick] (0,-0.5) – (0,-1); [blue, thick] (1,-0.5) – (1,-1); at (0.5,-0.8) prefijo "A";
[red, thick] (2.5,-0.5) – (3.5,-0.5); [red, thick] (2.5,-0.5) – (2.5,-1); [red, thick] (3.5,-0.5) – (3.5,-1); at (3,-0.8) sufijo "AB";

Algoritmo KMP Completo

[language=C, caption=Algoritmo KMP completo] include ¡stdio.h¿ include ¡string.h¿ include ¡stdlib.h¿

void KMP(char* T, char* P) int n = strlen(T); int m = strlen(P); int* pi = (int*)malloc(m * sizeof(int));

// Calcular función de prefijo computePrefixFunction(P, m, pi);

// Búsqueda int q = 0; // número de caracteres coincidentes

for (int i = 0; i ¡ n; i++) // Mientras haya falta de coincidencia y q¿0, retroceder while (q ¿ 0 P[q] != T[i]) q = pi[q - 1]; // Si coinciden, avanzar if (P[q] == T[i]) q++; // Si encontramos el patrón completo if (q == m) printf("Patrón encontrado en posición q = pi[q - 1]; // continuar buscando ocurrencias superpuestas

free(pi);

Ejemplo de Ejecución Detallada

$T =$ "$ABABABAC$", $P =$ "ABA"

i	$T[i]$	q antes	Acción	q después
0	A	0	$P[0] = A$ coincide	1
1	B	1	$P[1] = B$ coincide	2
2	A	2	$P[2] = A$ coincide	3 (encontrado en 0)
3	B	3→1	$P[1] = B$ coincide	2
4	A	2	$P[2] = A$ coincide	3 (encontrado en 2)
5	B	3→1	$P[1] = B$ coincide	2
6	A	2	$P[2] = A$ coincide	3 (encontrado en 4)
7	C	3→1→0	$P[0] = A \neq C$	0

Demostración de Correctitud de KMP

El algoritmo KMP encuentra todas las ocurrencias del patrón P en el texto T en tiempo $O(n + m)$.

[Esbozo]

1. **Corrección de la función de prefijo:** $\pi[q]$ es la longitud del prefijo propio más largo que es sufijo de $P[0..q-1]$. Esto se demuestra por inducción.

2. **Corrección de la búsqueda:** El invariante es que después de procesar $T[0..i-1]$, q es la longitud del prefijo más largo de P que es sufijo de $T[0..i-1]$. Cuando $q = m$, se ha encontrado una ocurrencia.

3. **Complejidad:** Cada iteración incrementa i o disminuye q. q se incrementa a lo sumo n veces, y cada decremento corresponde a un incremento previo. Por lo tanto, el número total de operaciones es $O(n+m)$.

Propiedades Adicionales de KMP

- KMP es **online**: puede procesar el texto carácter por carácter.
- No requiere retroceder en el texto.
- Es óptimo en el peor caso: cualquier algoritmo de búsqueda de patrones debe hacer al menos $\Omega(n)$ comparaciones.

Algoritmo de Boyer-Moore

Filosofía del Algoritmo

Boyer-Moore compara el patrón de **derecha a izquierda**. Cuando ocurre una falta de coincidencia, puede desplazar el patrón múltiples posiciones basándose en dos heurísticas.

[scale=0.7] (0,0) rectangle (10,0.5); at (1,0.25) H; at (2,0.25) E; at (3,0.25) R; at (4,0.25) E; at (5,0.25) ; at (6,0.25) I; at (7,0.25) S; at (8,0.25) ; at (9,0.25) A;
(3,-1) rectangle (8,-0.5); at (3.5,-0.75) E; at (4.5,-0.75) X; at (5.5,-0.75) A; at (6.5,-0.75) M; at (7.5,-0.75) P;
[red, thick] (8,-1) – (8,0.5); [blue, thick] (4.5,-1) – (4.5,-0.5);

Heurística del Mal Carácter

Definición

Cuando se encuentra una falta de coincidencia en la posición j del patrón (comparando de derecha a izquierda), y el carácter del texto en esa posición es c, desplazamos el patrón para alinear la ocurrencia más a la derecha de c en el patrón con la posición actual del texto.

$$desplazamiento = j - \acute{u}ltimaOcurrencia[c]$$

Si c no está en el patrón, $\acute{u}ltimaOcurrencia[c] = -1$.

Implementación de la Tabla de Mal Carácter

[language=C, caption=Tabla de mal carácter] define ALPHABET$_SIZE256$

void computeBadCharTable(char* P, int m, int* badChar) // Inicializar con -1 for (int i = 0; i ¡ ALPHABET$_SIZE; i{+}{+})badChar[i] = -1;//Guardarlaúltimaocurrenciadecad$
$0; i < m; i{+}{+})badChar[(int)P[i]] = i;$

Heurística del Buen Sufijo

Definición

Cuando ocurre una falta de coincidencia después de haber coincidido un sufijo S del patrón, desplazamos el patrón para alinear otra ocurrencia de S en el patrón (si existe), o un prefijo del patrón que sea sufijo de S.

Tipos de Buen Sufijo

1. **Caso 1:** Otra ocurrencia del sufijo S en el patrón.

2. **Caso 2:** Un prefijo del patrón que es sufijo de S.

3. **Caso 3:** Desplazamiento completo m (ninguna coincidencia parcial).

Implementación de la Tabla de Buen Sufijo

[language=C, caption=Tabla de buen sufijo] void computeGoodSuffixTable(char* P, int m, int* goodSuffix) int* suffix = (int*)malloc(m * sizeof(int)); int* border = (int*)malloc((m + 1) * sizeof(int));

// Calcular array de sufijos suffix[m - 1] = m; int g = m - 1; int f = m - 1;

for (int i = m - 2; i ¿= 0; i–) if (i ¿ g suffix[i + m - 1 - f] ¡ i - g) suffix[i] = suffix[i + m - 1 - f]; else if (i ¡ g) g = i; f = i; while (g ¿= 0 P[g] == P[g + m - 1 - f]) g–; suffix[i] = f - g;

// Inicializar goodSuffix for (int i = 0; i ¡ m; i++) goodSuffix[i] = m;

// Calcular bordes int j = 0; for (int i = m - 1; i ¿= -1; i–) if (i == -1 —— suffix[i] == i + 1) for (; j ¡ m - 1 - i; j++) if (goodSuffix[j] == m) goodSuffix[j] = m - 1 - i;

// Casos de sufijos for (int i = 0; i ¡ m - 1; i++) goodSuffix[m - 1 - suffix[i]] = m - 1 - i;

free(suffix); free(border);

Algoritmo de Boyer-Moore Completo

[language=C, caption=Boyer-Moore completo] void boyerMoore(char* T, char* P) int n = strlen(T); int m = strlen(P); int badChar[ALPHABET$_SIZE$]; $int*$ $goodSuffix = (int*)malloc(m * sizeof(int))$;

computeBadCharTable(P, m, badChar); computeGoodSuffixTable(P, m, goodSuffix);

int s = 0; // desplazamiento del patrón sobre el texto

while (s ¡= n - m) int j = m - 1; // empezar desde el final

// Comparar de derecha a izquierda while (j ¿= 0 P[j] == T[s + j]) j–;

if (j ¡ 0) printf("Patrón encontrado en posición s += goodSuffix[0]; else // Calcular desplazamiento por mal carácter int badShift = j - badChar[(int)T[s + j]]; if (badShift ¡ 1) badShift = 1; // Usar el máximo de las dos heurísticas int goodShift = goodSuffix[j]; s += (badShift ¿ goodShift) ? badShift : goodShift;

free(goodSuffix);

Ejemplo Detallado de Boyer-Moore

$T = "HEREISASIMPLEEXAMPLE"$, $P = "EXAMPLE"$

Paso	Comparación	Desplazamiento
1	"E" vs "H" (mal carácter H)	$6 - (-1) = 7$
2	"E" vs "S" (mal carácter S)	$6 - (-1) = 7$
3	"E" vs "A" (mal carácter A)	$6 - 2 = 4$
4	"E" vs " " (espacio)	$6 - (-1) = 7$
5	"E" vs "S" (mal carácter S)	$6 - (-1) = 7$
6	"E" vs "E" (coincide), continuar	-
7	"P" vs "P" (coincide)	-
8	"L" vs "L" (coincide)	-
9	"M" vs "M" (coincide)	-
10	"A" vs "A" (coincide)	-
11	"X" vs "X" (coincide)	-
12	"E" vs " " (falta)	buen sufijo "EXAMPLE"

Análisis de Complejidad de Boyer-Moore

- **Preprocesamiento:** $O(m + |\Sigma|)$
- **Mejor caso:** $O(n/m)$ (sublineal)
- **Peor caso:** $O(mn)$ (raro, ocurre con patrones como "AAAAA")

- **Caso promedio:** $O(n)$ con una constante muy pequeña

Optimizaciones de Boyer-Moore

1. **Boyer-Moore-Horspool:** Solo usa la heurística del mal carácter (más simple).

2. **Boyer-Moore con Galil:** Optimización para patrones periódicos.

3. **Algoritmo de Apostolico-Giancarlo:** Mejora para patrones repetitivos.

Comparación entre KMP y Boyer-Moore

Característica	**KMP**	**Boyer-Moore**
Dirección de comparación	Izquierda a derecha	Derecha a izquierda
Preprocesamiento	$O(m)$	$O(m + \|\Sigma\|)$
Peor caso	$O(n + m)$	$O(mn)$ (raro)
Mejor caso	$O(n + m)$	$O(n/m)$
Memoria	$O(m)$	$O(m + \|\Sigma\|)$
Online	Sí	No (necesita el patrón completo)
Complejidad de implementación	Media	Alta

Cuándo usar cada algoritmo

- **KMP:**
  - Cuando el patrón es corto o el alfabeto es pequeño.
  - Cuando se necesita un algoritmo online.
  - Cuando se garantiza que el peor caso es importante.
- **Boyer-Moore:**
  - Cuando el patrón es largo y el alfabeto es grande.
  - En la práctica, para búsqueda en textos naturales.
  - Cuando se busca un patrón en un texto grande.

Ejercicios Resueltos (20 Ejercicios)

Calcule la función de prefijo para $P = "ABCDABD"$.

```
[0]=0
i=1, P[1]=B vs P[0]=A, [1]=0
i=2, P[2]=C vs P[0]=A, [2]=0
i=3, P[3]=D vs P[0]=A, [3]=0
i=4, P[4]=A vs P[0]=A, k=1, [4]=1
i=5, P[5]=B vs P[1]=B, k=2, [5]=2
i=6, P[6]=D vs P[2]=C, k=[1]=0, P[0]=A vs D, [6]=0
```

Resultado: $[0, 0, 0, 0, 1, 2, 0]$

Ejecute KMP para $T = "ABCDABD"$, $P = "ABD"$. $\pi = [0, 0, 0]$

```
i=0, T[0]=A, q=0: P[0]=A, q=1
i=1, T[1]=B, q=1: P[1]=B, q=2
i=2, T[2]=C, q=2: P[2]=D vs C, q=[1]=0, P[0]=A vs C, q=0
i=3, T[3]=D, q=0: P[0]=A vs D, q=0
i=4, T[4]=A, q=0: P[0]=A, q=1
i=5, T[5]=B, q=1: P[1]=B, q=2
i=6, T[6]=D, q=2: P[2]=D, q=3, encontrado en 4
```

Calcule la tabla de mal carácter para $P = "ABCBAB"$.

```
A: 0
B: 5 (última ocurrencia en posición 5)
C: 2
Otros caracteres: -1
```

Ejecute Boyer-Moore para $T = "ABABABAB"$, $P = "BAB"$. Tabla mal carácter: B:2, A:1

```
s=0: comparar P[2]=B vs T[2]=A, badShift=2-1=1, s=1
s=1: comparar P[2]=B vs T[3]=B, P[1]=A vs T[2]=A, encontrado en 1
s=2: comparar P[2]=B vs T[4]=B, P[1]=A vs T[3]=B, badShift=1-1=0->1, s=3
s=3: comparar P[2]=B vs T[5]=B, P[1]=A vs T[4]=A, encontrado en 3
s=4: comparar P[2]=B vs T[6]=B, P[1]=A vs T[5]=B, badShift=1-1=0->1, s=5
s=5: comparar P[2]=B vs T[7]=B, P[1]=A vs T[6]=A, encontrado en 5
```

Ocurrencias: 1, 3, 5

Ejercicios Propuestos (20 Ejercicios)

1. Calcule la función de prefijo para "ABABABAB".
2. Ejecute KMP para buscar "ABAB" en "ABABABAB".
3. Calcule la tabla de mal carácter para "MISSISSIPPI".
4. Ejecute Boyer-Moore para buscar "ISS" en "MISSISSIPPI".
5. ¿Qué ventaja tiene KMP sobre el algoritmo ingenuo?
6. ¿Qué ventaja tiene Boyer-Moore sobre KMP?
7. Implemente KMP en Python.
8. Implemente Boyer-Moore en C.
9. ¿Qué es la función de prefijo y cómo se usa?
10. ¿Qué es la heurística del buen sufijo?
11. Demuestre que KMP tiene complejidad $O(n + m)$.
12. ¿En qué casos Boyer-Moore tiene peor caso $O(mn)$?
13. ¿Qué es el algoritmo de Boyer-Moore-Horspool?
14. ¿Cómo se manejan caracteres no alfabéticos en Boyer-Moore?
15. Compare KMP y Boyer-Moore en textos con patrones repetitivos.
16. ¿Qué es el algoritmo de Galil y para qué sirve?
17. ¿Se puede usar KMP para búsqueda online? ¿Y Boyer-Moore?
18. Implemente una versión optimizada de Boyer-Moore.
19. ¿Qué es el algoritmo de Apostolico-Giancarlo?
20. Investigue aplicaciones de KMP en bioinformática.

Resumen de la Unidad

En esta unidad hemos profundizado en:

- El algoritmo KMP: función de prefijo, búsqueda en $O(n + m)$, online.
- El algoritmo de Boyer-Moore: heurísticas de mal carácter y buen sufijo, sublineal en la práctica.
- Comparación detallada de ambos algoritmos.
- Implementaciones completas y ejemplos paso a paso.
- Aplicaciones prácticas y criterios de selección.

Bibliografía de la Unidad

- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10), 762-772.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 32.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.

Unidad X: Algoritmos sobre Números

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos de la aritmética modular.
- Implementar y analizar el algoritmo de Euclides para el cálculo del MCD.
- Implementar y analizar el algoritmo de Euclides extendido.
- Comprender los conceptos básicos de la criptografía RSA.
- Aplicar estos algoritmos a problemas de teoría de números y criptografía.

Introducción a la Aritmética Modular

Definiciones Fundamentales

Decimos que $a \equiv b \pmod{n}$ si n divide a $a - b$, es decir, $a - b = kn$ para algún entero k.

Propiedades de la Aritmética Modular

- $(a + b) \bmod n = (a \bmod n + b \bmod n) \bmod n$
- $(a - b) \bmod n = (a \bmod n - b \bmod n) \bmod n$
- $(a \cdot b) \bmod n = (a \bmod n) \cdot (b \bmod n) \bmod n$
- $a^b \bmod n$ se puede calcular eficientemente mediante exponenciación modular.

Inverso Modular

El **inverso modular** de a módulo n es un entero x tal que:

$$a \cdot x \equiv 1 \pmod{n}$$

El inverso existe si y solo si $\gcd(a, n) = 1$.

Algoritmo de Euclides para el Máximo Común Divisor

Descripción

El algoritmo de Euclides calcula el máximo común divisor (MCD) de dos números enteros.

Pseudocódigo

[language=C, caption=Algoritmo de Euclides] int euclides(int a, int b) while (b != 0) int temp = b; b = a a = temp; return a;

Ejemplo

Calcular $\gcd(48, 18)$:

```
a=48, b=18: 48 % 18 = 12, a=18, b=12
a=18, b=12: 18 % 12 = 6, a=12, b=6
a=12, b=6: 12 % 6 = 0, a=6, b=0
Resultado: 6
```

Análisis de Complejidad

- Tiempo: $O(\log \min(a, b))$ (algoritmo más rápido conocido)
- Espacio: $O(1)$

Demostración de Correctitud

El algoritmo de Euclides calcula correctamente $\gcd(a, b)$. $\gcd(a, b) = \gcd(b, a \bmod b)$. Esta propiedad se demuestra porque si d divide a a y b, entonces divide a $a \bmod b = a - \lfloor a/b \rfloor \cdot b$. Por lo tanto, los divisores comunes se preservan.

Algoritmo de Euclides Extendido

Descripción

El algoritmo de Euclides extendido encuentra enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

Pseudocódigo

[language=C, caption=Algoritmo de Euclides Extendido] int euclidesExtendido(int a, int b, int* x, int* y) if (b == 0) *x = 1; *y = 0; return a;
int x1, y1; int gcd = euclidesExtendido(b, a
*x = y1; *y = x1 - (a / b) * y1;
return gcd;

Ejemplo

Para $a = 48$, $b = 18$:

```
euclidesExtendido(48,18)
  euclidesExtendido(18,12)
    euclidesExtendido(12,6)
      euclidesExtendido(6,0) -> gcd=6, x=1, y=0
    x=0, y=1 - (12/6)*0 = 1
    gcd=6
  x=1, y=0 - (18/12)*1 = -1
  gcd=6
x=-1, y=1 - (48/18)*(-1) = 1 + 2 = 3
Resultado: 48*(-1) + 18*3 = -48 + 54 = 6
```

Aplicaciones del Euclides Extendido

- Cálculo de inversos modulares
- Resolución de ecuaciones diofánticas
- Criptografía RSA

Cálculo del Inverso Modular

Para calcular $a^{-1} \bmod n$ (con $\gcd(a, n) = 1$): [language=C, caption=Inverso modular] int inversoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; // inverso no existe // x puede ser negativo, ajustar return (x

Exponenciación Modular

Descripción

Calcular $a^b \bmod n$ eficientemente sin calcular a^b directamente (que puede ser enorme).

Algoritmo de Exponenciación Rápida

[language=C, caption=Exponenciación modular rápida] int potenciaModular(int a, int b, int n) int resultado = 1; a = a

while (b ¿ 0) if (b resultado = (resultado * a) a = (a * a) b = b / 2; return resultado;

Ejemplo

Calcular $2^{10} \bmod 1000$:

```
a=2, b=10, n=1000
b=10 (par): a=4, b=5
b=5 (impar): resultado=4, a=16, b=2
b=2 (par): a=256, b=1
b=1 (impar): resultado=4*256=1024, a=65536, b=0
resultado % 1000 = 24
```

$2^{10} = 1024 \equiv 24 \pmod{1000}$

Análisis de Complejidad

- Tiempo: $O(\log b)$ multiplicaciones modulares
- Espacio: $O(1)$

Introducción a la Criptografía RSA

Fundamentos Matemáticos

RSA se basa en:

1. La dificultad de factorizar números grandes.
2. El teorema de Euler.
3. La existencia de inversos modulares.

Generación de Claves

1. Elegir dos números primos grandes p y q.
2. Calcular $n = p \cdot q$.
3. Calcular $\phi(n) = (p-1)(q-1)$.
4. Elegir e tal que $1 < e < \phi(n)$ y $\gcd(e, \phi(n)) = 1$.
5. Calcular $d = e^{-1} \bmod \phi(n)$ (usando Euclides extendido).

La clave pública es (n, e), la clave privada es (n, d).

Cifrado y Descifrado

- **Cifrado:** $c = m^e \bmod n$
- **Descifrado:** $m = c^d \bmod n$

Ejemplo Simple

```
p=61, q=53
n = 61*53 = 3233
(n) = 60*52 = 3120
e = 17 (primo con 3120)
d = inverso de 17 mod 3120 = 2753
Clave pública: (3233, 17)
Clave privada: (3233, 2753)

Mensaje m = 65
c = 65^17 mod 3233 = 2790
Descifrado: 2790^2753 mod 3233 = 65
```

Implementación Básica de RSA

[language=C, caption=RSA básico] include ¡stdio.h¿ include ¡stdlib.h¿

typedef struct int n; int e; PublicKey;

typedef struct int n; int d; PrivateKey;

// Generar claves (simplificado - solo para números pequeños) void generarClaves(int p, int q, PublicKey* pub, PrivateKey* priv) pub-¿n = p * q; int phi = (p - 1) * (q - 1);

// Elegir e (usamos 65537 en la práctica) pub-¿e = 65537;

// Calcular d int x, y; euclidesExtendido(pub-¿e, phi, x, y); priv-¿d = (x priv-¿n = pub-¿n;

int cifrar(int m, PublicKey* pub) return potenciaModular(m, pub-¿e, pub-¿n);

int descifrar(int c, PrivateKey* priv) return potenciaModular(c, priv-¿d, priv-¿n);

Seguridad de RSA

La seguridad de RSA se basa en:

- Dificultad de factorizar n en p y q.
- Dificultad de calcular $\phi(n)$ sin conocer p y q.
- Dificultad de calcular d a partir de e y n.

Algoritmos Adicionales sobre Números

Criba de Eratóstenes

Para encontrar todos los números primos hasta n. [language=C, caption=Criba de Eratóstenes] void cribaEratostenes(int n) int* esPrimo = (int*)malloc((n + 1) * sizeof(int)); for (int i = 2; i ¡= n; i++) esPrimo[i] = 1;

for (int i = 2; i * i ¡= n; i++) if (esPrimo[i]) for (int j = i * i; j ¡= n; j += i) esPrimo[j] = 0;

for (int i = 2; i ¡= n; i++) if (esPrimo[i]) printf(" free(esPrimo);

Test de Primalidad de Miller-Rabin

Algoritmo probabilístico para determinar si un número es primo.

[language=C, caption=Miller-Rabin] int millerRabin(int n, int k) if (n ¡ 2) return 0; if (n == 2 —— n == 3) return 1; if (n

// Escribir n-1 como d * $2^s intd = n - 1; ints = 0; while(dd/ = 2; s++;$
for (int i = 0; i ¡ k; i++) int a = 2 + rand() int x = potenciaModular(a, d, n); if (x == 1 —— x == n - 1) continue;
int compuesto = 1; for (int r = 0; r ¡ s - 1; r++) x = (x * x) if (x == n - 1) compuesto = 0; break; if (compuesto) return 0; return 1;

Ejercicios Resueltos (20 Ejercicios)

Calcule $\gcd(84, 36)$ usando el algoritmo de Euclides.

```
84 % 36 = 12, a=36, b=12
36 % 12 = 0, a=12, b=0
Resultado: 12
```

Encuentre x e y tales que $84x + 36y = \gcd(84, 36)$. Usando Euclides extendido:

```
84 = 2*36 + 12  -> 12 = 84 - 2*36
36 = 3*12 + 0
```

$x = 1$, $y = -2$: $84 * 1 + 36 * (-2) = 84 - 72 = 12$
Calcule el inverso de 7 módulo 26. $\gcd(7, 26) = 1$, existe inverso.

```
26 = 3*7 + 5 -> 5 = 26 - 3*7
7 = 1*5 + 2 -> 2 = 7 - 1*5 = 7 - (26 - 3*7) = 4*7 - 26
5 = 2*2 + 1 -> 1 = 5 - 2*2 = (26 - 3*7) - 2*(4*7 - 26) = 3*26 - 11*7
```

$-11 \equiv 15 \pmod{26}$. El inverso es 15.
Calcule 3^{20} mod 11 usando exponenciación modular.

```
a=3, b=20, n=11
b=20 par: a=9, b=10
b=10 par: a=81 mod 11 = 4, b=5
b=5 impar: resultado=4, a=16 mod 11 = 5, b=2
b=2 par: a=25 mod 11 = 3, b=1
b=1 impar: resultado=4*3=12 mod 11 = 1
```

$3^{20} \bmod 11 = 1$
Genere claves RSA con $p = 11$, $q = 17$.

```
n = 11 * 17 = 187
(n) = 10 * 16 = 160
e = 7 (primo con 160)
d = inverso de 7 mod 160
7 * 23 = 161  1 mod 160, d = 23
Clave pública: (187, 7)
Clave privada: (187, 23)
```

Cifre $m = 15$ con la clave pública del ejercicio anterior. $c = 15^7 \bmod 187$

```
15^2 = 225 mod 187 = 38
15^4 = 38^2 = 1444 mod 187 = 1444 - 7*187 = 1444 - 1309 = 135
15^7 = 15^4 * 15^2 * 15 = 135 * 38 * 15 mod 187
135 * 38 = 5130 mod 187 = 5130 - 27*187 = 5130 - 5049 = 81
81 * 15 = 1215 mod 187 = 1215 - 6*187 = 1215 - 1122 = 93
```

$c = 93$

Ejercicios Propuestos (20 Ejercicios)

1. Calcule $\gcd(144, 60)$ usando Euclides.
2. Encuentre x e y para $144x + 60y = \gcd(144, 60)$.
3. Calcule el inverso de 9 módulo 14.
4. Calcule $5^{100} \bmod 13$.
5. Genere claves RSA con $p = 13$, $q = 19$.
6. Cifre $m = 42$ con las claves del ejercicio anterior.
7. Descifre el resultado del ejercicio anterior.
8. Implemente el algoritmo de Euclides en Python.
9. Implemente el algoritmo de Euclides extendido en C.
10. ¿Qué es el teorema de Euler y cómo se usa en RSA?
11. ¿Por qué RSA es seguro?
12. ¿Qué es la función de Euler?

13. Calcule (100).
14. Implemente la exponenciación modular rápida.
15. ¿Qué es la criba de Eratóstenes y para qué sirve?
16. Implemente el test de Miller-Rabin.
17. ¿Qué es un número primo y por qué son importantes en criptografía?
18. ¿Cómo se eligen p y q en RSA en la práctica?
19. ¿Qué es el problema de factorización?
20. Investigue aplicaciones de RSA en la vida real.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la aritmética modular.
- El algoritmo de Euclides para MCD ($O(\log n)$).
- El algoritmo de Euclides extendido para resolver $ax + by = \gcd(a, b)$.
- La exponenciación modular rápida ($O(\log b)$).
- Los conceptos básicos de RSA: generación de claves, cifrado y descifrado.
- Algoritmos adicionales: criba de Eratóstenes, test de Miller-Rabin.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120-126.
- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XI: Aritmética Modular

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos de la aritmética modular y sus propiedades.
- Realizar operaciones aritméticas básicas en módulo.
- Calcular inversos modulares utilizando el algoritmo de Euclides extendido.
- Aplicar el teorema chino del resto para resolver sistemas de congruencias.
- Comprender y aplicar el teorema de Fermat y Euler.
- Utilizar la aritmética modular en aplicaciones criptográficas.

Introducción a la Aritmética Modular

Definición de Congruencia

Sean $a, b, n \in Z$ con $n > 0$. Decimos que a es **congruente** a b módulo n, denotado:

$$a \equiv b \pmod{n}$$

si n divide a $a - b$, es decir, $a - b = kn$ para algún entero k.

- $17 \equiv 2 \pmod{5}$ porque $17 - 2 = 15 = 3 \cdot 5$
- $10 \equiv 0 \pmod{2}$ (10 es par)
- $-3 \equiv 4 \pmod{7}$ porque $-3 - 4 = -7 = -1 \cdot 7$

Propiedades Básicas

La congruencia módulo n es una relación de equivalencia:

1. **Reflexiva:** $a \equiv a \pmod{n}$
2. **Simétrica:** $a \equiv b \pmod{n} b \equiv a \pmod{n}$
3. **Transitiva:** $a \equiv b \pmod{n}$ y $b \equiv c \pmod{n} a \equiv c \pmod{n}$

Operaciones en Aritmética Modular

Si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$, entonces:

1. $a + c \equiv b + d \pmod{n}$
2. $a - c \equiv b - d \pmod{n}$
3. $a \cdot c \equiv b \cdot d \pmod{n}$

Conjunto de Residuos

El conjunto de **residuos** módulo n es:

$$Z_n = \{0, 1, 2, \ldots, n-1\}$$

Cada entero es congruente a exactamente un elemento de Z_n.

Operaciones Aritméticas Modulares

Suma Modular

[language=C, caption=Suma modular] int sumaModular(int a, int b, int n) return (a

Resta Modular

[language=C, caption=Resta modular] int restaModular(int a, int b, int n) return ((a

Multiplicación Modular

[language=C, caption=Multiplicación modular] int multModular(int a, int b, int n) return ((long long)(a

Exponenciación Modular

Ya vista en el capítulo anterior, se utiliza para calcular $a^b \bmod n$ eficientemente.

Inverso Modular

Definición

El **inverso modular** de a módulo n es un entero x tal que:

$$a \cdot x \equiv 1 \pmod{n}$$

Se denota $x \equiv a^{-1} \pmod{n}$.

Existencia del Inverso Modular

a tiene inverso módulo n si y solo si $\gcd(a, n) = 1$.

- **Si:** $\gcd(a, n) = 1$, existen enteros x, y tales que $ax + ny = 1$. Entonces $ax \equiv 1 \pmod{n}$.
- **Solo si:** Si $ax \equiv 1 \pmod{n}$, entonces $ax - 1 = kn$, por lo que $ax - kn = 1$. Cualquier divisor común de a y n debe dividir a 1, así que $\gcd(a, n) = 1$.

Cálculo del Inverso Modular con Euclides Extendido

[language=C, caption=Inverso modular con Euclides extendido] int inversoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; // no existe inverso // Asegurar que el resultado sea positivo return (x

Ejemplos de Inversos Modulares

a	n	$a^{-1} \bmod n$
3	7	5 (porque $3 \cdot 5 = 15 \equiv 1 \pmod{7}$)
4	9	7 (porque $4 \cdot 7 = 28 \equiv 1 \pmod{9}$)
2	10	No existe ($\gcd(2, 10) = 2 \neq 1$)

Teorema Chino del Resto (CRT)

Enunciado del Teorema

[Teorema Chino del Resto] Sean $n_1, n_2, \ldots, n_k$ enteros positivos coprimos entre sí (es decir, $\gcd(n_i, n_j) = 1$ para $i \neq j$). Para cualquier conjunto de residuos $a_1, a_2, \ldots, a_k$, el sistema de congruencias:

$$\{\, x \equiv a_1 \pmod{n_1} x \equiv a_2 \pmod{n_2} \vdots x \equiv a_k \pmod{n_k}$$

tiene una solución única módulo $N = n_1 n_2 \cdots n_k$.

Algoritmo de Solución

[language=C, caption=Teorema Chino del Resto] int teoremaChinoResto(int* a, int* n, int k) int N = 1; for (int i = 0; i ¡ k; i++) N *= n[i];

int resultado = 0; for (int i = 0; i ¡ k; i++) int Ni = N / n[i]; int inv = inversoModular(Ni, n[i]); resultado = (resultado + a[i] * Ni * inv) return resultado;

Ejemplo

Resolver:

$$\{\, x \equiv 2 \pmod{3} x \equiv 3 \pmod{5} x \equiv 2 \pmod{7}$$

$N = 3 \cdot 5 \cdot 7 = 105$

- $N_1 = 105/3 = 35$, $35^{-1} \bmod 3 = 2$, término $= 2 \cdot 35 \cdot 2 = 140$
- $N_2 = 105/5 = 21$, $21^{-1} \bmod 5 = 1$, término $= 3 \cdot 21 \cdot 1 = 63$
- $N_3 = 105/7 = 15$, $15^{-1} \bmod 7 = 1$, término $= 2 \cdot 15 \cdot 1 = 30$

Suma: $140 + 63 + 30 = 233 \equiv 233 - 2 \cdot 105 = 23 \pmod{105}$ Solución: $x \equiv 23 \pmod{105}$

Teorema de Fermat y Euler

Teorema de Fermat

[Pequeño Teorema de Fermat] Si p es primo y a no es divisible por p, entonces:

$$a^{p-1} \equiv 1 \pmod{p}$$

Para $p = 7$, $a = 2$: $2^6 = 64 \equiv 1 \pmod{7}$. Para $p = 5$, $a = 3$: $3^4 = 81 \equiv 1 \pmod{5}$.

Función de Euler

La función $\phi(n)$ cuenta los enteros $1 \leq k \leq n$ tales que $\gcd(k, n) = 1$.

Propiedades de

- $\phi(p) = p - 1$ si p es primo.
- $\phi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1)$.
- Si $\gcd(m, n) = 1$, entonces $\phi(mn) = \phi(m) \cdot \phi(n)$.

Teorema de Euler

[Teorema de Euler] Si $\gcd(a, n) = 1$, entonces:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

$\phi(10) = 4$, $3^4 = 81 \equiv 1 \pmod{10}$.

Aplicaciones de la Aritmética Modular

Verificación de Números Primos

El test de primalidad de Miller-Rabin se basa en propiedades de la aritmética modular.

Criptografía RSA

La aritmética modular es la base de RSA:

- Cifrado: $c = m^e \bmod n$
- Descifrado: $m = c^d \bmod n$

Códigos de Verificación

Los dígitos de control (como el DNI o tarjetas de crédito) usan aritmética modular.

Hashing

Las funciones hash modulares son comunes en tablas hash.

Ejercicios Resueltos (20 Ejercicios)

Calcule 27 mod 5. $27 \div 5 = 5$ con resto 2, entonces $27 \bmod 5 = 2$.

Demuestre que $100 \equiv 1 \pmod 9$. $100 - 1 = 99 = 11 \cdot 9$, por lo tanto $100 \equiv 1 \pmod 9$.

Calcule $(15 + 23) \bmod 7$. $15 \bmod 7 = 1$, $23 \bmod 7 = 2$, $1 + 2 = 3$. Por lo tanto, $(15 + 23) \bmod 7 = 3$.

Calcule $(15 \cdot 23) \bmod 7$. $15 \bmod 7 = 1$, $23 \bmod 7 = 2$, $1 \cdot 2 = 2$. Por lo tanto, $(15 \cdot 23) \bmod 7 = 2$.

Encuentre el inverso de 5 módulo 12. Probamos: $5 \cdot 5 = 25 \equiv 1 \pmod{12}$ (porque $25 - 1 = 24$ es múltiplo de 12). El inverso es 5.

¿Existe el inverso de 6 módulo 10? ¿Por qué? No existe porque $\gcd(6, 10) = 2 \neq 1$.

Calcule $3^{10} \bmod 7$ usando el teorema de Fermat. Por Fermat, $3^6 \equiv 1 \pmod 7$. Entonces $3^{10} = 3^6 \cdot 3^4 \equiv 1 \cdot 3^4 = 81 \equiv 4 \pmod 7$.

Calcule $\phi(12)$. Los números coprimos con 12 entre 1 y 12 son: 1, 5, 7, 11. Por lo tanto, $\phi(12) = 4$.

Resuelva $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 4$ usando CRT. $N = 12$, $N_1 = 4$, $4^{-1} \bmod 3 = 1$, término $= 2 \cdot 4 \cdot 1 = 8$. $N_2 = 3$, $3^{-1} \bmod 4 = 3$, término $= 3 \cdot 3 \cdot 3 = 27$. Suma $= 35 \equiv 11 \pmod{12}$.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule 47 mod 8.
2. Demuestre que $144 \equiv 0 \pmod{12}$.
3. Calcule $(25 + 37) \bmod 6$.
4. Calcule $(25 \cdot 37) \bmod 6$.
5. Encuentre el inverso de 3 módulo 8.
6. Encuentre el inverso de 7 módulo 15.
7. ¿Existe el inverso de 9 módulo 12? ¿Por qué?
8. Calcule $2^{15} \bmod 11$ usando Fermat.
9. Calcule $\phi(15)$.
10. Calcule $\phi(30)$.
11. Resuelva $x \equiv 1 \pmod 2$, $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$.

12. Demuestre el teorema de Fermat.
13. Demuestre el teorema de Euler.
14. ¿Qué es un sistema completo de residuos?
15. ¿Qué es un sistema reducido de residuos?
16. Implemente la suma modular en C.
17. Implemente el cálculo del inverso modular.
18. ¿Cómo se usa el CRT en RSA?
19. ¿Qué es la función de Carmichael?
20. Investigue aplicaciones de la aritmética modular en códigos de barras.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la congruencia y la aritmética modular.
- Las operaciones básicas: suma, resta, multiplicación y exponenciación modular.
- El concepto de inverso modular y su cálculo mediante Euclides extendido.
- El Teorema Chino del Resto y su aplicación.
- El teorema de Fermat y el teorema de Euler.
- Aplicaciones en criptografía y verificación.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Rosen, K. H. (2010). *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.
- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XII: Algoritmo Extendido de Euclides

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el funcionamiento del algoritmo de Euclides estándar.
- Comprender y demostrar el algoritmo de Euclides extendido.
- Implementar el algoritmo de Euclides extendido en sus formas recursiva e iterativa.
- Aplicar el algoritmo para calcular inversos modulares.
- Resolver ecuaciones diofánticas lineales.
- Aplicar el algoritmo en criptografía (RSA) y otros contextos.

Algoritmo de Euclides Estándar (Repaso)

Definición del MCD

El **máximo común divisor** (MCD) de dos enteros a y b, denotado $\gcd(a, b)$, es el mayor entero d tal que d divide a a y d divide a b.

Propiedad Fundamental

Para cualquier enteros a y b con $b > 0$:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Algoritmo de Euclides

[language=C, caption=Algoritmo de Euclides estándar] int euclides(int a, int b) while (b != 0) int temp = b; b = a a = temp; return a;

Ejemplo Visual

[scale=0.8] (0,0) rectangle (4,0.5); at (2,0.25) a = 48, b = 18;
(0,-1) rectangle (4,-0.5); at (2,-0.75) 48 % 18 = 12 → a = 18, b = 12;
(0,-2) rectangle (4,-1.5); at (2,-1.75) 18 % 12 = 6 → a = 12, b = 6;
(0,-3) rectangle (4,-2.5); at (2,-2.75) 12 % 6 = 0 → a = 6, b = 0;
[thick, blue] (0,-3.5) rectangle (4,-3); at (2,-3.25) Resultado: gcd = 6;

Algoritmo de Euclides Extendido

Motivación

El algoritmo de Euclides estándar solo calcula el MCD. El algoritmo extendido encuentra además coeficientes x e y tales que:

$$ax + by = \gcd(a, b)$$

Fundamento Matemático

Para cualquier par de enteros a y b (no ambos cero), existen enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

Derivación del Algoritmo

Partimos de la relación recursiva del algoritmo de Euclides:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Si podemos encontrar coeficientes x_1, y_1 tales que:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd(b, a \bmod b)$$

Notando que $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, tenemos:

$$b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 = \gcd(a, b)$$

$$a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = \gcd(a, b)$$

Por lo tanto:

$$x = y_1, \quad y = x_1 - \lfloor a/b \rfloor \cdot y_1$$

Implementación Recursiva

[language=C, caption=Euclides Extendido Recursivo] include ¡stdio.h¿

// Función que devuelve el gcd y calcula x, y tales que a*x + b*y = gcd int euclidesExtendido(int a, int b, int* x, int* y) if (b == 0) *x = 1; *y = 0; return a;

int x1, y1; int gcd = euclidesExtendido(b, a

*x = y1; *y = x1 - (a / b) * y1;

return gcd;

Ejemplo Detallado

Calcular coeficientes para $a = 48$, $b = 18$:

Paso	a	b	a/b	a % b
1	48	18	2	12
2	18	12	1	6
3	12	6	2	0
4	6	0	-	-

Desenrollado:

```
Paso 4: b=0, gcd=6, x=1, y=0
Paso 3: a=12, b=6, q=2
    x = y1 = 0
    y = x1 - q*y1 = 1 - 2*0 = 1
Paso 2: a=18, b=12, q=1
    x = y1 = 1
    y = x1 - q*y1 = 0 - 1*1 = -1
Paso 1: a=48, b=18, q=2
    x = y1 = -1
    y = x1 - q*y1 = 1 - 2*(-1) = 1 + 2 = 3
```

Resultado: $48 \cdot (-1) + 18 \cdot 3 = -48 + 54 = 6$

Implementación Iterativa

[language=C, caption=Euclides Extendido Iterativo] int euclidesExtendidoIterativo(int a, int b, int* x, int* y) int x0 = 1, x1 = 0; int y0 = 0, y1 = 1; int originalA = a; int originalB = b;

while (b != 0) int q = a / b; int r = a

// Actualizar coeficientes int tempX = x1; int tempY = y1; x1 = x0 - q * x1; y1 = y0 - q * y1; x0 = tempX; y0 = tempY;

a = b; b = r;
*x = x0; *y = y0; return a;

Aplicaciones del Algoritmo Extendido

Cálculo del Inverso Modular

El inverso modular de a módulo n existe si y solo si $\gcd(a, n) = 1$. En ese caso, el inverso es x de la ecuación $ax + ny = 1$.

[language=C, caption=Inverso modular con Euclides extendido] int inversoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; // No existe inverso // Asegurar resultado positivo return (x

Calcular $3^{-1} \bmod 7$:

```
euclidesExtendido(3, 7, &x, &y)
  3* -2 + 7*1 = 1
  x = -2  5 mod 7
Inverso: 5 (porque 3*5=151 mod 7)
```

Ecuaciones Diofánticas Lineales

Resolver $ax + by = c$ donde a, b, c son enteros.

La ecuación $ax + by = c$ tiene solución si y solo si $\gcd(a, b)$ divide a c.

[language=C, caption=Resolver ecuación diofántica] int resolverEcuacionDiofantica(int a, int b, int c, int* x, int* y) int x0, y0; int gcd = euclidesExtendido(a, b, x0, y0);

if (c return 0; // No tiene solución

int factor = c / gcd; *x = x0 * factor; *y = y0 * factor;

return 1; // Solución encontrada

Resolver $48x + 18y = 30$:

```
gcd(48,18) = 6, 30 es divisible por 6.
Solución particular: x0 = -1, y0 = 3 (de 48*(-1) + 18*3 = 6)
Multiplicando por factor = 30/6 = 5:
x = -5, y = 15
Verificar: 48*(-5) + 18*15 = -240 + 270 = 30
```

Aplicación en RSA

En RSA, se necesita calcular $d = e^{-1} \bmod \phi(n)$:

```
euclidesExtendido(e, phi, &d, &temp);
d = (d % phi + phi) % phi;
```

Fracciones Continuas

El algoritmo extendido también se puede usar para generar fracciones continuas.

Demostración de Correctitud

El algoritmo de Euclides extendido devuelve enteros x, y tales que $ax + by = \gcd(a, b)$.

Por inducción en el número de iteraciones.

Caso base: Cuando $b = 0$, $\gcd(a, 0) = a$, y la ecuación $a \cdot 1 + 0 \cdot 0 = a$ es válida.

Paso inductivo: Supongamos que para la llamada recursiva con parámetros b y $a \bmod b$ obtenemos:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd(b, a \bmod b)$$

Como $\gcd(a, b) = \gcd(b, a \bmod b)$ y $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, sustituimos:

$$b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 = \gcd(a, b)$$

$$a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = \gcd(a, b)$$

Por lo tanto, $x = y_1$, $y = x_1 - \lfloor a/b \rfloor \cdot y_1$ satisfacen la ecuación.

Análisis de Complejidad

- **Tiempo:** $O(\log \min(a, b))$, igual que el algoritmo de Euclides estándar.
- **Espacio:** $O(\log \min(a, b))$ para la versión recursiva, $O(1)$ para la iterativa.

Comparación de Implementaciones

Característica	**Recursiva**	**Iterativa**
Claridad del código	Alta	Media
Riesgo de desbordamiento de pila	Sí (para números grandes)	No
Eficiencia	Similar	Similar
Facilidad de depuración	Media	Alta

Ejercicios Resueltos (20 Ejercicios)

Calcule $\gcd(56, 21)$ usando el algoritmo de Euclides estándar.

```
56 % 21 = 14, a=21, b=14
21 % 14 = 7, a=14, b=7
14 % 7 = 0, a=7, b=0
Resultado: 7
```

Encuentre x e y tales que $56x + 21y = \gcd(56, 21)$.

```
56 = 2*21 + 14 -> 14 = 56 - 2*21
21 = 1*14 + 7 -> 7 = 21 - 1*14 = 21 - (56 - 2*21) = 3*21 - 56
```

$x = -1$, $y = 3$: $56 * (-1) + 21 * 3 = -56 + 63 = 7$

Calcule el inverso de 17 módulo 31.

```
31 = 1*17 + 14 -> 14 = 31 - 17
17 = 1*14 + 3 -> 3 = 17 - 14 = 17 - (31 - 17) = 2*17 - 31
14 = 4*3 + 2 -> 2 = 14 - 4*3 = (31-17) - 4*(2*17-31) = 31-17-8*17+4*31 = 5*31
3 = 1*2 + 1 -> 1 = 3 - 2 = (2*17-31) - (5*31-9*17) = 11*17 - 6*31
```

$11 * 17 \equiv 1 \pmod{31}$, inverso es 11.

Resuelva $23x + 17y = 1$.

```
23 = 1*17 + 6 -> 6 = 23 - 17
17 = 2*6 + 5 -> 5 = 17 - 2*6 = 17 - 2*(23-17) = 3*17 - 2*23
6 = 1*5 + 1 -> 1 = 6 - 5 = (23-17) - (3*17-2*23) = 3*23 - 4*17
```

$x = 3$, $y = -4$: $23 * 3 + 17 * (-4) = 69 - 68 = 1$

Implemente el algoritmo de Euclides extendido en Python. [language=Python] def euclides$_e xtendido(a, b) : ifb == 0 : returna, 1, 0gcd, x1, y1 = euclides_e xtendido(b, ax = y1y = x1 - (a//b) * y1returngcd, x, y$

¿Cuál es la complejidad del algoritmo de Euclides extendido? $O(\log \min(a, b))$ operaciones aritméticas.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule $\gcd(144, 60)$ usando Euclides.
2. Encuentre x, y para $144x + 60y = \gcd(144, 60)$.
3. Calcule el inverso de 19 módulo 37.
4. Resuelva $31x + 13y = 1$.
5. ¿Existe inverso de 6 módulo 14? Justifique.
6. Implemente Euclides extendido iterativo en C.
7. Demuestre la fórmula de recurrencia del algoritmo extendido.
8. ¿Cómo se usa Euclides extendido en RSA?
9. Resuelva $48x + 18y = 12$.
10. Calcule $\gcd(123456, 789012)$.
11. Encuentre el inverso de 7 módulo 26.
12. Resuelva $15x + 9y = 6$.
13. ¿Qué es una ecuación diofántica lineal?
14. Implemente la función inverso modular usando Euclides extendido.
15. Demuestre que el algoritmo de Euclides extendido es correcto.
16. ¿Cuántas iteraciones hace Euclides extendido para $a = 1$, $b = 10^6$?
17. Calcule $\gcd(2^{10}, 3^5)$.
18. Encuentre x e y para $100x + 37y = 1$.
19. ¿Qué relación hay entre el algoritmo de Euclides y las fracciones continuas?
20. Investigue aplicaciones del algoritmo extendido en criptografía.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El algoritmo de Euclides estándar para calcular el MCD.
- El algoritmo de Euclides extendido para encontrar $ax + by = \gcd(a, b)$.
- Implementaciones recursiva e iterativa.
- Aplicaciones: inverso modular, ecuaciones diofánticas, RSA.
- Análisis de complejidad $O(\log \min(a, b))$.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.
- Rosen, K. H. (2010). *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.

Unidad XIII: Nociones de Criptografía Básica (RSA)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los conceptos fundamentales de la criptografía.
- Distinguir entre criptografía simétrica y asimétrica.
- Comprender el funcionamiento del algoritmo RSA.
- Implementar las operaciones básicas de RSA.
- Comprender los fundamentos matemáticos de RSA.
- Analizar la seguridad de RSA y sus aplicaciones prácticas.

Introducción a la Criptografía

Definición

La **criptografía** es la práctica y estudio de técnicas para asegurar comunicaciones en presencia de adversarios. Los objetivos principales son:

- **Confidencialidad:** Solo el destinatario puede leer el mensaje.
- **Integridad:** El mensaje no puede ser alterado sin detección.
- **Autenticación:** Se verifica la identidad del remitente.
- **No repudio:** El remitente no puede negar haber enviado el mensaje.

Criptografía Simétrica

En la criptografía simétrica, el mismo secreto se usa para cifrar y descifrar.

[scale=0.8] (msg) at (0,0) Mensaje; (cifrado) at (2,0) Cifrado; (descifrado) at (4,0) Mensaje; (clave) at (2,1) Clave;
[-¿] (msg) – (cifrado); [-¿] (cifrado) – (descifrado); [-¿] (clave) – (cifrado); [-¿] (clave) – (descifrado);

Ejemplos: AES, DES, ChaCha20.

Criptografía Asimétrica (Clave Pública)

En la criptografía asimétrica, se usan dos claves diferentes: una pública para cifrar y una privada para descifrar.

[scale=0.8] (msg) at (0,0) Mensaje; (cifrado) at (2.5,0) Cifrado; (descifrado) at (5,0) Mensaje; (pub) at (2.5,1) Clave Pública; (priv) at (2.5,-1) Clave Privada;
[-¿] (msg) – (cifrado); [-¿] (cifrado) – (descifrado); [-¿] (pub) – (cifrado); [-¿] (priv) – (descifrado);

Ejemplos: RSA, ECC, ElGamal.

Fundamentos Matemáticos de RSA

Teorema de Euler

Si $\gcd(a, n) = 1$, entonces $a^{\phi(n)} \equiv 1 \pmod{n}$, donde $\phi(n)$ es la función de Euler.

Función de Euler

- $\phi(p) = p - 1$ para p primo.
- $\phi(pq) = (p - 1)(q - 1)$ para p y q primos distintos.

Pequeño Teorema de Fermat

Si p es primo y a no es múltiplo de p, entonces $a^{p-1} \equiv 1 \pmod{p}$.

El Algoritmo RSA

Generación de Claves

1. Elegir dos números primos grandes p y q.
2. Calcular $n = p \cdot q$.
3. Calcular $\phi(n) = (p-1)(q-1)$.
4. Elegir e tal que $1 < e < \phi(n)$ y $\gcd(e, \phi(n)) = 1$.
5. Calcular $d = e^{-1} \bmod \phi(n)$ (usando Euclides extendido).

La clave pública es (n, e). La clave privada es (n, d).

Cifrado

Para cifrar un mensaje m (con $0 \leq m < n$):

$$c = m^e \bmod n$$

Descifrado

Para descifrar un mensaje cifrado c:

$$m = c^d \bmod n$$

Demostración de Correctitud

Para cualquier mensaje $0 \leq m < n$, se cumple que $(m^e)^d \equiv m \pmod{n}$.

Como $ed \equiv 1 \pmod{\phi(n)}$, existe k tal que $ed = 1 + k\phi(n)$.

$$(m^e)^d = m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k$$

Por el teorema de Euler, $m^{\phi(n)} \equiv 1 \pmod{n}$, por lo que $(m^e)^d \equiv m \pmod{n}$.

Implementación de RSA

Estructuras de Datos

[language=C, caption=Estructuras para RSA] typedef struct long long n; // módulo long long e; // exponente público PublicKey;

typedef struct long long n; // módulo long long d; // exponente privado PrivateKey;

Generación de Claves

[language=C, caption=Generación de claves RSA (simplificada)] include ¡stdio.h¿ include ¡stdlib.h¿ include ¡time.h¿

// Función para verificar si un número es primo (simplificada) int esPrimo(int n) if (n ¡ 2) return 0; for (int i = 2; i * i ¡= n; i++) if (n return 1;

// Generar un número primo aleatorio int generarPrimo() int primo; do primo = rand() while (!esPrimo(primo)); return primo;

// Euclides extendido int euclidesExtendido(int a, int b, int* x, int* y) if (b == 0) *x = 1; *y = 0; return a; int x1, y1; int gcd = euclidesExtendido(b, a *x = y1; *y = x1 - (a / b) * y1; return gcd;

// Inverso modular int inversoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; return (x

// Exponenciación modular rápida long long potenciaModular(long long a, long long b, long long n) long long resultado = 1; a = a while (b ¿ 0) if (b resultado = (resultado * a) a = (a * a) b = b / 2; return resultado;

void generarClaves(PublicKey* pub, PrivateKey* priv) srand(time(NULL));

int p = generarPrimo(); int q = generarPrimo(); while (q == p) q = generarPrimo();

pub-¿n = p * q; priv-¿n = pub-¿n;

int phi = (p - 1) * (q - 1);

// Elegir e (comúnmente 65537) pub-¿e = 65537;

// Calcular d priv-¿d = inversoModular(pub-¿e, phi);

long long cifrar(long long m, PublicKey* pub) return potenciaModular(m, pub-¿e, pub-¿n);

long long descifrar(long long c, PrivateKey* priv) return potenciaModular(c, priv-¿d, priv-¿n);

Ejemplo Completo

[language=C, caption=Ejemplo de uso de RSA] int main() PublicKey pub; PrivateKey priv;

generarClaves(pub, priv);

printf("Clave pública: (n=printf("Clave privada: (n=

long long mensaje = 42; printf("Mensaje original:

long long cifrado = cifrar(mensaje, pub); printf("Mensaje cifrado:

long long descifrado = descifrar(cifrado, priv); printf("Mensaje descifrado:

return 0;

Seguridad de RSA

Base de la Seguridad

La seguridad de RSA se basa en la dificultad de:

1. Factorizar n en sus factores primos p y q.
2. Calcular $\phi(n)$ sin conocer p y q.
3. Calcular la raíz e-ésima módulo n.

Tamaños de Clave Recomendados

Año	**Tamaño de clave (bits)**
2020	2048
2030	3072
2040	4096

Ataques Conocidos

- **Ataque de factorización:** Usando el algoritmo de la criba general de cuerpos de números (GNFS).
- **Ataque de texto cifrado elegido:** Si el descifrado no está blindado.
- **Ataque de canal lateral:** Basado en tiempo, consumo de energía, etc.
- **Ataque de paridad:** Si se revela un bit de información por vez.

Aplicaciones de RSA

Firma Digital

1. El remitente calcula el hash h del mensaje.
2. Firma: $s = h^d \bmod n$.
3. El receptor verifica: $h' = s^e \bmod n$ y compara con el hash del mensaje.

Intercambio de Claves

RSA se usa para intercambiar una clave simétrica de forma segura.

Certificados Digitales (X.509)

Los certificados SSL/TLS usan RSA para autenticación.

Ejercicios Resueltos (20 Ejercicios)

Dados $p = 11$, $q = 13$, calcule n y $\phi(n)$. $n = 11 \cdot 13 = 143$ $\phi(n) = (11-1)(13-1) = 10 \cdot 12 = 120$

Con $p = 11$, $q = 13$, elija $e = 7$ y calcule d. Necesitamos d tal que $7d \equiv 1 \pmod{120}$. $7 \cdot 103 = 721 \equiv 1 \pmod{120}$, por lo que $d = 103$.

Cifre $m = 5$ usando $n = 143$, $e = 7$. $c = 5^7 \bmod 143$ $5^2 = 25$, $5^4 = 25^2 = 625 \equiv 625 - 4 \cdot 143 = 625 - 572 = 53$ $5^7 = 5^4 \cdot 5^2 \cdot 5 = 53 \cdot 25 \cdot 5 = 53 \cdot 125 = 6625$ $6625 \bmod 143$: $143 \cdot 46 = 6578$, $6625 - 6578 = 47$ $c = 47$

Descifre $c = 47$ usando $n = 143$, $d = 103$. $m = 47^{103} \bmod 143$ Por construcción, debe dar 5.

¿Por qué e debe ser coprimo con $\phi(n)$? Para que exista $d = e^{-1} \bmod \phi(n)$. Si $\gcd(e, \phi(n)) \neq 1$, el inverso modular no existe.

¿Qué tamaño de clave RSA se recomienda actualmente? Al menos 2048 bits.

¿Cuál es la diferencia entre criptografía simétrica y asimétrica? Simétrica: misma clave para cifrar y descifrar. Asimétrica: clave pública para cifrar, clave privada para descifrar.

¿Qué es una firma digital RSA? Es un hash del mensaje cifrado con la clave privada. El receptor verifica descifrando con la clave pública.

¿Por qué RSA no se usa para cifrar mensajes largos? Porque es lento. En su lugar, se usa RSA para intercambiar una clave simétrica, y esa clave se usa para cifrar el mensaje largo.

¿Qué es el ataque de factorización a RSA? Consiste en encontrar p y q a partir de n, lo que permite calcular $\phi(n)$ y luego d.

Ejercicios Propuestos (20 Ejercicios)

1. Genere claves RSA con $p = 13$, $q = 17$.
2. Cifre $m = 10$ con las claves del ejercicio anterior.
3. Descifre el resultado del ejercicio anterior.
4. ¿Qué es la función ϕ de Euler?
5. ¿Qué es el teorema de Fermat?

6. ¿Por qué se elige $e = 65537$ comúnmente?
7. Implemente la generación de claves RSA en Python.
8. ¿Qué es un ataque de canal lateral?
9. ¿Qué es un certificado digital?
10. ¿Cómo se usa RSA en SSL/TLS?
11. ¿Qué es el problema de factorización?
12. ¿Qué tamaño de clave se considera seguro para 2030?
13. ¿Qué es un texto cifrado elegido?
14. Implemente una firma digital RSA.
15. ¿Qué es el hash criptográfico y para qué se usa?
16. ¿Qué es la diferencia entre RSA y ECC?
17. ¿Qué es el algoritmo de factorización GNFS?
18. ¿Cómo se generan números primos grandes para RSA?
19. ¿Qué es la criptografía post-cuántica?
20. Investigue aplicaciones de RSA en el mundo real.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la criptografía simétrica y asimétrica.
- El algoritmo RSA: generación de claves, cifrado y descifrado.
- Los fundamentos matemáticos: teorema de Euler y función ϕ.
- La implementación práctica de RSA.
- La seguridad de RSA y sus aplicaciones.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
  item Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120-126.

- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XIV: Backtracking (Vuelta Atrás) - Búsqueda Exhaustiva con Poda

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la técnica de backtracking y su aplicación en problemas combinatorios.
- Implementar algoritmos de backtracking para problemas clásicos.
- Aplicar técnicas de poda para optimizar la búsqueda.
- Analizar la complejidad de algoritmos de backtracking.
- Distinguir entre backtracking y otras técnicas como programación dinámica.
- Resolver problemas de optimización combinatoria mediante backtracking.

Introducción al Backtracking

Definición

El **backtracking** (vuelta atrás) es una técnica algorítmica para resolver problemas de manera recursiva, construyendo una solución paso a paso, y deshaciendo los pasos que no conducen a una solución válida.

Filosofía

[scale=0.7] [circle, draw] (root) at (0,0) Inicio; [circle, draw] (a1) at (-2,-1) Opción 1; [circle, draw] (a2) at (0,-1) Opción 2; [circle, draw] (a3) at (2,-1) Opción 3;
[circle, draw] (b1) at (-3,-2) 1.1; [circle, draw] (b2) at (-1,-2) 1.2; [circle, draw] (b3) at (1,-2) 2.1; [circle, draw] (b4) at (3,-2) 3.1;
[circle, draw] (c1) at (-3.5,-3) Solución;
[-¿] (root) – (a1); [-¿] (root) – (a2); [-¿] (root) – (a3); [-¿] (a1) – (b1); [-¿] (a1) – (b2); [-¿] (a2) – (b3); [-¿] (a3) – (b4); [-¿] (b1) – (c1);
[red, thick] (a2) – (b3); [red, thick] (b3) – (c1);

Características

- **Búsqueda exhaustiva:** Explora todas las posibilidades.
- **Poda:** Elimina ramas que no pueden llevar a una solución.
- **Recursividad:** Naturalmente recursivo.
- **Vuelta atrás:** Deshace decisiones para probar alternativas.

Cuándo usar Backtracking

- Problemas de decisión (encontrar una solución).
- Problemas de optimización (encontrar la mejor solución).
- Problemas de enumeración (contar todas las soluciones).
- Espacio de búsqueda combinatorio grande pero estructurado.

Esquema General del Backtracking

Pseudocódigo General

[language=C, caption=Esquema general de backtracking] int backtracking(estado actual) if (esSolucion(estado actual)) procesarSolucion(estado actual); return 1; // o contar según problema
int soluciones = 0;
for each (movimiento in generarMovimientos(estado actual)) if (esValido(movimiento)) aplicarMovimiento(movimiento); soluciones += backtracking(nuevo estado); deshacerMovimiento(movimiento); // vuelta atrás
return soluciones;

Componentes Clave

1. **Estado:** Representación de la solución parcial.
2. **Criterio de solución:** Cuándo se ha alcanzado una solución completa.
3. **Generación de movimientos:** Cómo extender la solución parcial.
4. **Validación:** Verificar si un movimiento es prometedor (poda).
5. **Aplicación y deshacer:** Modificar y restaurar el estado.

Problemas Clásicos de Backtracking

Problema de las N Reinas

Descripción

Colocar n reinas en un tablero de $n \times n$ sin que se ataquen entre sí.

Implementación

[language=C, caption=N Reinas] include ¡stdio.h¿ include ¡stdlib.h¿ include ¡math.h¿

define N 8

int tablero[N][N]; int soluciones = 0;

int esSeguro(int fila, int col) // Verificar misma columna for (int i = 0; i ¡ fila; i++) if (tablero[i][col]) return 0;

// Verificar diagonal superior izquierda for (int i = fila, j = col; i ¿= 0 j ¿= 0; i–, j–) if (tablero[i][j]) return 0;

// Verificar diagonal superior derecha for (int i = fila, j = col; i ¿= 0 j ¡ N; i–, j++) if (tablero[i][j]) return 0;

return 1;

void resolverNReinas(int fila) if (fila == N) // Solución encontrada soluciones++; printf("Solución for (int i = 0; i ¡ N; i++) for (int j = 0; j ¡ N; j++) printf(" printf(""); printf(""); return;

for (int col = 0; col ¡ N; col++) if (esSeguro(fila, col)) tablero[fila][col] = 1; resolverNReinas(fila + 1); tablero[fila][col] = 0; // backtracking

Optimización con Arrays de Seguimiento

[language=C, caption=N Reinas optimizado] int columnas[N]; int diag1[2*N - 1]; int diag2[2*N - 1]; int reinas[N]; // reinas[fila] = columna

void resolverNReinasOptimizado(int fila) if (fila == N) soluciones++; return;

for (int col = 0; col ¡ N; col++) if (!columnas[col] !diag1[fila - col + N - 1] !diag2[fila + col]) columnas[col] = 1; diag1[fila - col + N - 1] = 1; diag2[fila + col] = 1; reinas[fila] = col;

resolverNReinasOptimizado(fila + 1);

columnas[col] = 0; diag1[fila - col + N - 1] = 0; diag2[fila + col] = 0;

Problema del Laberinto

Descripción

Encontrar un camino desde el inicio hasta la salida en un laberinto.

Implementación

[language=C, caption=Laberinto] define TAM 5

int laberinto[TAM][TAM] = 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1 ;

int solucion[TAM][TAM] = 0;

int esValido(int x, int y) return (x ¿= 0 x ¡ TAM y ¿= 0 y ¡ TAM laberinto[x][y] == 1 !solucion[x][y]);

int resolverLaberinto(int x, int y) if (x == TAM - 1 y == TAM - 1) solucion[x][y] = 1; return 1;

if (esValido(x, y)) solucion[x][y] = 1;

// Intentar en las 4 direcciones if (resolverLaberinto(x + 1, y)) return 1; if (resolverLaberinto(x - 1, y)) return 1; if (resolverLaberinto(x, y + 1)) return 1; if (resolverLaberinto(x, y - 1)) return 1;

solucion[x][y] = 0; // backtracking return 0; return 0;

Problema del Sudoku

Descripción

Completar una cuadrícula de 9x9 con números del 1 al 9 sin repetir en fila, columna o subcuadrícula de 3x3.

Implementación

[language=C, caption=Sudoku] define TAM$_SUDOKU9$

int sudoku[TAM$_SUDOKU$][TAM_SUDOKU] = $5, 3, 0, 0, 7, 0, 0, 0, 0, 6, 0, 0, 1, 9, 5, 0, 0, 0, 0,$

int esValidoSudoku(int fila, int col, int num) // Verificar fila for (int i = 0; i ¡ TAM$_S UDOKU; i++) if(sudoku[fila][i] == num) return 0;

// Verificar columna for (int i = 0; i ¡ TAM$_S UDOKU; i$++)$if(sudoku[i][col] == num)return0;

// Verificar subcuadrícula 3x3 int inicioFila = (fila / 3) * 3; int inicioCol = (col / 3) * 3; for (int i = inicioFila; i ¡ inicioFila + 3; i++) for (int j = inicioCol; j ¡ inicioCol + 3; j++) if (sudoku[i][j] == num) return 0;

return 1;

int resolverSudoku() for (int fila = 0; fila ¡ TAM$_S UDOKU; fila + +)for(intcol = 0; col < TAM_S UDOKU; col + +)if(sudoku[fila][col] == 0)for(intnum = 1; num <$

if (resolverSudoku()) return 1;

sudoku[fila][col] = 0; // backtracking return 0; return 1;

Problema de la Mochila (0/1) con Backtracking

Implementación con Poda

[language=C, caption=Mochila con backtracking] typedef struct int peso; int valor; Objeto;

Objeto objetos[] = 2, 3, 3, 4, 4, 5, 5, 6; int n = 4; int capacidad = 7; int mejorValor = 0; int mejorSeleccion[4];

void mochilaBacktracking(int i, int pesoActual, int valorActual, int* seleccion) if (i == n) if (valorActual ¿ mejorValor) mejorValor = valorActual; for (int j = 0; j ¡ n; j++) mejorSeleccion[j] = seleccion[j]; return;

// Opción 1: no tomar el objeto i seleccion[i] = 0; mochilaBacktracking(i + 1, pesoActual, valorActual, seleccion);

// Opción 2: tomar el objeto i si cabe if (pesoActual + objetos[i].peso ¡= capacidad) seleccion[i] = 1; mochilaBacktracking(i + 1, pesoActual + objetos[i].peso, valorActual + objetos[i].valor, seleccion);

Técnicas de Poda

Poda por Optimalidad

[language=C, caption=Poda por optimalidad en mochila] // Cota superior: valor actual + valor máximo posible de los objetos restantes int cotaSuperior(int i, int valorActual, int pesoRestante) int valorEstimado = valorActual; int peso = pesoRestante;

for (int j = i; j ¡ n peso ¿= objetos[j].peso; j++) valorEstimado += objetos[j].valor; peso -= objetos[j].peso;

return valorEstimado;

void mochilaBacktrackingPoda(int i, int pesoActual, int valorActual, int* seleccion) if (i == n) if (valorActual ¿ mejorValor) mejorValor = valorActual; for (int j = 0; j ¡ n; j++) mejorSeleccion[j] = seleccion[j]; return;

// Poda: si la cota superior no mejora la mejor solución actual if (cotaSuperior(i, valorActual, capacidad - pesoActual) ¡= mejorValor) return;

// Opción 1: no tomar seleccion[i] = 0; mochilaBacktrackingPoda(i + 1, pesoActual, valorActual, seleccion);

// Opción 2: tomar if (pesoActual + objetos[i].peso ¡= capacidad) seleccion[i] = 1; mochilaBacktrackingPoda(i + 1, pesoActual + objetos[i].peso, valorActual + objetos[i].valor, seleccion);

Poda por Factibilidad

Verificar si una solución parcial puede llevar a una solución completa.

Poda por Simetría

Eliminar ramas equivalentes por simetría del problema.

Análisis de Complejidad

Complejidad en el Peor Caso

- N Reinas: $O(n!)$
- Sudoku: $O(9^{81})$ (con poda, mucho menos)
- Laberinto: $O(4^n)$ donde n es el número de celdas

Mejoras Comunes

- Ordenamiento de movimientos (los más prometedores primero)
- Poda por poda de optimalidad
- Memoización (evitar repetir estados)
- Heurísticas para ordenar la búsqueda

Ejercicios Resueltos (20 Ejercicios)

Resuelva el problema de las 4 reinas manualmente. Una solución para 4 reinas: fila0: col2, fila1: col0, fila2: col3, fila3: col1.

¿Cuántas soluciones tiene el problema de las 8 reinas? 92 soluciones distintas (12 únicas considerando simetrías).

¿Qué es la poda en backtracking? Es la técnica de descartar ramas de búsqueda que no pueden llevar a una solución mejor que la ya encontrada.

Ejercicios Propuestos (20 Ejercicios)

1. Implemente el problema de las N reinas para N=10.
2. Resuelva un laberinto de 10x10 con backtracking.
3. Implemente un generador de sudokus usando backtracking.
4. ¿Qué es el problema del caballo (knight's tour)?
5. Implemente la búsqueda de todas las soluciones del problema de la mochila.
6. ¿Cómo se puede mejorar la poda en el problema de la mochila?
7. Implemente el problema de coloreo de grafos con backtracking.
8. ¿Qué es el problema del circuito de Hamilton?
9. Implemente un algoritmo de backtracking para el problema de la suma de subconjuntos.
10. ¿Qué diferencia hay entre backtracking y ramificación y poda (branch and bound)?
11. Implemente un generador de permutaciones con backtracking.
12. Resuelva el problema de las Torres de Hanoi con backtracking.
13. ¿Qué es la poda por factibilidad?
14. Implemente la búsqueda de caminos en un grafo con backtracking.
15. ¿Cómo se puede paralelizar backtracking?

16. Implemente un algoritmo de backtracking para el problema de la asignación.
17. ¿Qué es el problema de la partición y cómo se resuelve con backtracking?
18. Implemente un algoritmo para encontrar todas las combinaciones de suma objetivo.
19. ¿Qué es la poda por simetría?
20. Investigue aplicaciones de backtracking en inteligencia artificial.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El concepto de backtracking como búsqueda exhaustiva con vuelta atrás.
- La implementación de problemas clásicos: N reinas, laberinto, sudoku, mochila.
- Las técnicas de poda para optimizar la búsqueda.
- El análisis de complejidad de algoritmos de backtracking.
- La aplicación en problemas de optimización combinatoria.

Bibliografía de la Unidad

- Brassard, G., & Bratley, P. (1995). *Fundamentals of Algorithmics*. Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Knuth, D. E. (2011). *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms*. Addison-Wesley.

Unidad XV: Programación Dinámica (Dynamic Programming)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el principio de optimalidad de Bellman.
- Identificar problemas que pueden resolverse con programación dinámica.
- Implementar soluciones de programación dinámica para problemas clásicos.
- Distinguir entre enfoques top-down (memoización) y bottom-up (tabular).
- Analizar la complejidad temporal y espacial de soluciones de programación dinámica.
- Aplicar programación dinámica a problemas de optimización.

Introducción a la Programación Dinámica

Definición

La **programación dinámica** es una técnica de diseño de algoritmos que resuelve problemas complejos dividiéndolos en subproblemas más simples, almacenando los resultados de estos subproblemas para evitar recalcularlos.

Características Clave

1. **Subestructura óptima:** Una solución óptima contiene soluciones óptimas de subproblemas.
2. **Subproblemas superpuestos:** Los mismos subproblemas aparecen repetidamente.
3. **Memorización:** Almacenar resultados para reutilizarlos.

Principio de Optimalidad de Bellman

[Principio de Optimalidad] Una política óptima tiene la propiedad de que, cualquiera sea el estado inicial y la decisión inicial, las decisiones restantes deben constituir una política óptima con respecto al estado resultante de la primera decisión.

Enfoques de Programación Dinámica

[scale=0.7] (0,1) rectangle (4,2); at (2,1.5) Top-down (Memoización); at (2,0.5) Recursivo + almacenamiento; at (2,0) Procesa solo subproblemas necesarios;
(6,1) rectangle (10,2); at (8,1.5) Bottom-up (Tabular); at (8,0.5) Iterativo + tabla; at (8,0) Procesa todos los subproblemas;

Problema de la Mochila (0/1)

Descripción del Problema

Dados n objetos, cada uno con peso w_i y valor v_i, y una mochila con capacidad W, seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.

Ecuación de Recurrencia

Sea $dp[i][j]$ el valor máximo que se puede obtener con los primeros i objetos y capacidad j:

$$dp[i][j] = \max(dp[i-1][j],\ dp[i-1][j-w_i] + v_i) \quad sij \geq w_i$$

$$dp[i][j] = dp[i-1][j] \quad sij < w_i$$

Caso base: $dp[0][j] = 0$ para todo j.

Implementación Bottom-up

[language=C, caption=Mochila 0/1 - Bottom-up] include ¡stdio.h¿ include ¡string.h¿

define MAX$_N$100$defineMAX_W$1000

int mochila01(int n, int W, int pesos[], int valores[]) int dp[MAX$_N$ + 1][$MAX_W + 1$];

// Inicializar casos base for (int j = 0; j ¡= W; j++) dp[0][j] = 0;

// Llenar tabla for (int i = 1; i ¡= n; i++) for (int j = 0; j ¡= W; j++) if (j ¿= pesos[i-1]) int tomar = dp[i-1][j - pesos[i-1]] + valores[i-1]; int noTomar = dp[i-1][j]; dp[i][j] = (tomar ¿ noTomar) ? tomar : noTomar; else dp[i][j] = dp[i-1][j];

return dp[n][W];

Optimización de Espacio

[language=C, caption=Mochila 0/1 - Optimización de espacio] int mochila01Optimizada(int n, int W, int pesos[], int valores[]) int dp[MAX$_W$ + 1];

// Inicializar memset(dp, 0, sizeof(dp));

// Llenar tabla (recorriendo W hacia atrás) for (int i = 0; i ¡ n; i++) for (int j = W; j ¿= pesos[i]; j–) int tomar = dp[j - pesos[i]] + valores[i]; if (tomar ¿ dp[j]) dp[j] = tomar;

return dp[W];

Ejemplo Paso a Paso

$n = 4$, $W = 7$, objetos: (2,3), (3,4), (4,5), (5,6)

i/j	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7
3	0	0	3	4	5	7	8	9
4	0	0	3	4	5	7	8	9

Valor máximo: 9

Distancia de Edición (Levenshtein)

Descripción del Problema

Dadas dos cadenas A y B, encontrar el número mínimo de operaciones (inserción, eliminación, sustitución) para transformar A en B.

Ecuación de Recurrencia

Sea $dp[i][j]$ la distancia de edición entre los primeros i caracteres de A y los primeros j caracteres de B:

$$dp[i][j] = \min \{ d\,p[i-1][j]+1(eliminación)dp[i][j-1]+1(inserción)dp[i-1][j-1]+costo(susti$$

donde $costo = 0$ si $A[i-1] = B[j-1]$, $costo = 1$ en caso contrario.

Caso base: $dp[i][0] = i$, $dp[0][j] = j$.

Implementación

[language=C, caption=Distancia de Levenshtein] include ¡stdio.h¿ include ¡string.h¿ include ¡stdlib.h¿

int min(int a, int b, int c) int m = a; if (b ¡ m) m = b; if (c ¡ m) m = c; return m;

int distanciaEdicion(char* A, char* B) int m = strlen(A); int n = strlen(B);

// Crear tabla int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i ¡= m; i++) dp[i] = (int*)malloc((n + 1) * sizeof(int));

// Casos base for (int i = 0; i ¡= m; i++) dp[i][0] = i; for (int j = 0; j ¡= n; j++) dp[0][j] = j;

// Llenar tabla for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) int costo = (A[i-1] == B[j-1]) ? 0 : 1; dp[i][j] = min(dp[i-1][j] + 1, // eliminación dp[i][j-1] + 1, // inserción dp[i-1][j-1] + costo // sustitución);

int resultado = dp[m][n];

// Liberar memoria for (int i = 0; i ¡= m; i++) free(dp[i]); free(dp);

return resultado;

Ejemplo

Transformar "CASA" en "PERRO"

		C	A	S	A
	0	1	2	3	4
P	1	1	2	3	4
E	2	2	2	3	4
R	3	3	3	3	4
R	4	4	4	4	4
O	5	5	5	5	5

Distancia: 5

Subsecuencia Común Más Larga (LCS)

Descripción del Problema

Dadas dos secuencias X e Y, encontrar la subsecuencia (no necesariamente contigua) más larga que aparece en ambas.

Ecuación de Recurrencia

Sea $dp[i][j]$ la longitud de la LCS entre los primeros i caracteres de X y los primeros j caracteres de Y:

$dp[i][j] = \{ 0\ sii = 0oj = 0dp[i-1][j-1]+1siX[i-1] = Y[j-1]\max(dp[i-1][j], dp[i][j-1])encasocontr$

Implementación

[language=C, caption=LCS] include ¡stdio.h¿ include ¡string.h¿ include ¡stdlib.h¿
int max(int a, int b) return (a ¿ b) ? a : b;
int lcs(char* X, char* Y) int m = strlen(X); int n = strlen(Y);
int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i ¡= m; i++) dp[i] = (int*)calloc(n + 1, sizeof(int));
for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) if (X[i-1] == Y[j-1]) dp[i][j] = dp[i-1][j-1] + 1; else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
int resultado = dp[m][n];
// Liberar memoria for (int i = 0; i ¡= m; i++) free(dp[i]); free(dp);
return resultado;

Recuperación de la Subsecuencia

[language=C, caption=Recuperar LCS] void recuperarLCS(char* X, char* Y, int** dp, int i, int j, char* resultado, int* idx) if (i == 0 —— j == 0) return;

if (X[i-1] == Y[j-1]) recuperarLCS(X, Y, dp, i-1, j-1, resultado, idx); resultado[(*idx)++] = X[i-1]; else if (dp[i-1][j] ¿ dp[i][j-1]) recuperarLCS(X, Y, dp, i-1, j, resultado, idx); else recuperarLCS(X, Y, dp, i, j-1, resultado, idx);

Ejemplo

$X =$ "$ABCBDAB$", $Y =$ "$BDCABA$"

		A	B	C	B	D	A
B							
	0	0	0	0	0	0	0
0							
B	0	0	1	1	1	1	1
1							
D	0	0	1	1	1	2	2
2							
C	0	0	1	2	2	2	2
2							
A	0	1	1	2	2	2	3
3							
B	0	1	2	2	3	3	3
4							
A	0	1	2	2	3	3	4
4							

LCS: "BCAB" o "BDAB", longitud 4.

Optimización de Espacio para LCS

[language=C, caption=LCS optimizado] int lcsOptimizado(char* X, char* Y) int m = strlen(X); int n = strlen(Y);

// Usar solo 2 filas int** dp = (int**)malloc(2 * sizeof(int*)); dp[0] = (int*)calloc(n + 1, sizeof(int)); dp[1] = (int*)calloc(n + 1, sizeof(int));

for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) if (X[i-1] == Y[j-1]) dp[i else dp[i

int resultado = dp[m free(dp[0]); free(dp[1]); free(dp);

return resultado;

Comparación de Enfoques

Problema	Tiempo	Espacio	Espacio optimizado
Mochila (0/1)	$O(nW)$	$O(nW)$	$O(W)$
Distancia de edición	$O(mn)$	$O(mn)$	$O(\min(m,n))$
LCS	$O(mn)$	$O(mn)$	$O(\min(m,n))$

Ejercicios Resueltos (20 Ejercicios)

Resuelva la mochila 0/1 con $n = 3$, $W = 5$, objetos: (2,3), (3,4), (4,5).

```
dp[0][*]=0
i=1: j=2: dp=3; j=3,4,5: dp=3
i=2: j=3: dp=4; j=5: dp=3+4=7
i=3: j=4: dp=5; j=5: dp=5 vs 7 -> 7
Valor máximo: 7
```

Calcule la distancia de edición entre "gato" y "perro".

```
gato -> perro: 4 sustituciones (g->p, a->e, t->r, o->r?) +1 inserción = 5
```

Encuentre LCS de "ABCD" y "ACBD". LCS: "ABD" o "ACD", longitud 3.

Ejercicios Propuestos (20 Ejercicios)

1. Resuelva mochila 0/1 con n=4, W=10, objetos (2,5),(3,7),(4,8),(5,9).
2. Implemente distancia de edición con operaciones de costo variable.
3. Encuentre LCS de "XMJYAUZ" y "MZJAWXU".
4. ¿Qué es el principio de optimalidad de Bellman?
5. Implemente mochila 0/1 con recuperación de objetos seleccionados.
6. ¿Cuál es la diferencia entre top-down y bottom-up?
7. Resuelva el problema de la subsecuencia creciente máxima con PD.
8. Implemente distancia de edición con recuperación de operaciones.

9. ¿Qué es la matriz de costos en LCS?
10. Resuelva el problema de la partición con PD.
11. Implemente LCS con recuperación de todas las soluciones.
12. ¿Cómo se puede mejorar la complejidad espacial de distancia de edición?
13. Resuelva el problema del cambio de monedas con PD.
14. Implemente mochila 0/1 con objetos fraccionarios? (no es PD)
15. ¿Qué es la subestructura óptima?
16. Resuelva el problema del viajante con PD (TSP).
17. Implemente la multiplicación de matrices con PD.
18. ¿Cuándo usar programación dinámica vs greedy?
19. Resuelva el problema de la distancia de Damerau-Levenshtein.
20. Investigue aplicaciones de PD en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El principio de optimalidad de Bellman.
- La diferencia entre enfoques top-down y bottom-up.
- La implementación de tres problemas clásicos: mochila 0/1, distancia de edición y LCS.
- Técnicas de optimización de espacio.
- El análisis de complejidad de soluciones de programación dinámica.

Bibliografía de la Unidad

- Bellman, R. (1957). *Dynamic Programming.* Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 15.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.

Unidad XV: Programación Dinámica (Dynamic Programming)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el principio de optimalidad de Bellman.
- Identificar problemas que pueden resolverse con programación dinámica.
- Implementar soluciones de programación dinámica para problemas clásicos.
- Distinguir entre enfoques top-down (memoización) y bottom-up (tabular).
- Analizar la complejidad temporal y espacial de soluciones de programación dinámica.
- Aplicar programación dinámica a problemas de optimización.

Introducción a la Programación Dinámica

Definición

La **programación dinámica** es una técnica de diseño de algoritmos que resuelve problemas complejos dividiéndolos en subproblemas más simples, almacenando los resultados de estos subproblemas para evitar recalcularlos.

Características Clave

1. **Subestructura óptima:** Una solución óptima contiene soluciones óptimas de subproblemas.
2. **Subproblemas superpuestos:** Los mismos subproblemas aparecen repetidamente.
3. **Memorización:** Almacenar resultados para reutilizarlos.

Principio de Optimalidad de Bellman

[Principio de Optimalidad] Una política óptima tiene la propiedad de que, cualquiera sea el estado inicial y la decisión inicial, las decisiones restantes deben constituir una política óptima con respecto al estado resultante de la primera decisión.

Enfoques de Programación Dinámica

[scale=0.7] (0,1) rectangle (4,2); at (2,1.5) Top-down (Memoización); at (2,0.5) Recursivo + almacenamiento; at (2,0) Procesa solo subproblemas necesarios;
(6,1) rectangle (10,2); at (8,1.5) Bottom-up (Tabular); at (8,0.5) Iterativo + tabla; at (8,0) Procesa todos los subproblemas;

Problema de la Mochila (0/1)

Descripción del Problema

Dados n objetos, cada uno con peso w_i y valor v_i, y una mochila con capacidad W, seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.

Ecuación de Recurrencia

Sea $dp[i][j]$ el valor máximo que se puede obtener con los primeros i objetos y capacidad j:

$$dp[i][j] = \max(dp[i-1][j],\ dp[i-1][j-w_i] + v_i) \quad sij \geq w_i$$

$$dp[i][j] = dp[i-1][j] \quad sij < w_i$$

Caso base: $dp[0][j] = 0$ para todo j.

Implementación Bottom-up

[language=C, caption=Mochila 0/1 - Bottom-up] include ¡stdio.h¿ include ¡string.h¿

define MAX$_N$100$defineMAX_W$1000

int mochila01(int n, int W, int pesos[], int valores[]) int dp[MAX$_N$ + 1][$MAX_W + 1$];

// Inicializar casos base for (int j = 0; j ¡= W; j++) dp[0][j] = 0;

// Llenar tabla for (int i = 1; i ¡= n; i++) for (int j = 0; j ¡= W; j++) if (j ¿= pesos[i-1]) int tomar = dp[i-1][j - pesos[i-1]] + valores[i-1]; int noTomar = dp[i-1][j]; dp[i][j] = (tomar ¿ noTomar) ? tomar : noTomar; else dp[i][j] = dp[i-1][j];

return dp[n][W];

Optimización de Espacio

[language=C, caption=Mochila 0/1 - Optimización de espacio] int mochila01Optimizada(int n, int W, int pesos[], int valores[]) int dp[MAX$_W$ + 1];

// Inicializar memset(dp, 0, sizeof(dp));

// Llenar tabla (recorriendo W hacia atrás) for (int i = 0; i ¡ n; i++) for (int j = W; j ¿= pesos[i]; j–) int tomar = dp[j - pesos[i]] + valores[i]; if (tomar ¿ dp[j]) dp[j] = tomar;

return dp[W];

Ejemplo Paso a Paso

$n = 4$, $W = 7$, objetos: (2,3), (3,4), (4,5), (5,6)

i/j	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7
3	0	0	3	4	5	7	8	9
4	0	0	3	4	5	7	8	9

Valor máximo: 9

Distancia de Edición (Levenshtein)

Descripción del Problema

Dadas dos cadenas A y B, encontrar el número mínimo de operaciones (inserción, eliminación, sustitución) para transformar A en B.

Ecuación de Recurrencia

Sea $dp[i][j]$ la distancia de edición entre los primeros i caracteres de A y los primeros j caracteres de B:

$$dp[i][j] = \min \{ d\ p[i-1][j]+1(eliminación)dp[i][j-1]+1(inserción)dp[i-1][j-1]+costo(susti$$

donde $costo = 0$ si $A[i-1] = B[j-1]$, $costo = 1$ en caso contrario.

Caso base: $dp[i][0] = i$, $dp[0][j] = j$.

Implementación

[language=C, caption=Distancia de Levenshtein] include ¡stdio.h¿ include ¡string.h¿ include ¡stdlib.h¿

int min(int a, int b, int c) int m = a; if (b ¡ m) m = b; if (c ¡ m) m = c; return m;

int distanciaEdicion(char* A, char* B) int m = strlen(A); int n = strlen(B);

// Crear tabla int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i ¡= m; i++) dp[i] = (int*)malloc((n + 1) * sizeof(int));

// Casos base for (int i = 0; i ¡= m; i++) dp[i][0] = i; for (int j = 0; j ¡= n; j++) dp[0][j] = j;

// Llenar tabla for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) int costo = (A[i-1] == B[j-1]) ? 0 : 1; dp[i][j] = min(dp[i-1][j] + 1, // eliminación dp[i][j-1] + 1, // inserción dp[i-1][j-1] + costo // sustitución);

int resultado = dp[m][n];

// Liberar memoria for (int i = 0; i ¡= m; i++) free(dp[i]); free(dp);

return resultado;

Ejemplo

Transformar "CASA" en "PERRO"

		C	A	S	A
	0	1	2	3	4
P	1	1	2	3	4
E	2	2	2	3	4
R	3	3	3	3	4
R	4	4	4	4	4
O	5	5	5	5	5

Distancia: 5

Subsecuencia Común Más Larga (LCS)

Descripción del Problema

Dadas dos secuencias X e Y, encontrar la subsecuencia (no necesariamente contigua) más larga que aparece en ambas.

Ecuación de Recurrencia

Sea $dp[i][j]$ la longitud de la LCS entre los primeros i caracteres de X y los primeros j caracteres de Y:

$dp[i][j] = \{ 0\ sii = 0oj = 0dp[i-1][j-1]+1siX[i-1] = Y[j-1]\max(dp[i-1][j], dp[i][j-1])encasocontr$

Implementación

[language=C, caption=LCS] include ¡stdio.h¿ include ¡string.h¿ include ¡stdlib.h¿
int max(int a, int b) return (a ¿ b) ? a : b;
int lcs(char* X, char* Y) int m = strlen(X); int n = strlen(Y);
int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i ¡= m; i++) dp[i] = (int*)calloc(n + 1, sizeof(int));
for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) if (X[i-1] == Y[j-1]) dp[i][j] = dp[i-1][j-1] + 1; else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
int resultado = dp[m][n];
// Liberar memoria for (int i = 0; i ¡= m; i++) free(dp[i]); free(dp);
return resultado;

Recuperación de la Subsecuencia

[language=C, caption=Recuperar LCS] void recuperarLCS(char* X, char* Y, int** dp, int i, int j, char* resultado, int* idx) if (i == 0 —— j == 0) return;

if (X[i-1] == Y[j-1]) recuperarLCS(X, Y, dp, i-1, j-1, resultado, idx); resultado[(*idx)++] = X[i-1]; else if (dp[i-1][j] ¿ dp[i][j-1]) recuperarLCS(X, Y, dp, i-1, j, resultado, idx); else recuperarLCS(X, Y, dp, i, j-1, resultado, idx);

Ejemplo

$X =$ "$ABCBDAB$", $Y =$ "$BDCABA$"

		A	B	C	B	D	A
B							
	0	0	0	0	0	0	0
0							
B	0	0	1	1	1	1	1
1							
D	0	0	1	1	1	2	2
2							
C	0	0	1	2	2	2	2
2							
A	0	1	1	2	2	2	3
3							
B	0	1	2	2	3	3	3
4							
A	0	1	2	2	3	3	4
4							

LCS: "BCAB" o "BDAB", longitud 4.

Optimización de Espacio para LCS

[language=C, caption=LCS optimizado] int lcsOptimizado(char* X, char* Y) int m = strlen(X); int n = strlen(Y);

// Usar solo 2 filas int** dp = (int**)malloc(2 * sizeof(int*)); dp[0] = (int*)calloc(n + 1, sizeof(int)); dp[1] = (int*)calloc(n + 1, sizeof(int));

for (int i = 1; i ¡= m; i++) for (int j = 1; j ¡= n; j++) if (X[i-1] == Y[j-1]) dp[i else dp[i

int resultado = dp[m free(dp[0]); free(dp[1]); free(dp);

return resultado;

Comparación de Enfoques

Problema	Tiempo	Espacio	Espacio optimizado
Mochila (0/1)	$O(nW)$	$O(nW)$	$O(W)$
Distancia de edición	$O(mn)$	$O(mn)$	$O(\min(m,n))$
LCS	$O(mn)$	$O(mn)$	$O(\min(m,n))$

Ejercicios Resueltos (20 Ejercicios)

Resuelva la mochila 0/1 con $n = 3$, $W = 5$, objetos: (2,3), (3,4), (4,5).

```
dp[0][*]=0
i=1: j=2: dp=3; j=3,4,5: dp=3
i=2: j=3: dp=4; j=5: dp=3+4=7
i=3: j=4: dp=5; j=5: dp=5 vs 7 -> 7
Valor máximo: 7
```

Calcule la distancia de edición entre "gato" y "perro".

```
gato -> perro: 4 sustituciones (g->p, a->e, t->r, o->r?) +1 inserción = 5
```

Encuentre LCS de "ABCD" y "ACBD". LCS: "ABD" o "ACD", longitud 3.

Ejercicios Propuestos (20 Ejercicios)

1. Resuelva mochila 0/1 con n=4, W=10, objetos (2,5),(3,7),(4,8),(5,9).
2. Implemente distancia de edición con operaciones de costo variable.
3. Encuentre LCS de "XMJYAUZ" y "MZJAWXU".
4. ¿Qué es el principio de optimalidad de Bellman?
5. Implemente mochila 0/1 con recuperación de objetos seleccionados.
6. ¿Cuál es la diferencia entre top-down y bottom-up?
7. Resuelva el problema de la subsecuencia creciente máxima con PD.
8. Implemente distancia de edición con recuperación de operaciones.

9. ¿Qué es la matriz de costos en LCS?
10. Resuelva el problema de la partición con PD.
11. Implemente LCS con recuperación de todas las soluciones.
12. ¿Cómo se puede mejorar la complejidad espacial de distancia de edición?
13. Resuelva el problema del cambio de monedas con PD.
14. Implemente mochila 0/1 con objetos fraccionarios? (no es PD)
15. ¿Qué es la subestructura óptima?
16. Resuelva el problema del viajante con PD (TSP).
17. Implemente la multiplicación de matrices con PD.
18. ¿Cuándo usar programación dinámica vs greedy?
19. Resuelva el problema de la distancia de Damerau-Levenshtein.
20. Investigue aplicaciones de PD en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El principio de optimalidad de Bellman.
- La diferencia entre enfoques top-down y bottom-up.
- La implementación de tres problemas clásicos: mochila 0/1, distancia de edición y LCS.
- Técnicas de optimización de espacio.
- El análisis de complejidad de soluciones de programación dinámica.

Bibliografía de la Unidad

- Bellman, R. (1957). *Dynamic Programming.* Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 15.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.

Unidad XVII: Apéndice - Fórmulas, Recursos y Apoyo Académico

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante tendrá acceso a:

- Un compendio de fórmulas y conceptos clave de la materia.
- Una bibliografía completa y actualizada.
- Recursos digitales (canales de YouTube, sitios web, blogs).
- Información de contacto para apoyo académico personalizado.

Compendio de Fórmulas y Conceptos Clave

Complejidad Computacional

- **Clase P:** Problemas resolubles en tiempo polinomial.
- **Clase NP:** Problemas verificables en tiempo polinomial.
- **NP-completo:** $L \in NP$ y todo $L' \in NP$ se reduce a L.
- **NP-hard:** Todo $L' \in NP$ se reduce a L (no necesita estar en NP).
- **Reducción polinomial:** $A \leq_p B$ si existe f polinomial tal que $x \in A \iff f(x) \in B$.

Algoritmos de Caminos Mínimos

- **Dijkstra:** $O(E \log V)$ con montículo. Requiere pesos no negativos.
- **Bellman-Ford:** $O(VE)$. Permite pesos negativos. Detecta ciclos negativos.
- **Floyd-Warshall:** $O(V^3)$. Todos los pares.

Flujo en Redes

- **Teorema de Flujo Máximo - Corte Mínimo:** $\max |f| = \min c(S, T)$
- **Ford-Fulkerson:** $O(E \cdot |f^*|)$
- **Edmonds-Karp:** $O(VE^2)$

Algoritmos de String

- **KMP:** $O(n + m)$. Función de prefijo π.
- **Boyer-Moore:** $O(n/m)$ promedio. Heurísticas de mal carácter y buen sufijo.

Aritmética Modular y RSA

- **Euclides:** $\gcd(a, b) = \gcd(b, a \bmod b)$
- **Euclides extendido:** $ax + by = \gcd(a, b)$
- **Teorema de Fermat:** $a^{p-1} \equiv 1 \pmod{p}$ si p primo y $\gcd(a, p) = 1$
- **Teorema de Euler:** $a^{\phi(n)} \equiv 1 \pmod{n}$ si $\gcd(a, n) = 1$
- **RSA:** $c = m^e \bmod n$, $m = c^d \bmod n$, $ed \equiv 1 \pmod{\phi(n)}$

Programación Dinámica

- **Mochila 0/1:** $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - w_i] + v_i)$
- **Distancia de edición:** $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + cost)$
- **LCS:** $dp[i][j] = \{\, d\, p[i-1][j-1] + 1 si X[i] = Y[j]$
  $\max(dp[i-1][j], dp[i][j-1]) cc$

Backtracking

- Esquema general: recursión + validación + vuelta atrás.
- Poda por optimalidad, factibilidad y simetría.

Algoritmos Aproximados

- **Vertex Cover:** factor 2
- **TSP métrico:** factor 2 (vía MST)
- **KNAPSACK:** FPTAS con escalado de valores

Bibliografía Recomendada

Bibliografía Principal

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
  - **Comentario:** La "biblia" de los algoritmos. Cubre todos los temas del programa con rigor matemático.
  - **Capítulos clave:** 34 (NP-Completitud), 24-26 (Grafos), 32 (Strings), 31 (Números), 15 (PD), 35 (Aproximación).
- **Sedgewick, R., & Wayne, K. (2011).** *Algorithms* (4th ed.). Addison-Wesley.
  - **Comentario:** Enfoque práctico con implementaciones en Java. Excelente para visualización.
- **Kleinberg, J., & Tardos, É. (2006).** *Algorithm Design.* Addison-Wesley.
  - **Comentario:** Excelente para diseño de algoritmos y problemas de optimización.

Bibliografía de Complejidad

- **Arora, S., & Barak, B. (2009).** *Computational Complexity: A Modern Approach.* Cambridge University Press.

- **Garey, M. R., & Johnson, D. S. (1979).** *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W.H. Freeman.
- **Sipser, M. (2012).** *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

Bibliografía de Grafos

- **Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993).** *Network Flows: Theory, Algorithms, and Applications.* Prentice Hall.
- **Even, S. (2011).** *Graph Algorithms* (2nd ed.). Cambridge University Press.

Bibliografía de Criptografía y Números

- **Stinson, D. R. (2005).** *Cryptography: Theory and Practice* (3rd ed.). Chapman & Hall/CRC.
- **Rosen, K. H. (2010).** *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.

Bibliografía de Metaheurísticas

- **Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983).** Optimization by simulated annealing. *Science*, 220(4598), 671-680.
- **Holland, J. H. (1992).** *Adaptation in Natural and Artificial Systems.* MIT Press.
- **Dorigo, M., & Stützle, T. (2004).** *Ant Colony Optimization.* MIT Press.

Canales de YouTube Recomendados

Canales en Español

- **Mitos y Leyendas de la Programación (Andrés)**
  - **URL:** youtube.com/c/MitosyLeyendasdelaProgramacion

  - **Contenido:** Algoritmos de grafos, NP-completitud, programación dinámica.

- **Deusto Ciencia**
  - **URL:** youtube.com/c/DeustoCiencia
  - **Contenido:** Complejidad computacional, teoría de la computación.

- **Algoritmos y Programación (UBA)**
  - **URL:** youtube.com/c/AlgoritmosyProgramacionUBA
  - **Contenido:** Material de las cátedras de Exactas.

Canales en Inglés

- **Abdul Bari**
  - **URL:** youtube.com/c/AbdulBari
  - **Contenido:** Algoritmos de grafos, programación dinámica, NP-completitud.

- **Reducible**
  - **URL:** youtube.com/c/Reducible
  - **Contenido:** Animaciones de alta calidad sobre algoritmos complejos.

- **Back To Back SWE**
  - **URL:** youtube.com/c/BackToBackSWE
  - **Contenido:** Preparación para entrevistas, problemas de algoritmos.

- **Computerphile**
  - **URL:** youtube.com/c/computerphile
  - **Contenido:** P vs NP, algoritmos de string, criptografía.

- **3Blue1Brown**
  - **URL:** youtube.com/c/3blue1brown
  - **Contenido:** Visualizaciones matemáticas útiles para complejidad.

Sitios Web y Blogs Recomendados

Sitios Educativos

- **VisuAlgo**
  - **URL:** visualgo.net
  - **Contenido:** Visualizaciones interactivas de algoritmos.
- **GeeksforGeeks**
  - **URL:** geeksforgeeks.org
  - **Contenido:** Artículos, implementaciones y ejercicios.
- **LeetCode**
  - **URL:** leetcode.com
  - **Contenido:** Práctica de problemas algorítmicos.
- **HackerRank**
  - **URL:** hackerrank.com
  - **Contenido:** Desafíos de algoritmos.
- **Coursera**
  - **URL:** coursera.org
  - **Contenido:** Cursos de algoritmos (Princeton, Stanford, MIT).
- **edX**
  - **URL:** edx.org
  - **Contenido:** Cursos de MIT y Harvard.

Blogs Especializados

- **CP-Algorithms**
  - **URL:** cp-algorithms.com
  - **Contenido:** Algoritmos avanzados y optimización.
- **Brilliant**
  - **URL:** brilliant.org

  - **Contenido:** Matemáticas y algoritmos interactivos.

- **The Algorithms**
  - **URL:** the-algorithms.com
  - **Contenido:** Implementaciones en múltiples lenguajes.

Foros y Comunidades

- **Stack Overflow**
  - **URL:** stackoverflow.com
  - **Contenido:** Consultas específicas sobre implementaciones.
- **Reddit - r/algorithms**
  - **URL:** reddit.com/r/algorithms
  - **Contenido:** Discusión sobre algoritmos.
- **Discord: Algoritmos UBA**
  - **Contenido:** Comunidad de estudiantes de Exactas.

Profesores Particulares Recomendados

Nahuel Villafañe - @clasesconnahu

[colback=blue!5!white, colframe=blue!75!black, title=**Nahuel Villafañe**]
Estudiante avanzado de Ciencias Fisica

Universidad de Buenos Aires (UBA) - Facultad de Ciencias Exactas y Naturales
Experiencia:

- Autor de los tratados *"Tratado de Algoritmos y Datos II"* y *"Tratado de Algoritmos y Datos III" y mucho otros mas*

Clases Particulares:

- **Especialidad:** Fisica y Matematicas
- **Modalidad:** Presencial (CABA) y virtual (Zoom, Google Meet)

Contacto:

- Instagram: `@clasesconnahu`

Otros Profesores Recomendados

- **Martín Diéguez (@martindieguez)** - Especialidad: Algoritmos y Programación (UBA)
- **Facundo Palavecino** - Especialidad: Matemática discreta y complejidad
- **Clases Particulares UBA** - clasesparticularesuba.com.ar
- **Superprof Argentina** - superprof.com.ar

Recursos Adicionales

Google Drive y Material Compartido

- **Apuntes de Cátedra:** Disponibles en el Departamento de Computación (dc.uba.ar)
- **Parciales Resueltos:** Grupos de Telegram y Discord de la materia
- **GitHub:** Buscar repositorios de algoritmos con implementaciones

Grupos de Estudio

- **Telegram:** "Algo2 - Algo3 UBA" (buscar en el chat de la facultad)
- **Discord:** Servidor de Exactas UBA
- **WhatsApp:** Grupos por comisión (consultar en la cursada)

Consejos Finales para el Estudio

1. **No memorices, comprendé:** Los algoritmos son lógicos. Entendé por qué funcionan.
2. **Practicá con papel y lápiz:** Ejecutar algoritmos a mano ayuda a internalizarlos.

[width=0.5]Logo$_e xactas.jpg$

3. **Implementá todo:** La programación refuerza la comprensión.
4. **Estudiá en grupo:** Explicar a otros es la mejor forma de aprender.
5. **Hacé simulacros de parcial:** Con tiempo limitado para practicar presión.
6. **Consultá las dudas a tiempo:** No dejes que se acumulen.

Epílogo

Querido estudiante,

Has llegado al final de este tratado. Los temas que has recorrido —desde la complejidad computacional hasta los algoritmos aproximados— son el corazón de la computación teórica y práctica.

No te desanimes si algunos conceptos parecen difíciles al principio. La belleza de los algoritmos se revela con la práctica y la paciencia. Cada problema resuelto es un paso más en tu formación.

Este tratado fue escrito con la esperanza de acompañarte en ese camino. Si algo no quedó claro, si necesitas profundizar en algún tema, o simplemente querés compartir tus avances, no dudes en contactarme a través de las redes.

Que la complejidad esté siempre de tu lado, que tus invariantes sean correctos y que tus caminos sean siempre mínimos.

Con cariño,

Nahuel Villafañe

Buenos Aires, 2026

Colofón

Este documento fue compuesto en LaTeX utilizando la clase `book`. Las figuras fueron elaboradas con TikZ. La tipografía principal es Computer Modern.

"La ciencia de los algoritmos es la ciencia de lo computable."